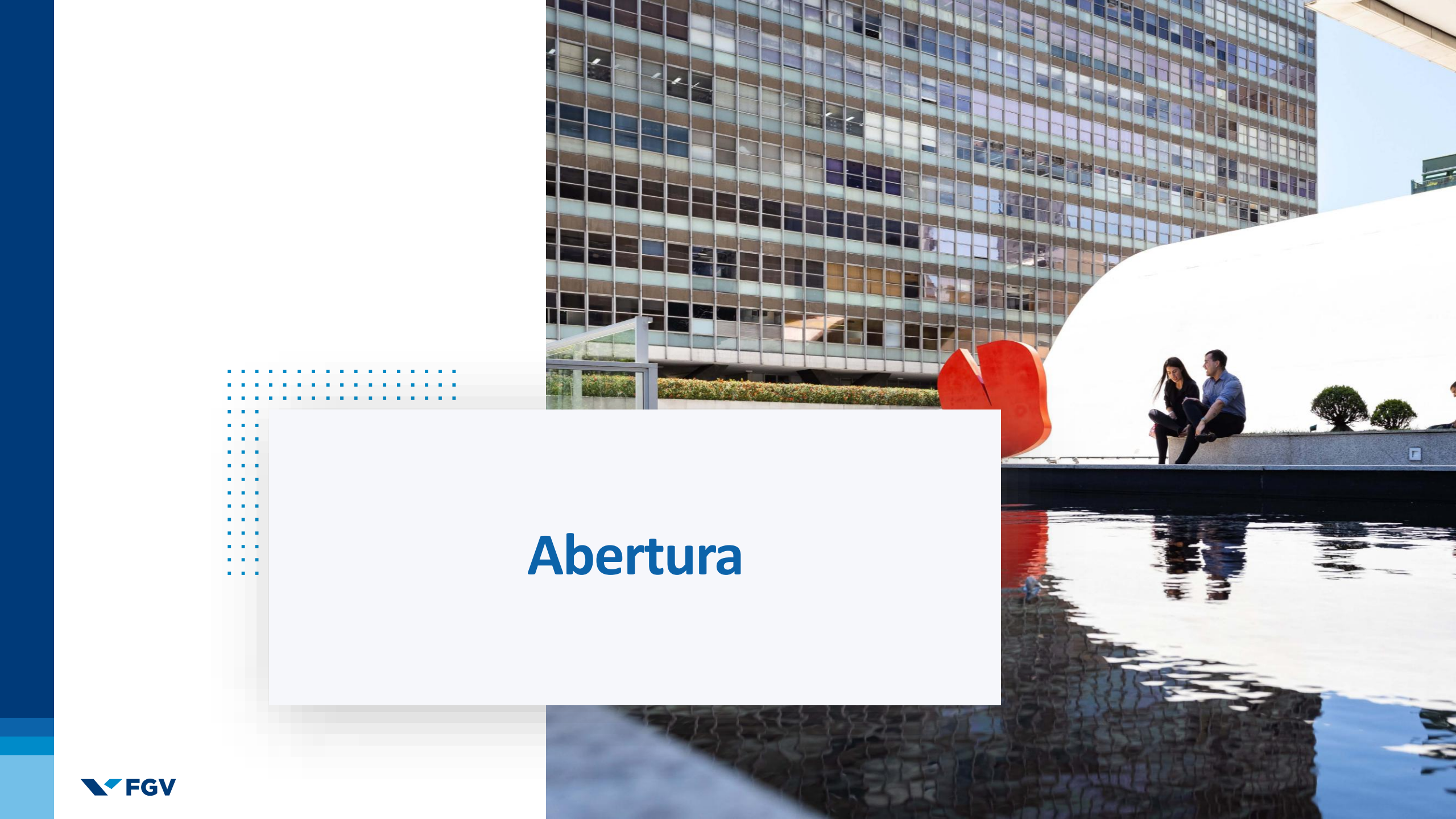

Modelagem Preditiva

Roberto Pina Rizzo, MSC, PMP, PMI-ACP, SAFe 6 Agilist





Abertura

Objetivos do curso

- Rever / aprofundar o ciclo de execução dos projetos analíticos.
- Recapitular as principais modelagens preditivas de Ciência de Dados.
- Conhecer os princípios da Engenharia de Variáveis (*Feature Engineering*).
- Recapitular / aprofundar os processos de construção e validação de modelagens preditivas.
- Entender como ocorre a consolidação das modelagens (*ensemble*) para uma predição mais confiável.



Orientações gerais

- Havendo dúvidas ou curiosidades, levante a mão. Faremos pausas estratégicas para responder perguntas e realizar discussões.
- Evite se distrair com o celular / computador.
- Procure participar ativamente das reflexões e atividades.
- Nas sessões online, mantenha o microfone desligado quando não for falar, para evitar que sons do ambiente interfiram na aula.



Conteúdo do curso

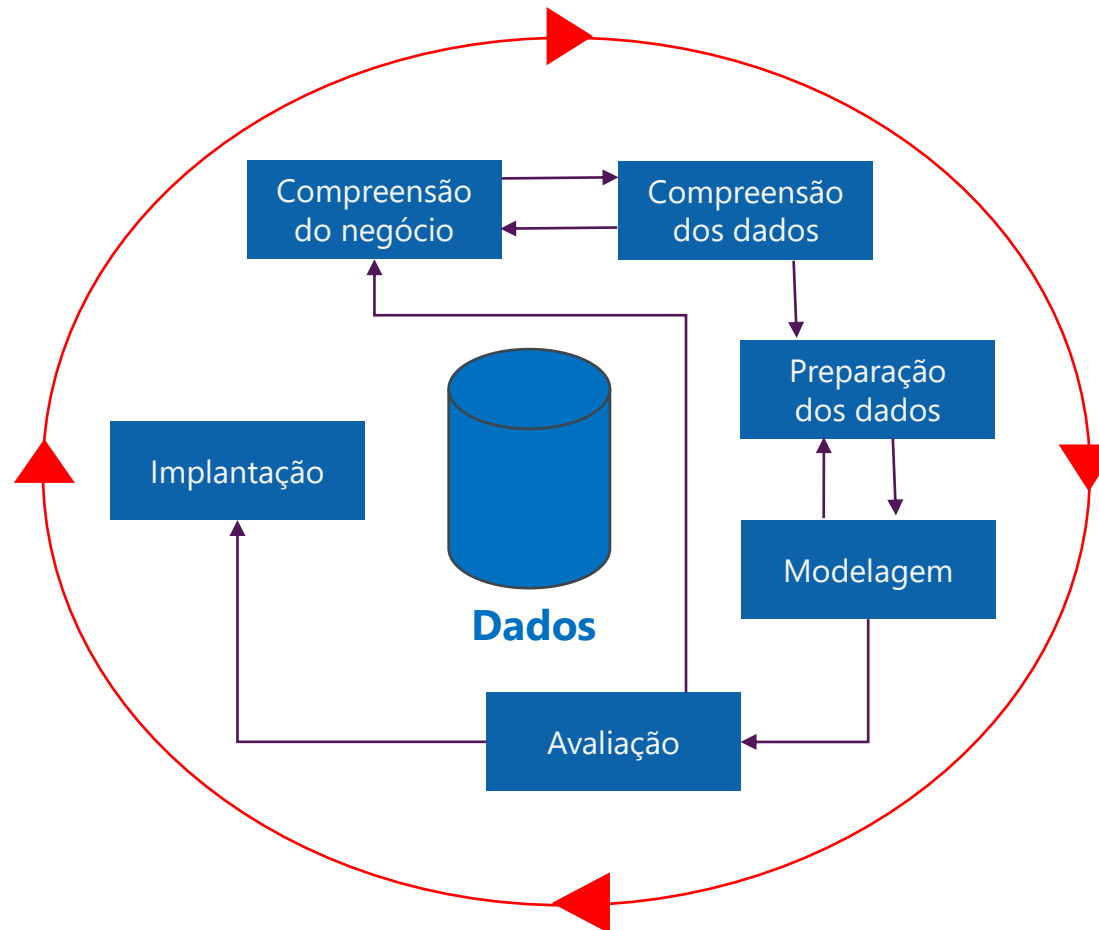
- **Módulo 1** – Ciclo de execução dos projetos analíticos
- **Módulo 2** – Recapitulação – modelagens preditivas de Ciência de Dados
- **Módulo 3** – Engenharia de Variáveis (*Feature Engineering*)
- **Módulo 4** – Construção e validação de modelagens
- **Módulo 5** – Consolidação de modelagens preditivas (*Ensemble*)



**Módulo 1 – Ciclo de
execução dos projetos
analíticos**

Ciclo de execução de projetos analíticos

- O processo de mineração de dados, para encontrar padrões e construir modelos, envolve repetições.



Trata-se de algo bem mais investigativo e experimental do que um desenvolvimento de software.

Ciclo de execução de projetos analíticos

- **Compreensão do negócio**
 - O problema a ser resolvido deve ser formulado, muitas vezes de forma iterativa (processo de exploração e descoberta).
 - Aqui, doses de arte e criatividade são necessárias.
 - Precisamos chegar na tradução do problema de negócio para um problema analítico.



Ciclo de execução de projetos analíticos

- **Compreensão do negócio – *cont.***
 - **Exemplo:**
 - **Pergunta de negócio:** "Como reduzir a perda de clientes?"
 - **Pergunta analítica:** "Quais clientes apresentam maior probabilidade de cancelar nos próximos 90 dias?"
 - **Variável-alvo:** churn = 1 se cancelar nos próximos 90 dias; 0 caso contrário.
 - **Ação de negócio:** priorizar ações de retenção para clientes de alto risco de cancelamento.



Ciclo de execução de projetos analíticos

- **Compreensão dos dados**
 - Raramente os dados têm uma correspondência exata com o problema.
 - Alguns dados podem vir de graça, outros precisam ser obtidos e até comprados.
 - Poderá haver necessidade de realizar diferentes tipos de tarefas de mineração de dados, combinando suas soluções.



Ciclo de execução de projetos analíticos

- Preparação dos dados
 - Exemplos:
 - Organização em forma tabular, com remoção de valores ou inferência de valores ausentes.
 - Conversões de dados para diferentes tipos.
 - Normalização, para nivelar as escalas dos valores.
 - Outras técnicas de preparação.
 - **Aqui entra a Engenharia de Variáveis (*Feature Engineering*) que veremos na disciplina.**



Ciclo de execução de projetos analíticos

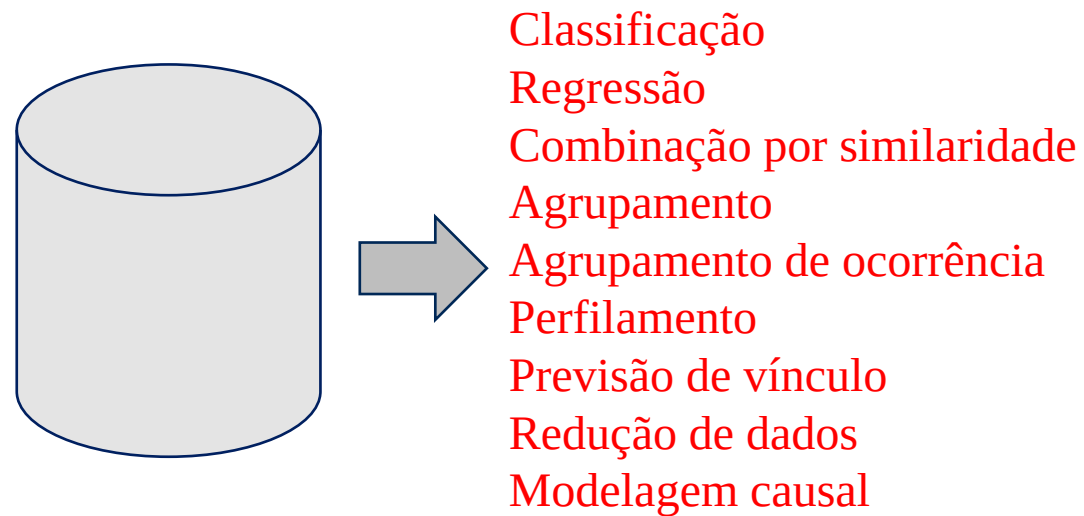
- Preparação dos dados – *cont.*
 - Preparação Inclui:
 - Tratamento de ausências
 - Codificação de variáveis categóricas
 - Transformação de variáveis
 - Normalização/padronização quando necessária
 - Criação de novas variáveis
 - Seleção de variáveis
 - Redução de dimensionalidade (veremos o PCA em *Feature Engineering*)
 - Balanceamento de classes, quando aplicável



Ciclo de execução de projetos analíticos

- Modelagem

- É a fase onde as técnicas de mineração são aplicadas aos dados.



Ciclo de execução de projetos analíticos

- Modelagem – *cont.*

Pergunta	Tipo
Cliente vai cancelar?	Classificação
Quanto venderemos?	Regressão
Quais clientes são semelhantes?	Clusterização

- Lembre-se: não se começa escolhendo o algoritmo - começa-se definindo o problema.



Ciclo de execução de projetos analíticos

- Modelagem – *cont.*
 - Aqui podemos aplicar diferentes modelagens, incluindo modalidades *ensemble* (combinadas), que veremos na disciplina.
 - Por exemplo, para uma classificação, podemos:
 - Aplicar árvore de segmentação por entropia / Gini.
 - Testar diferentes dados de treinamento / retenção e podas.
 - Aplicar também Regressão Logística, Random Forest e Gradient Boosting para comparar as predições.



Ciclo de execução de projetos analíticos

- **Avaliação**
 - É uma fase para averiguar se os resultados da mineração de dados são válidos e confiáveis (averiguação do desempenho do modelo).
 - Por exemplo, um modelo de detecção de fraudes pode estar produzindo muitos alarmes falsos.
 - Aqui também temos apuração e atenuação de sobreajustes.



Ciclo de execução de projetos analíticos

- Avaliação – *cont.*
 - Há indicadores de desempenho dos modelos, baseados na *matriz de confusão*.

		Classes de fato do exemplo	
		Positivo	Negativo
Classes previstas pelo modelo	Sim, é positivo	Verdadeiros positivo	Falsos positivo
	Não, é negativo	Falsos negativo	Verdadeiros negativo

- Quais indicadores considerar na avaliação dependem do cenário de negócio, como recapitularemos nesta disciplina.



Ciclo de execução de projetos analíticos

- **Implantação**
 - É a colocação no mundo real dos modelos produzidos por uma mineração de dados ou da própria mineração de dados em si.
 - Motivos para colocar a própria mineração em produção:
 - Adaptação às mudanças rápidas da realidade
 - Dificuldade em depuração manual de um modelo em um negócio
 - A implantação não encerra o projeto analítico.



Ciclo de execução de projetos analíticos

- Quem faz o quê

Etapa	Principal produto	Responsáveis
Compreensão do negócio	Problema e objetivos	Negócio + Data Scientist
Compreensão dos dados	Diagnóstico da base	Data Scientist / Data Engineer
Preparação	Dataset analítico	Data Scientist / Data Engineer
Modelagem	Modelos candidatos	Data Scientist
Avaliação	Modelo selecionado	Data Scientist + Negócio
Implantação	Modelo em produção	Data Scientist + Engenharia/TI
Monitoramento	Indicadores de desempenho	Data/ML + Negócio

**Módulo 2 – Recapitulação
– modelagens preditivas
de Ciência de Dados**

Modelagens preditivas - conceito

- Modelagem preditiva
 - É um conjunto de técnicas de **aprendizado supervisionado** que utiliza dados históricos **rotulados** para construir modelos capazes de prever, para novos indivíduos ou observações, uma **classe**, uma **probabilidade** de pertencimento a uma classe ou um valor numérico, com base nos padrões aprendidos durante o **treinamento**.
 - **Aprendizado supervisionado?**
 - **Dados rotulados?**
 - **Classes?**
 - **Probabilidades de classes?**
 - **Treinamento?**



Modelagens preditivas - conceito

- Vamos rever os principais modelos preditivos:
 - Classificação por árvore de decisão
 - Classificação por discriminante linear
 - Regressão
 - Regressão logística
 - Máquina vetorial de suporte



Modelagens preditivas - recapitulação

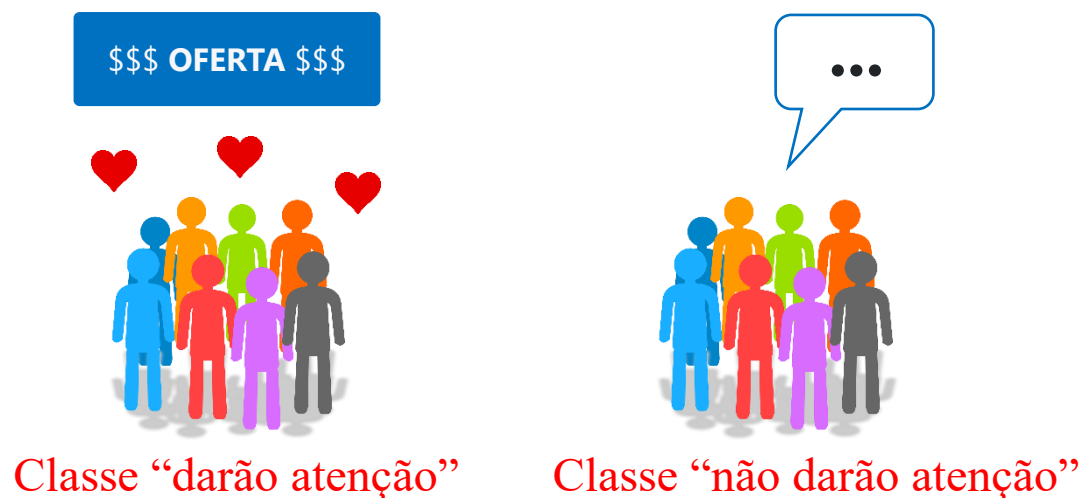
Classificação por árvore de decisão



Classificação - conceito

■ Classificação

- Tentativa de prever a que classe ou categoria cada indivíduo de uma população pertence.
- Tarefa intimamente relacionada: pontuação ou estimativa da probabilidade da classe.



Exemplo: “**Entre os clientes, quem dará atenção e quem não dará atenção a uma determinada oferta?**”

Indivíduo: entidade sobre a qual temos dados

Classificação por árvore de decisão

- Medida de impureza de segmento – **entropia**
 - Um segmento misto com muitos valores de variável alvo = “sim” e muitos com valor = “não” (ou vice-versa) possui muita desordem ou entropia alta.
 - Fórmula de entropia de Shannon:

$$\text{Entropia} = - p_1 \log_2(p_1) - p_2 \log_2(p_2) - \dots - p_n \log_2(p_n)$$

p_i = porcentagem relativa da propriedade i dentro do conjunto

$$0 \leq p_i \leq 1$$

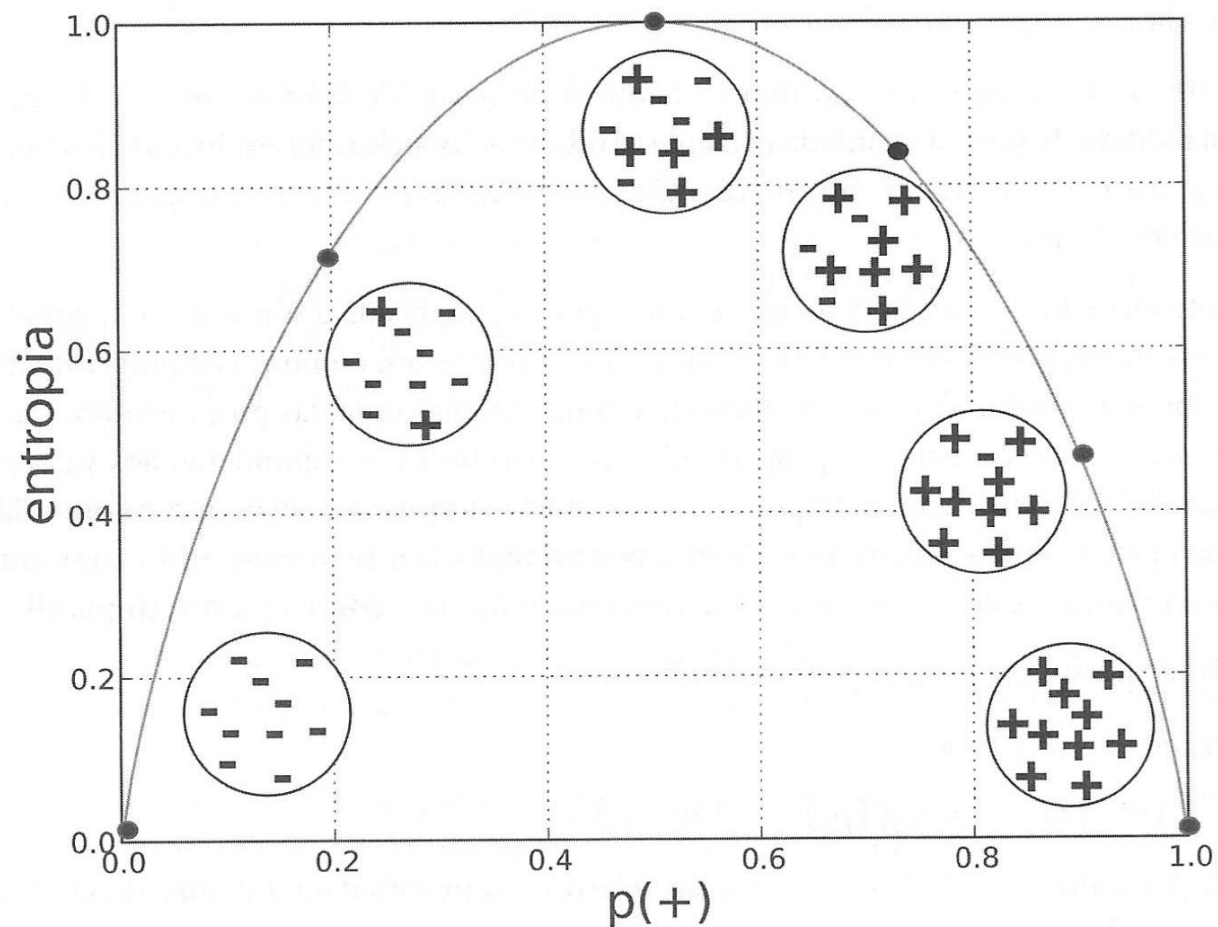
$p_i = 0 \Rightarrow$ nenhum membro do conjunto tem a propriedade i

$p_i = 1 \Rightarrow$ todos os membros do conjunto possuem a propriedade i



Classificação por árvore de decisão

- Medida de impureza de segmento – entropia



Entropia mínima (0):
Propriedades todas "–"
ou todas "+"

Entropia máxima (1):
50% das propriedades
"–" e 50% "+"

Classificação por árvore de decisão

- Exemplo de cálculo de entropia
 - Conjunto C de 10 pessoas, sendo 7 da classe “aprovadas” e 3 da classe “reprovadas”.
 - $p(\text{“aprovadas”}) = 7/10 = 0,7$
 - $p(\text{“reprovadas”}) = 3/10 = 0,3$

$$\begin{aligned}\text{Entropia}(C) &= - 0,7 \log_2(0,7) - 0,3 \log_2(0,3) \\ &= - [0,7(-0,51) + 0,3(-1,74)] \\ &= 0,88\end{aligned}$$



Classificação por árvore de decisão

- **Ganho de informação (GI)**

- É uma medida que nos diz quanto um atributo melhora (diminui) a entropia ao longo de toda a segmentação que ele cria.

$$\text{GI}(\text{pai}, \text{filhos}) = \text{entropia}(\text{pai}) - p(f_1) * \text{entropia}(f_1) - p(f_2) * \text{entropia}(f_2) - \dots - p(f_n) * \text{entropia}(f_n)$$

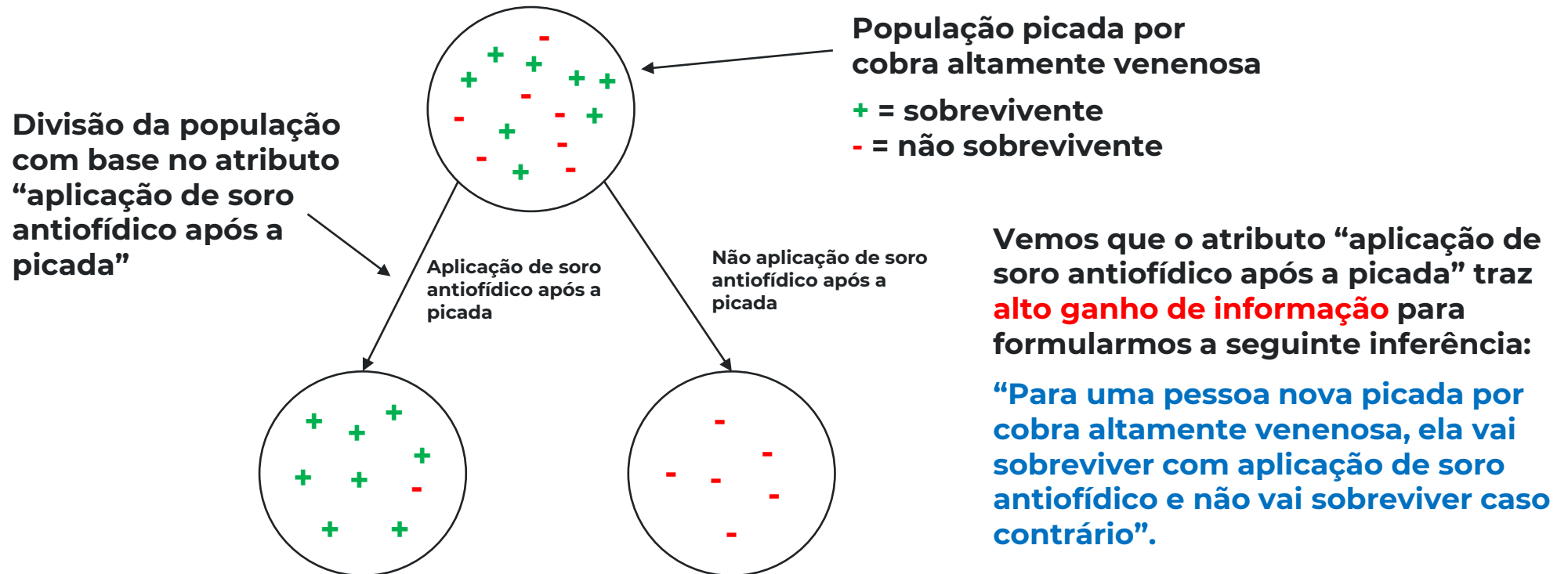
Conjunto pai = conjunto original de exemplos.

Conjuntos filhos = conjuntos resultados da divisão do conjunto pai em n subconjuntos após uma classificação com base em um atributo.



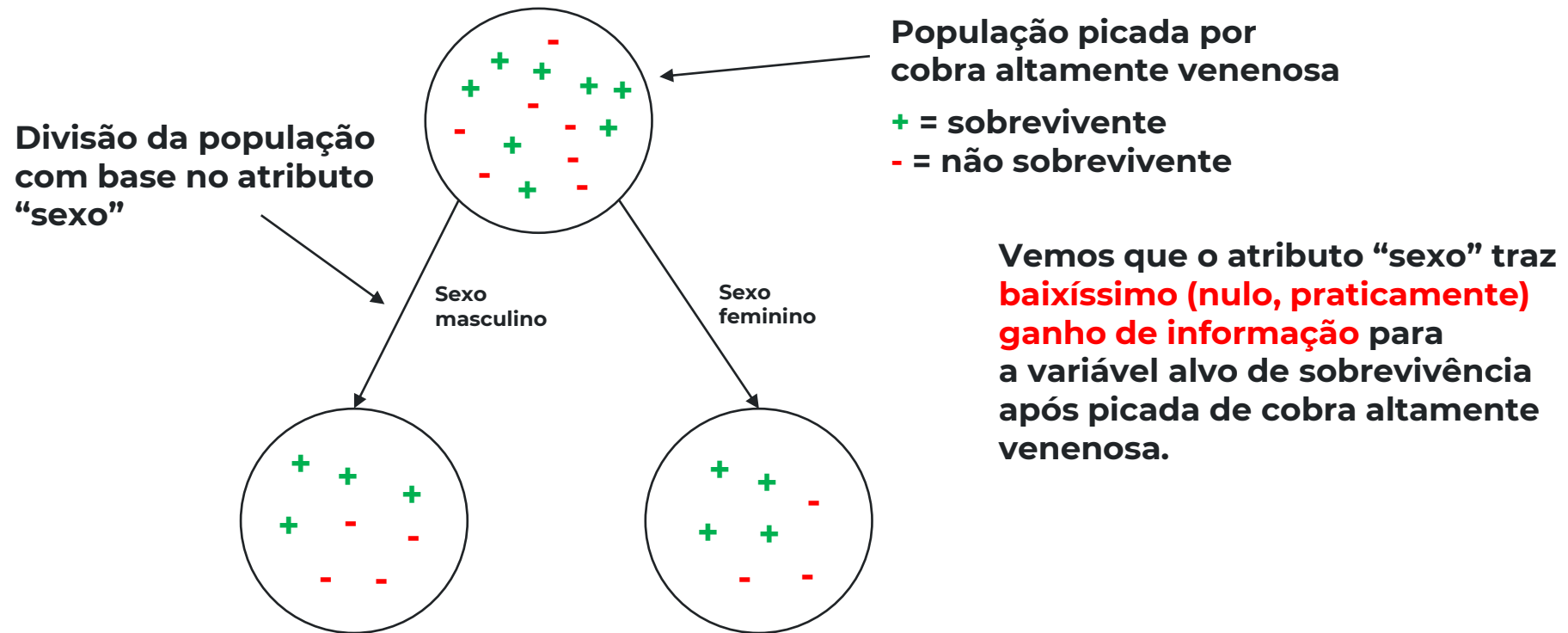
Classificação por árvore de decisão

- O ganho de informação de um atributo refere-se a quanto ele contribui para inferirmos uma previsão para novos exemplos.

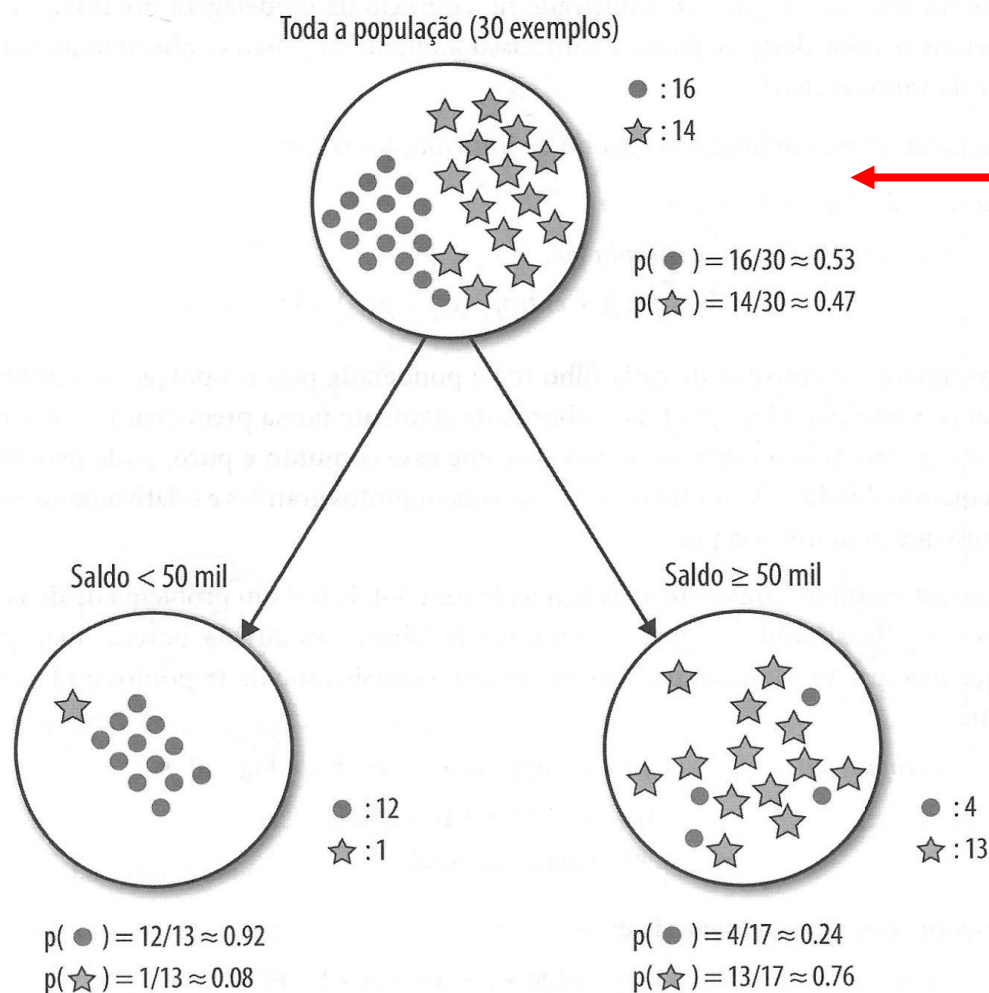


Classificação por árvore de decisão

- O ganho de informação de um atributo refere-se a quanto ele contribui para inferirmos uma previsão para novos exemplos.



Classificação por árvore de decisão



← **Conjunto pai**

Entropia do conjunto pai:

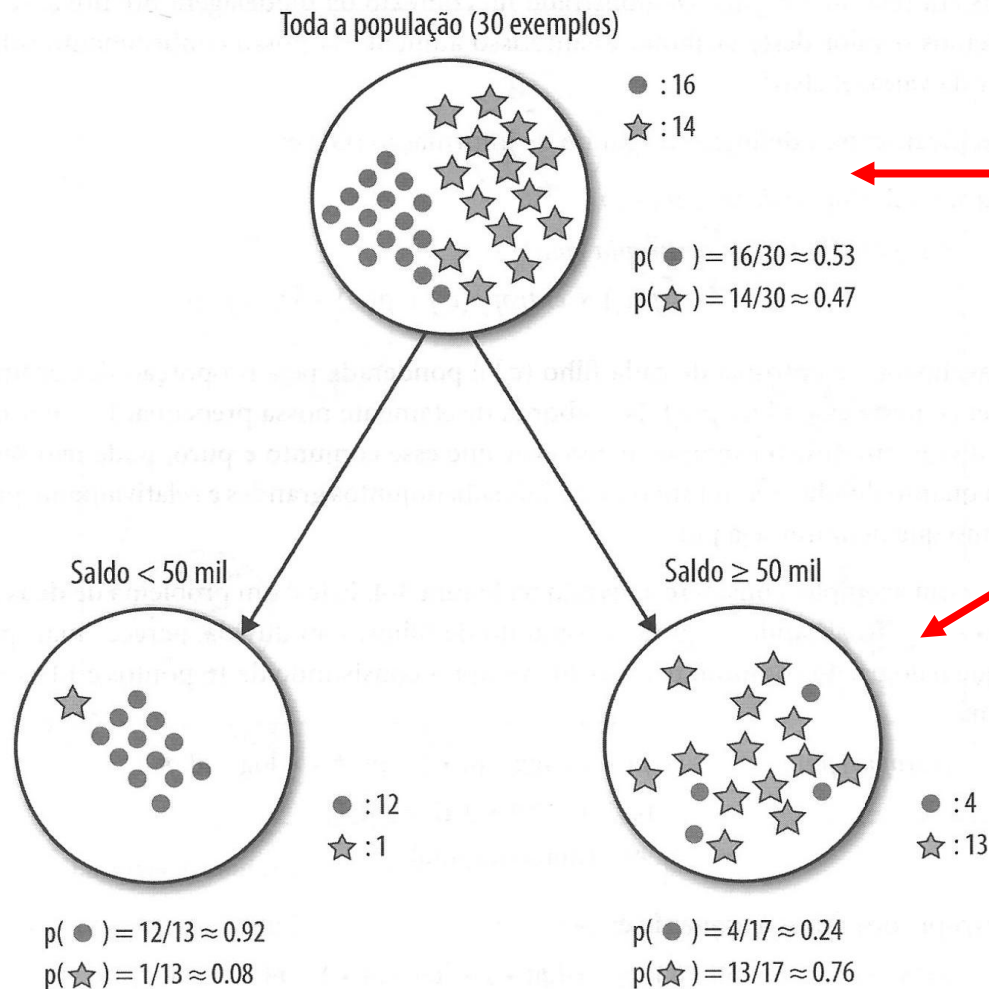
Entropia(pai)

$$= - [0,53 \log_2(0,53) + 0,47 \log_2(0,47)]$$

$$= - [0,53 (-0,9) + 0,47 (-1,1)]$$

$$= 0,99 \text{ (muito impuro)}$$

Classificação por árvore de decisão

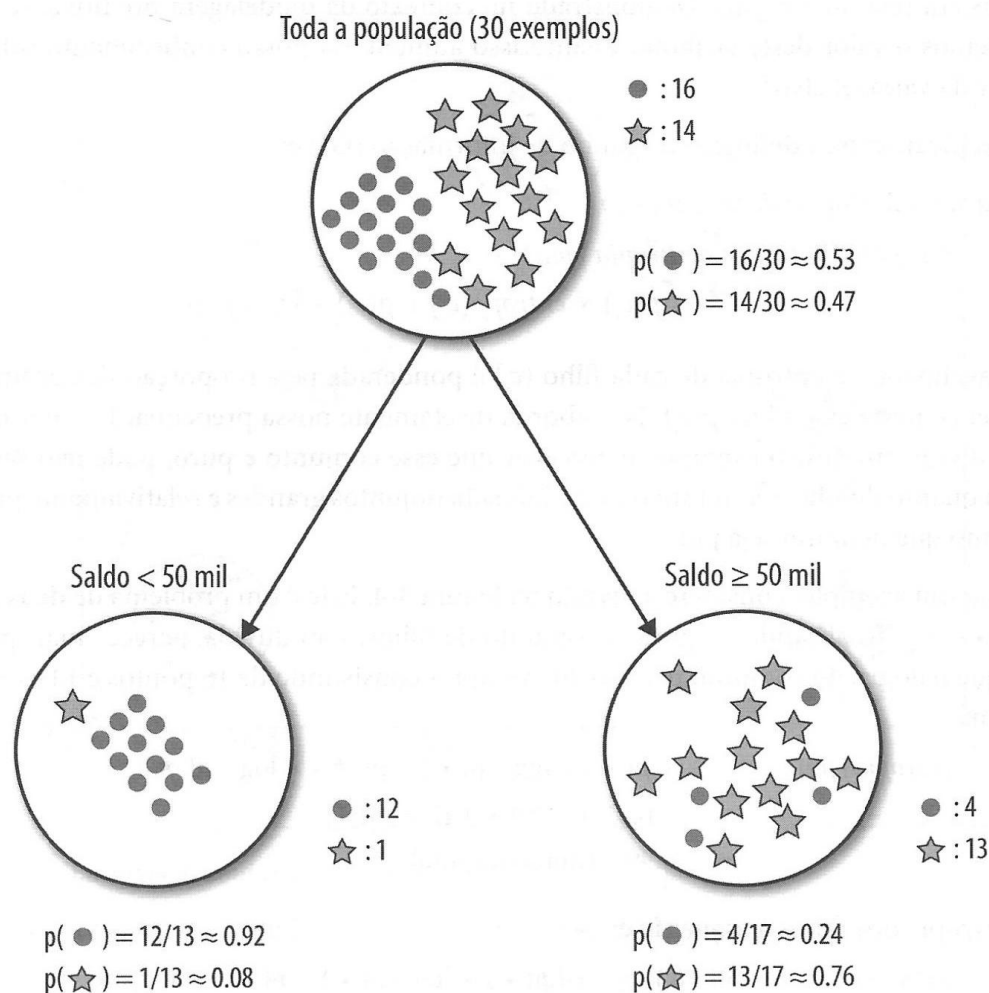


Conjunto pai
Entropia = 0,99

Conjuntos filhos
(divisão feita com base
no atributo “Saldo”)

Este atributo “Saldo”
traz ganho de informação
sobre o valor alvo (● ou ★)?
Ou seja, traz redução na
entropia?

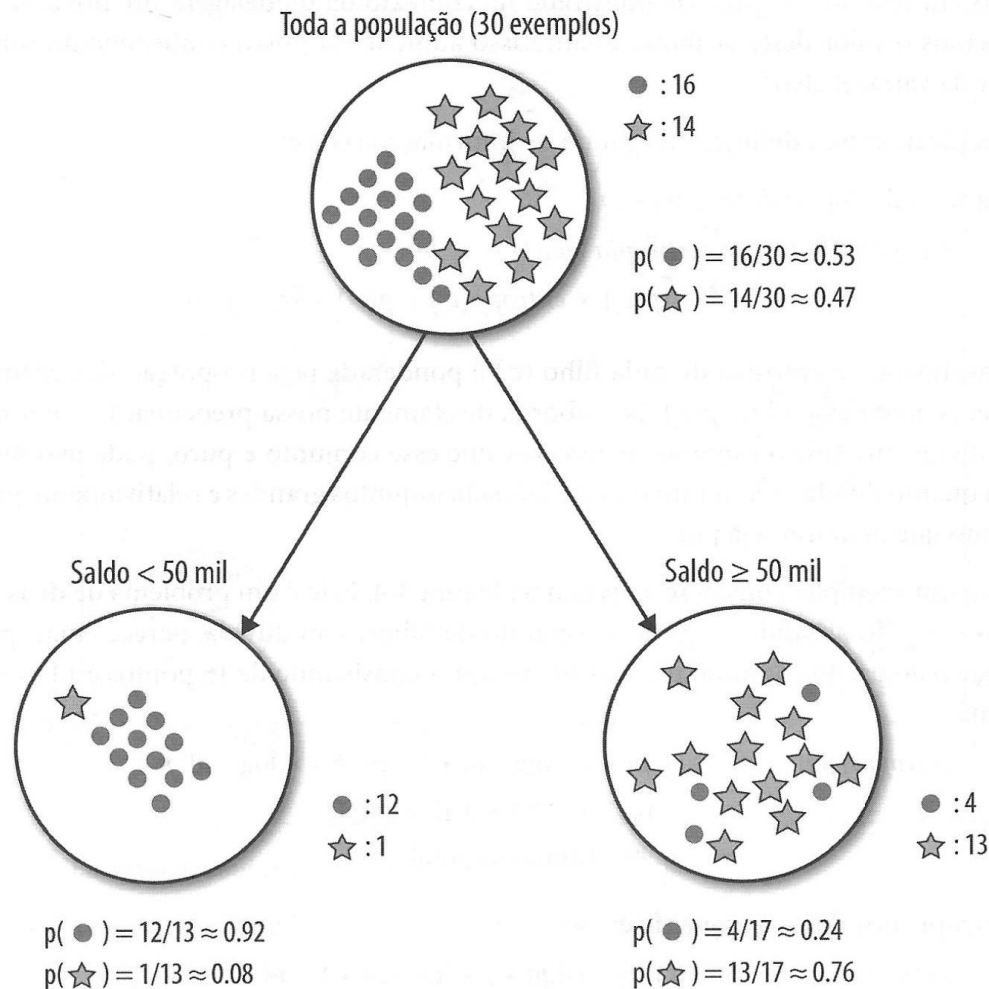
Classificação por árvore de decisão



Visualmente, podemos perceber que os conjuntos filhos são bem menos impuros

Calculemos o ganho de informação (GI)

Classificação por árvore de decisão



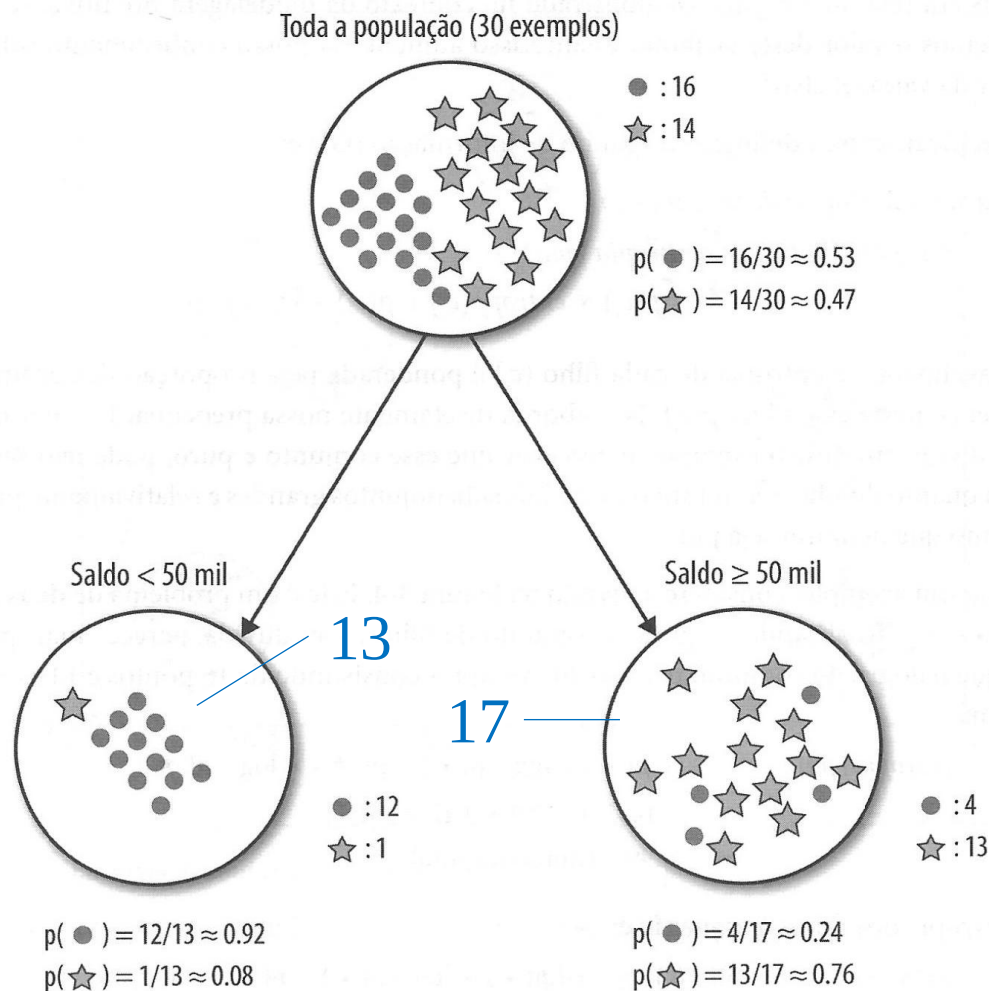
Entropia (Saldo < 50 mil)

$$= - [p(o) \log_2(o) + p(*) \log_2(*)]$$
$$= - [0,92 (-0,12) + 0,08 (-3,7)]$$
$$= 0,39$$

Entropia (Saldo $\geq$ 50 mil)

$$= - [p(o) \log_2(o) + p(*) \log_2(*)]$$
$$= - [0,24 (-2,1) + 0,76 (-0,39)]$$
$$= 0,79$$

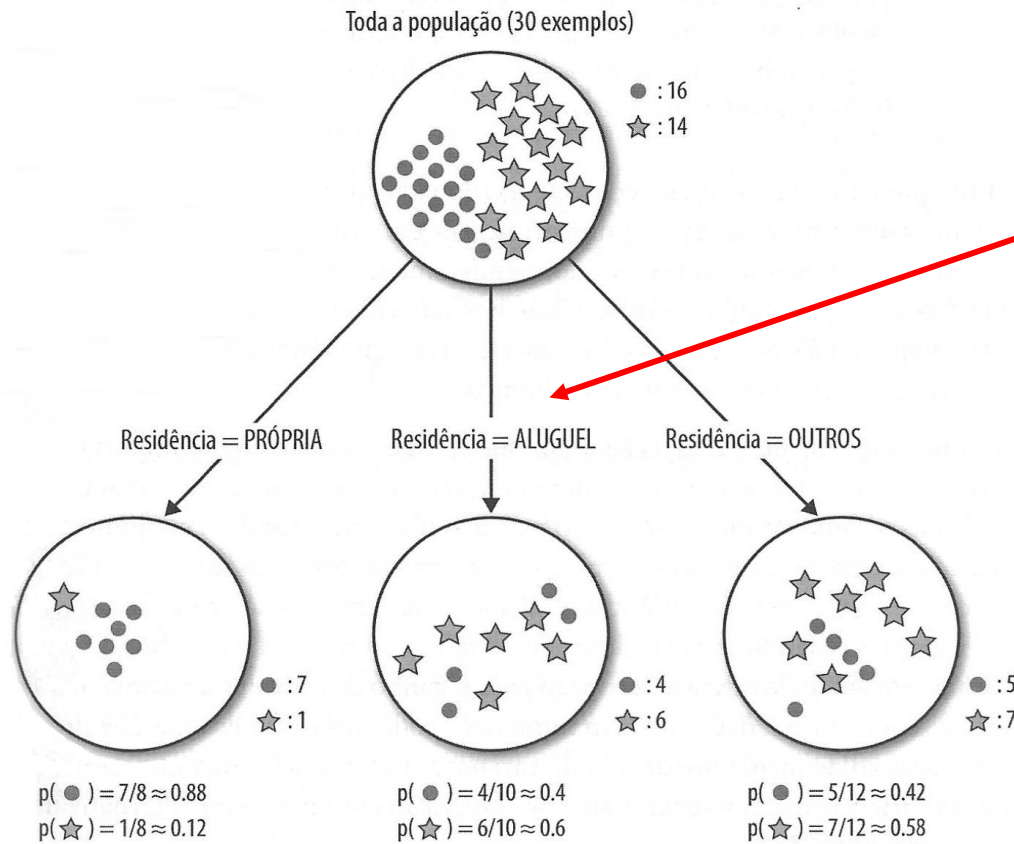
Classificação por árvore de decisão



Ganho de informação da divisão:

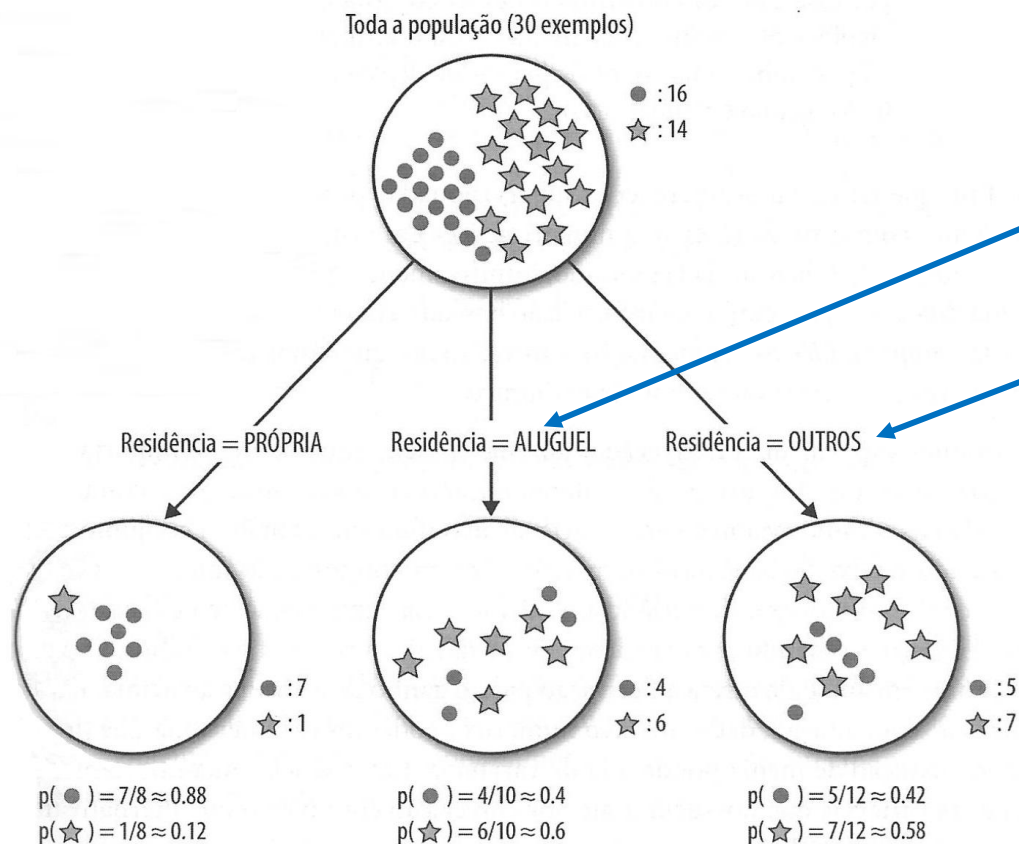
$$\begin{aligned} \text{GI} &= \text{Entropia}(\text{pai}) - \\ &\quad p(\text{Saldo} < 50 \text{ mil}) * \\ &\quad \text{Entropia}(\text{Saldo} < 50 \text{ mil}) \\ &\quad - p(\text{Saldo} \geq 50 \text{ mil}) * \\ &\quad \text{Entropia}(\text{Saldo} \geq 50 \text{ mil}) \\ &= 0,99 - (13/30)*0,39 \\ &\quad - (17/30)*0,79 \\ &= \mathbf{0,37} \end{aligned}$$

Classificação por árvore de decisão



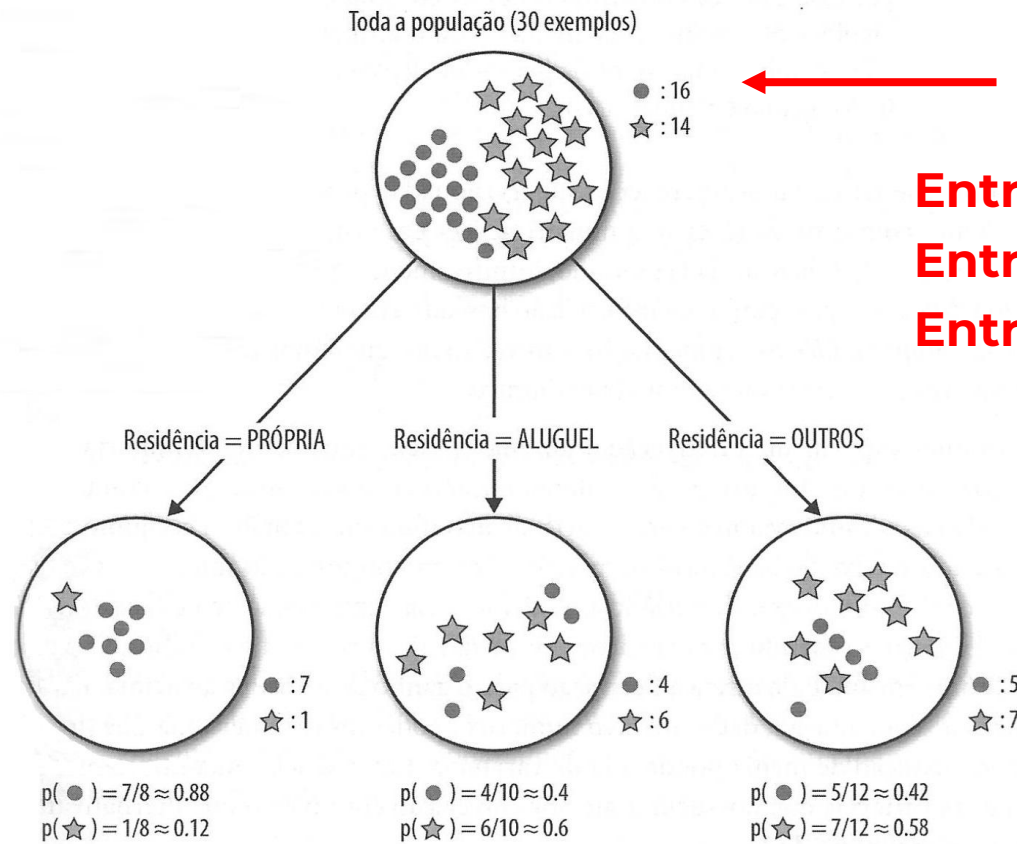
Façamos agora a divisão do conjunto com base no atributo “Residência” para ver se temos um ganho de informação maior

Classificação por árvore de decisão



Visualmente, podemos perceber que dois dos filhos são bem impuros

Classificação por árvore de decisão



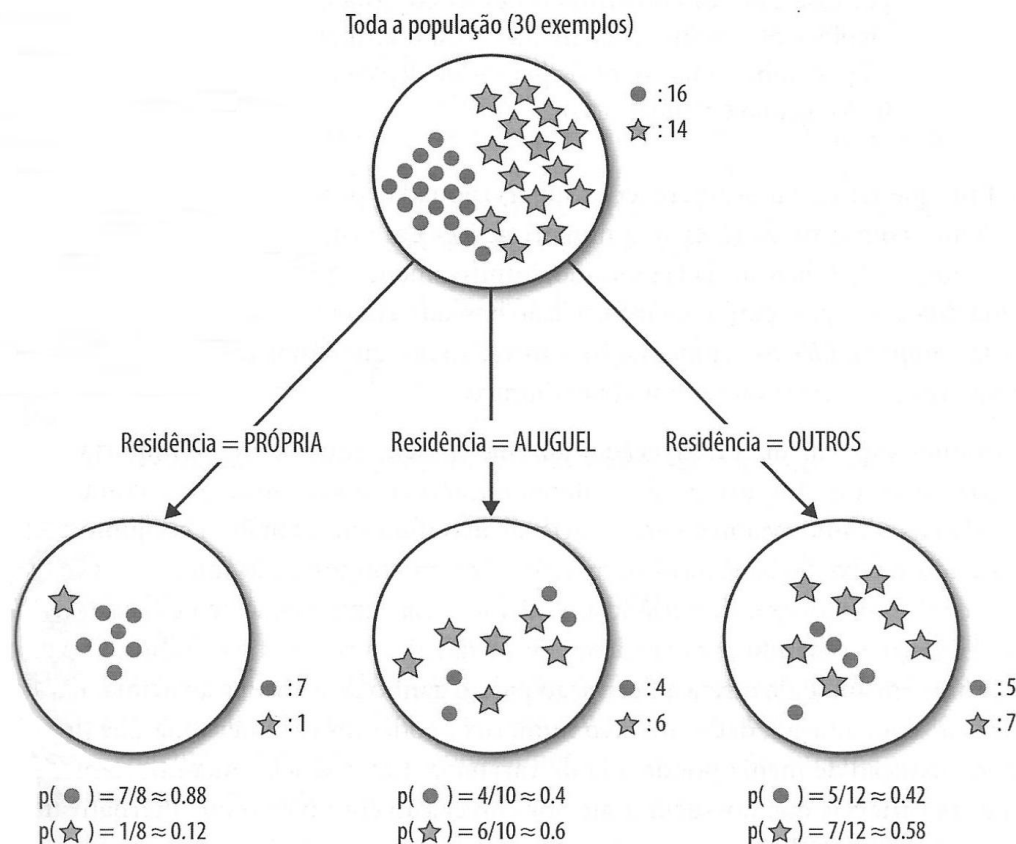
Entropia(pai) = 0,99

Entropia (Residência = PRÓPRIA) = 0,54

Entropia (Residência = ALUGUEL) = 0,97

Entropia (Residência = OUTROS) = 0,98

Classificação por árvore de decisão



Ganho de informação da divisão:

$$GI = 0,99 -$$

$$(8/30) * 0,54 -$$

$$(10/30) * 0,97 -$$

$$(12/30) * 0,98$$

$$= 0,13$$

$$0,13 < 0,37$$

Esta segunda divisão trouxe um ganho de informação menor que o do “Saldo”

Ou seja, a variável “Residência” é menos informativa

Classificação por árvore de decisão

- **Resumo da técnica:**
 - Identificar, na amostra pai (conjunto de treinamento), a variável alvo (classe) e seus possíveis valores.
 - Preparar os dados (por exemplo, para atributos numéricos do conjunto, criar faixas etc.).
 - Apurar a entropia da amostra pai para a variável alvo.
 - Classificar (segmentar / dividir) a amostra pai por diferentes atributos e apurar o ganho de informação gerado por cada divisão.
 - Identificar o(s) atributo(s) que gera(m) o(s) valor(es) mais alto(s) de ganho de informação.



Classificação por árvore de decisão

- Resumo da técnica – *cont.*:
 - Adotar então a subdivisão onde houve maior ganho de informação.
 - Para os conjuntos filhos de uma subdivisão, repetir o processo, fazendo a árvore crescer.
 - Parar quando não for possível mais prosseguir na subdivisão.
 - Apurar as probabilidades de classe dos nós mais baixos da árvore.
 - Avaliar a eficácia do modelo e tratar o sobreajuste, eventualmente realizando “poda da árvore”.



Classificação por árvore de decisão

- Além da construção de árvore por entropia (método ID3 clássico de Claude Shannon), outra técnica é a **Gini** (algoritmo CART).

$$\text{Gini}(D) = 1 - \sum_{i=1}^K p_i^2$$

$$\text{Gini}_{\text{divisão}} = \frac{n_{\text{esq}}}{n} \cdot \text{Gini}_{\text{esq}} + \frac{n_{\text{dir}}}{n} \cdot \text{Gini}_{\text{dir}}$$

$$\text{Ganho} = \text{Gini}_{\text{raiz}} - \text{Gini}_{\text{divisão}}$$

- Objetivo das subdivisões: maximizar o Ganho Gini.



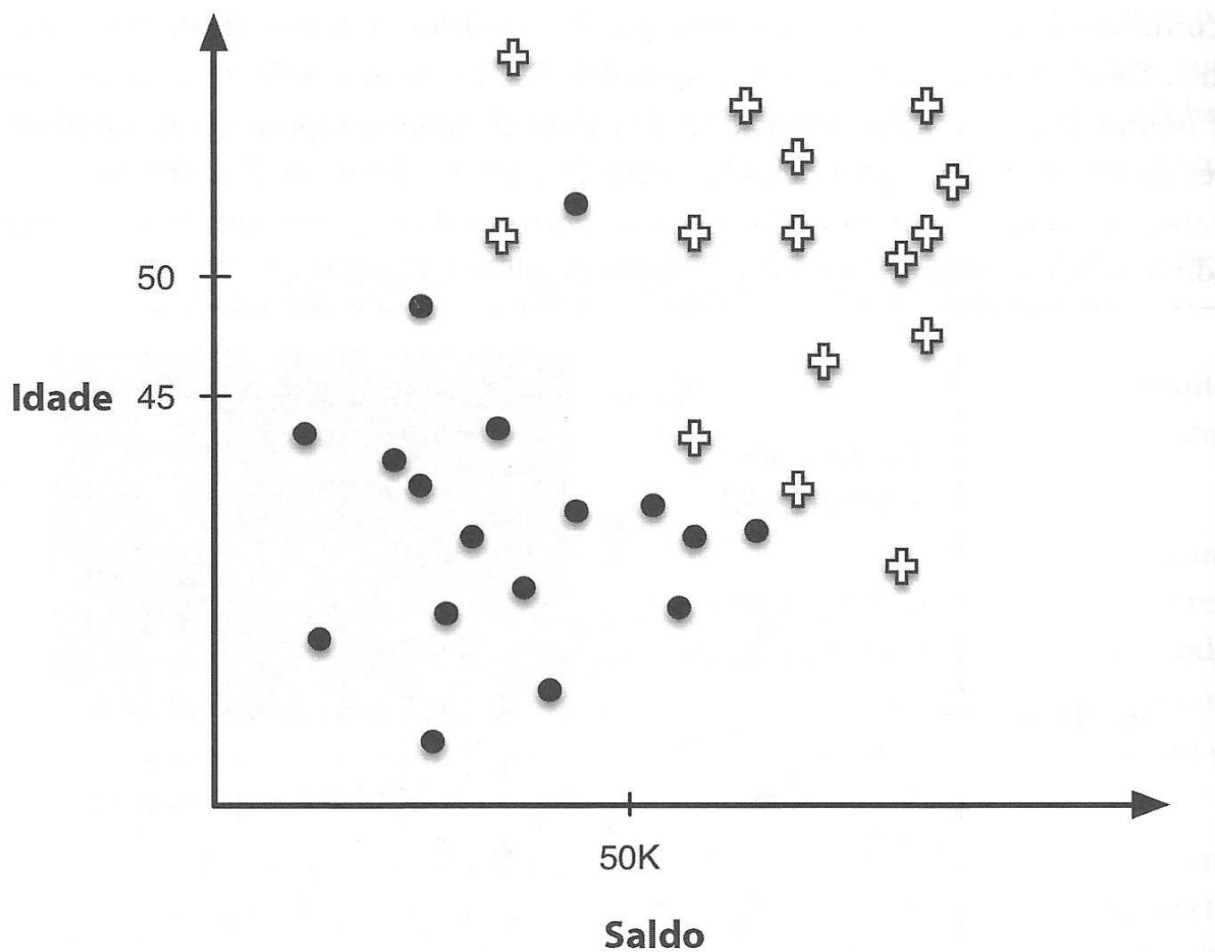
Modelagens preditivas - recapitulação

Classificação por discriminante linear



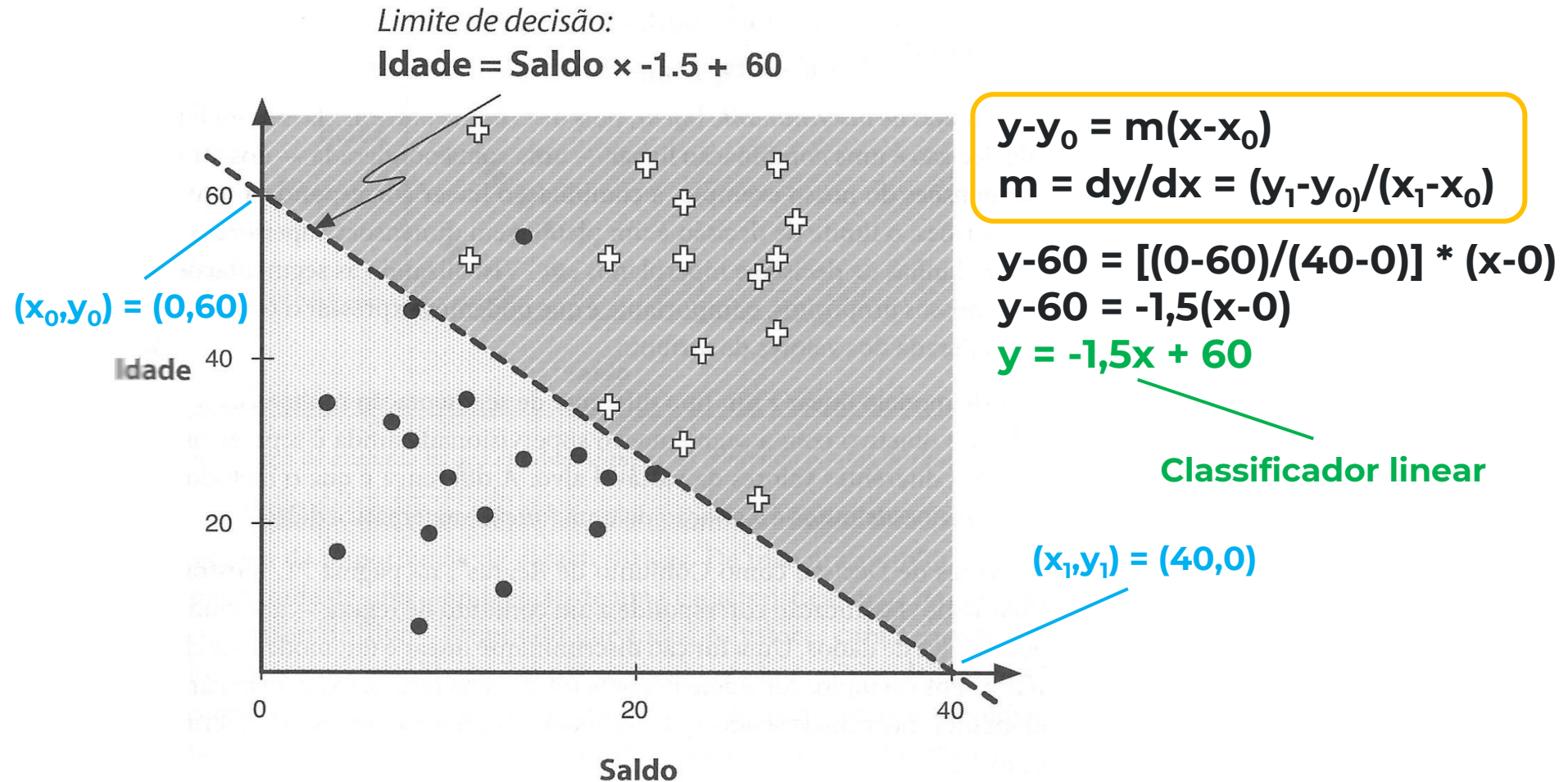
Classificação por discriminante linear

- Consideremos a seguinte distribuição de valores de variável alvo no conjunto de treinamento:



Classificação por discriminante linear

- Separando os indivíduos com uma equação de reta:



Classificação por discriminante linear

- **Discriminante linear**

- É uma reta que permite discriminar as classes.
- Modelo linear geral:
 - $f(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$
- No exemplo anterior, novos exemplos que se mostrarem acima da reta terão mais chance de ter variável alvo + e, abaixo, ●.
- Poderá haver modelos com 2, 3 ou N dimensões (retas, planos ou hiperplanos).



Classificação por discriminante linear

- Discriminante linear – *cont.*
 - Para o classificador linear $y = -1,5x + 60$
 - Exemplo (10,20):
 - $y = -1,5(10) + 60 = 45$
 - $20 < 45 \Rightarrow$ ponto abaixo da reta $\Rightarrow$ variável alvo = •
 - Exemplo (30,50):
 - $y = -1,5(30) + 60 = 15$
 - $50 > 15 \Rightarrow$ ponto acima da reta $\Rightarrow$ variável alvo = +



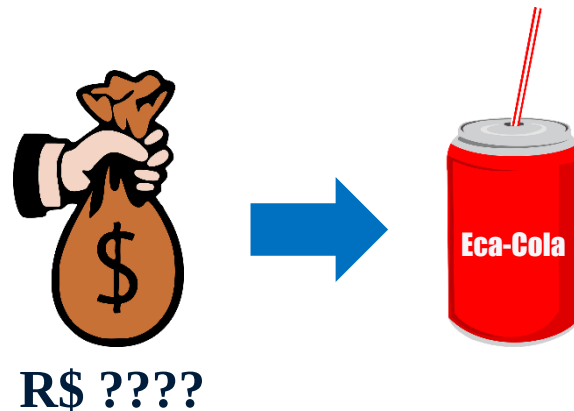
Modelagens preditivas - recapitulação

Regressão



Regressão - conceito

- **Regressão (“estimativa de valor”)**
 - Tentativa de estimar ou prever, para cada indivíduo, o valor numérico de alguma variável.



Exemplo: “Quanto um cliente vai consumir de determinado produto?”

A classificação tenta prever “se” e a regressão tenta prever “quanto”.

Regressão

- Regressão linear simples

- Para um conjunto de n pontos em um plano 2D $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots (x_n, y_n)$, uma reta da forma $y = ax + b$ que promova a regressão linear é obtida da seguinte forma:

- Coeficiente angular (a):

$$a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

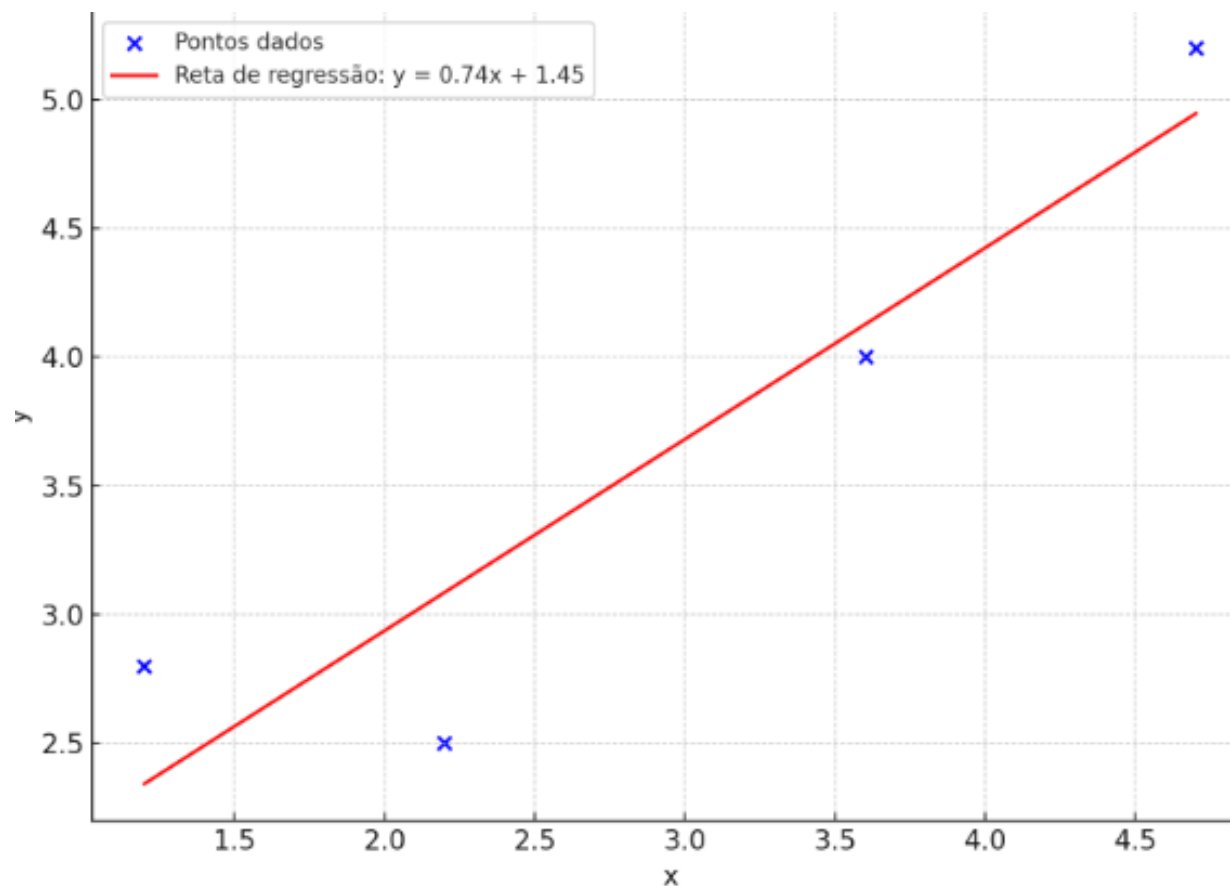
- Coeficiente linear (b):

$$b = \frac{\sum y_i - a \sum x_i}{n}$$



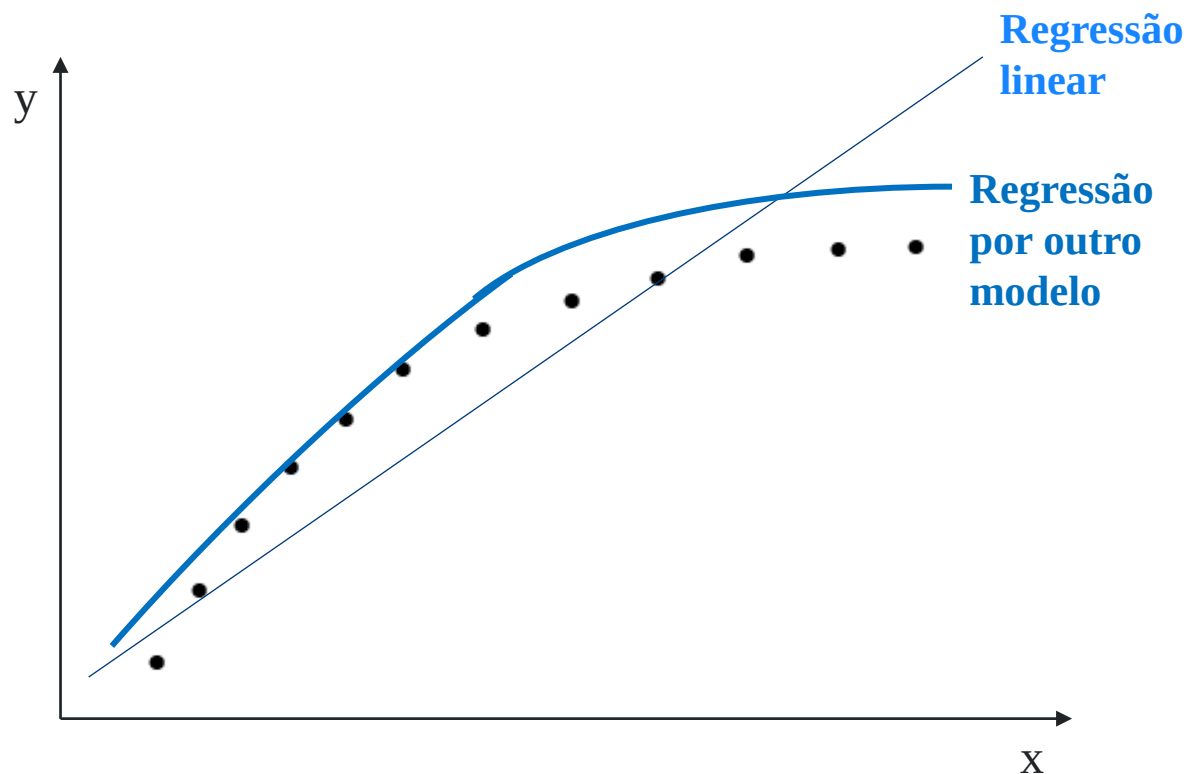
Regressão

- Regressão linear simples – *cont.*
 - Exemplo para os pontos (1.2 , 2.8), (2.2 , 2.5), (3.6 , 4.0) e (4.7 , 5.2):



Regressão

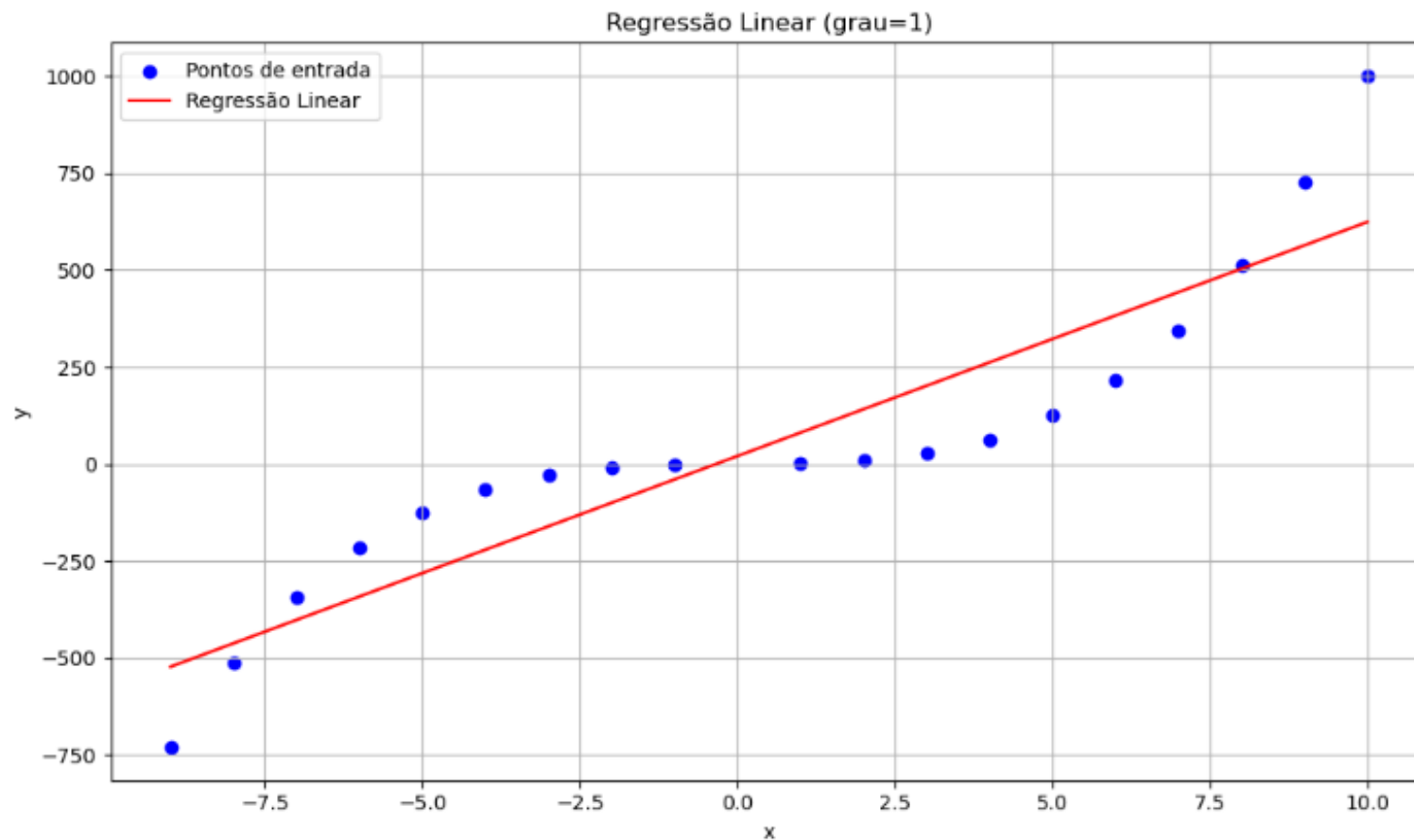
- A regressão linear simples resolve muitos problemas, mas não pode ser aplicada indiscriminadamente.



Regressão

- A regressão linear é um polinômio de grau 1:

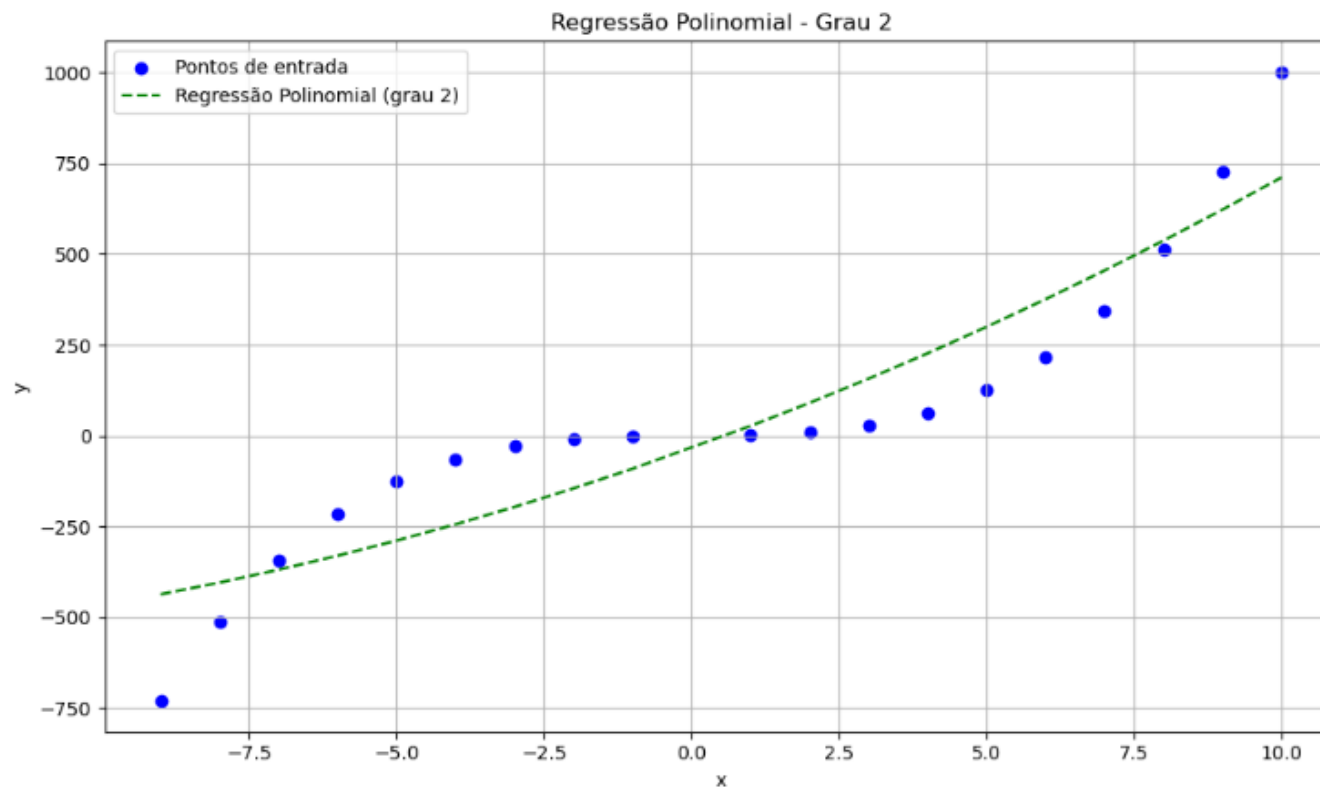
Equação da Regressão Linear (grau = 1):
 $y = 60.3843 * x + 20.8504$



Regressão

- Podemos testar polinômios de maior grau, para checar aderência – por exemplo, polinômio de grau 2 (regressão parabólica):

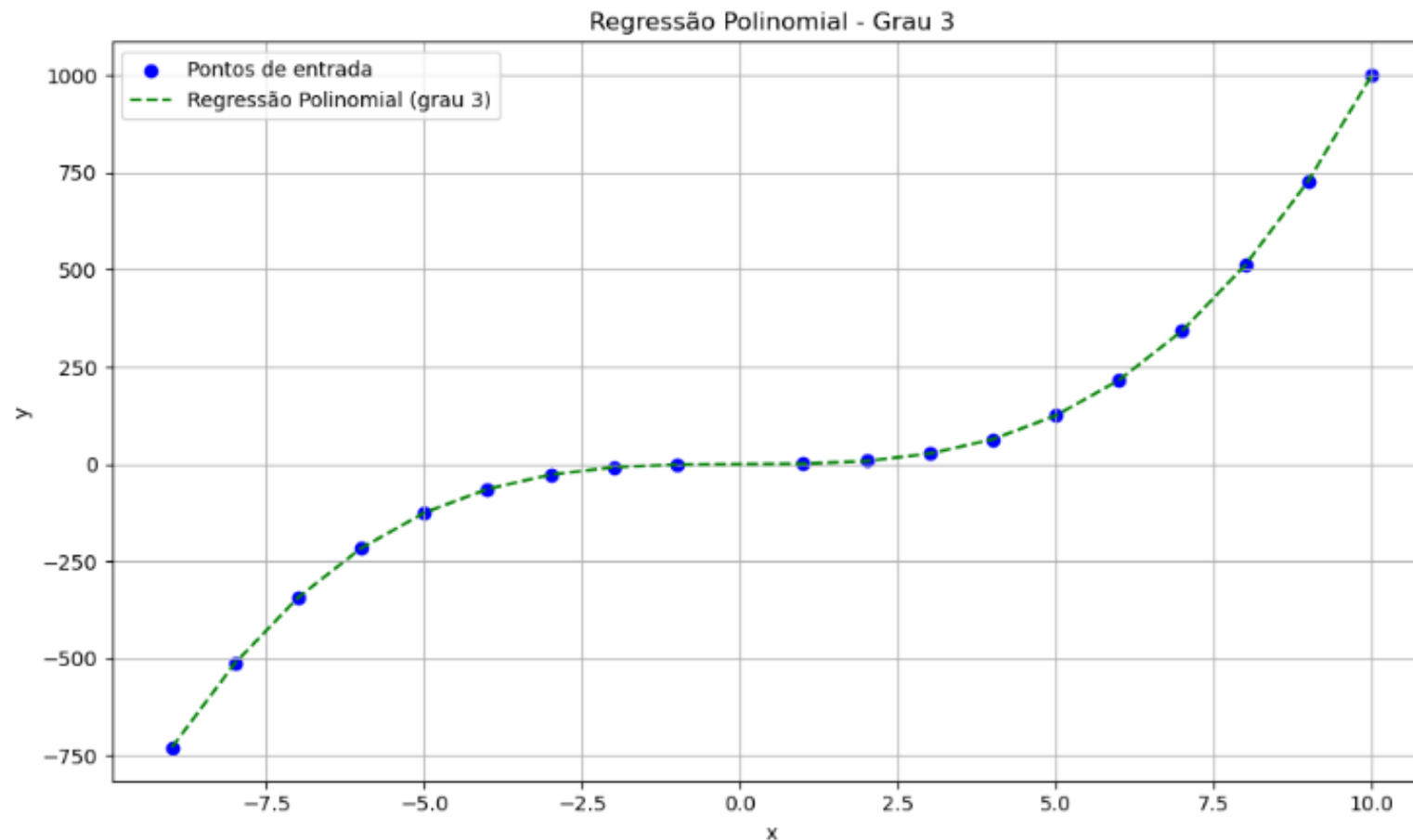
Equação da Regressão Polinomial (grau 2):
 $y = 1.5629 * x^2 + 58.8623 * x^1 + -33.4612$



Regressão

- Polinômio de grau 3 – no exemplo, gera ajuste total:

Equação da Regressão Polinomial (grau 3):
 $y = 1.0000 * x^3 + 0.0000 * x^2 + 0.0000 * x^1 + -0.0000$

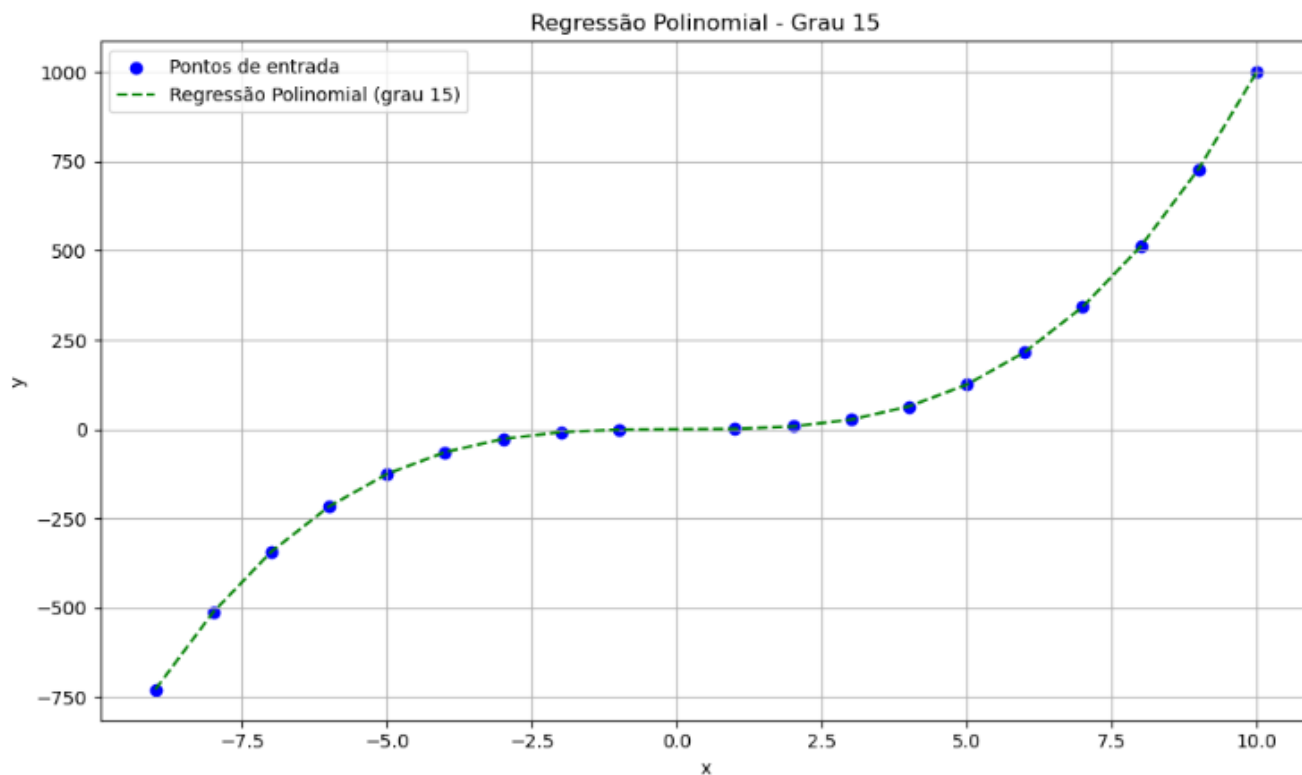


Regressão

- O aumento do grau do polinômio na regressão tende a gerar melhor aderência, mas também sobreajuste.

Equação da Regressão Polinomial (grau 15):

$y = 0.0000 * x^{15} + -0.0000 * x^{14} + -0.0000 * x^{13} + 0.0000 * x^{12} + 0.0000 * x^{11} + -0.0000 * x^{10} + -0.0000 * x^9 + 0.0000 * x^8 + 0.0000 * x^7 + -0.0000 * x^6 + -0.0000 * x^5 + 0.0000 * x^4 + 1.0000 * x^3 + -0.0000 * x^2 + 0.0000 * x^1 + 0.0000$



Observar que os coeficientes dos termos mais altos do polinômio que pedimos ao modelo que fosse de grau 15 ficam zerados, pois não ajudam a melhorar o ajuste.

Modelagens de Ciências de Dados - recapitulação

Regressão Logística



Regressão Logística - conceito

- Apesar do termo “regressão” no nome, esta técnica destina-se a resolver problemas de **classificação** binários (predição de probabilidade de classe com dois rótulos possíveis).
- A partir dos dados de treinamento, obtemos uma equação que, quando aplicada a uma expressão denominada **sigmoide**, permite chegarmos a probabilidades de classe positiva.



Regressão Logística

- **Razão de chances** = $p/(1-p)$
 - onde p = probabilidade de evento positivo
 - e, consecutivamente, $(1-p)$ = probabilidade de evento negativo
- A razão de chances é um número que pode adquirir os seguintes valores mínimo e máximo:
 - Mínimo ($p=0$): $0/(1-0) = 0$
 - Máximo ($p=1$): $1/1-1 = 1/0 = + \infty$



Regressão Logística

- **Função logit:**

- $\text{logit}(p) = \ln(p/(1-p))$

- Obs.: $\ln = \log_e$

- Na função logit:

- quando p é 0, $\text{logit}(p) = -\infty$

- quando p é 1, $\text{logit}(p) = +\infty$

- quando p é 0,5, $\text{logit}(p) = 0$

- Supondo $\text{logit}(p)$ como sendo uma função linear z :

- $\text{logit}(p) = \ln(p/(1-p)) = z = \beta_0 + \beta_1 x_1 + \dots + \beta_N x_N$



Regressão Logística

- Vamos isolar p na equação:

- $\log_e(p/(1-p)) = z$

- Lembrando da definição de logaritmo: $\log_a(b) = c \Leftrightarrow a^c = b$, temos:

- $p/(1-p) = e^z$

- $p = e^z * (1-p)$

- $p = e^z - e^z * p$

- $p + e^z * p = e^z$

- $p (1 + e^z) = e^z$

- $p = e^z / (1 + e^z)$

- Dividindo-se numerador e denominador por e^z , temos:

$$p = \frac{1}{1 + e^{-z}}$$



Regressão Logística

- A equação obtida é conhecida por **função sigmoide**:

$$p = \frac{1}{1 + e^{-z}}$$

- Lembrando que $z = \beta_0 + \beta_1 x_1 + \dots \beta_N x_N$:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots \beta_N x_N)}}$$

O grande objetivo da regressão logística é aprender, a partir de um conjunto de treinamento, a equação linear z . Depois, para qualquer novo indivíduo, calcula-se z e aplica-se a função sigmoide para obter a probabilidade de ele pertencer à classe positiva p .



Regressão Logística

- As funções **logit** e **sigmoide** são inversas uma da outra, ou seja:
 - logit(sigmoide(z))=z**
- Demonstração:

$$\begin{aligned}\text{Logit}(\text{Sigmoide}(z)) &= \ln \left(\frac{p}{1-p} \right) \\ &= \ln \left(\frac{\frac{1}{1+e^{-z}}}{1 - \frac{1}{1+e^{-z}}} \right) \\ &= \ln \left(\frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}} \right) \\ &= \ln \left(\frac{1}{e^{-z}} \right) \\ &= \ln(e^z) \\ &= z\end{aligned}$$



Regressão Logística

- Observar que, por exemplo, para uma probabilidade $p = 0,80$:

Aplicando o **logit**:

$$z = \ln \left(\frac{0,80}{0,20} \right) = \ln(4) \approx 1,386$$

Agora aplique a **sigmoide** a esse valor de z :

$$p = \frac{1}{1 + e^{-1,386}} = 0,80$$

- Ou seja, a partir de z , podemos chegar em uma probabilidade.



Regressão Logística

- Temos, por outro lado, que:
 - **sigmoide(logit(p))=p**
- Logit transforma probabilidade em escala infinita.
- Sigmoides transforma escala infinita em probabilidade.
- Para a aplicação da regressão logística, devemos, a partir dos dados de treinamento:
 - Obter uma equação $z = \beta_0 + \beta_1 x_1 + \dots + \beta_N x_N$, via iterações sucessivas que apuram β_N de forma a **minimizar a Log Loss média** (veremos do que se trata).
 - Alcançados os β_N melhores possíveis (com menor Log Loss média), calcular o valor de **z** para o exemplo cuja probabilidade de classe **p** se deseja prever.
 - **p** é obtido na função sigmoide: $p = 1 / (1 + e^{-z})$.



Regressão Logística

- **Preparação dos dados para a regressão logística**
 - Os atributos dos dados de treinamento, incluindo a variável alvo (classe), são convertidos para numéricos.
 - Exemplos:
 - Renda: em K reais
 - Imóvel:
 - 0 – não possui
 - 1 – possui
 - Classe (atributo alvo):
 - 0 – negativa (por exemplo, não é cliente de interesse)
 - 1 – positiva (por exemplo, é cliente de interesse)



Regressão Logística

- Preparação dos dados para a regressão logística – *cont.*
 - Para variáveis categóricas nominais, sem uma ordinalidade (sem grau de importância relativa):
 - Exemplo: variável “cor”:
 - Seria errado usar
 - Azul - 1
 - Verde - 2
 - Vermelho – 3pois isto faria o algoritmo interpretar que $3 > 2 > 1$.
 - Neste caso, o correto é lançar mão do **one-hot encoding** (transformação da variável em várias colunas).



Regressão Logística

- Preparação dos dados para a regressão logística – *cont.*
 - *One-hot encoding* para o exemplo:

Cor	Azul	Verde	Vermelho
Azul	1	0	0
Verde	0	1	0
Vermelho	0	0	1

- Para variáveis ordinais (onde há uma ordem de força natural):
 - Exemplo: variável “nível de escolaridade”:
 - Fundamental - 1
 - Médio - 2
 - Superior - 3
 - Pós - 4



Regressão Logística

- Preparação dos dados para a regressão logística – *cont.*
 - Para lidar com variáveis com valores muito amplos ou cujas ordens de grandeza são muito discrepantes em relação às demais, lançamos mão da **normalização**.
 - Exemplos:
 - Idade: variando de 18 a 80 na amostra
 - Valor do patrimônio: variando entre R\$ 10.000 a R\$ 10.000.000
 - A normalização (ou *z-score*) é obtida com:

$$x' = \frac{x - \mu}{\sigma}$$

x' = valor normalizado

x = valor original da variável

μ = média dos valores da amostra

σ = desvio padrão

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n}}$$



Regressão Logística

- Preparação dos dados para a regressão logística – *cont.*
 - Resumo das transformações para preparação

Tipo de variável	Preparação recomendada
Classe binária	Codificar como 0 e 1
Numérica	Manter (e, em geral, padronizar)
Binária	Codificar como 0 e 1
Categórica nominal	One-Hot Encoding
Categórica ordinal	Codificação ordinal (ou One-Hot, dependendo do problema)
Valores ausentes	Imputar ou remover



Regressão Logística

- Preparação dos dados para a regressão logística – *cont.*
 - Valores normalizados podem ter sinal negativo.
 - A normalização faz com que as variáveis fiquem com escalas semelhantes, tornando o treinamento mais estável.
 - Veremos mais adiante um exemplo de como a falta da normalização impacta no treinamento do modelo.



Regressão Logística

- Exemplo de aplicação da regressão logística
 - Consideremos estes dados de treinamento (classe $y = 1$ = positiva)

Cliente	x_1 (Renda)	x_2 (Imóvel)	Classe y
A	3	0	0
B	4	0	0
C	6	1	1
D	7	1	1
E	10	1	1
F	8	1	1



Regressão Logística

- $z = \beta_0 + \beta_1(\text{idade}) + \beta_2(\text{renda}) + \beta_3(\text{imóvel})$
- **Chute inicial (iteração zero):**
 - $\beta_0 = \beta_1 = \beta_2 = 0 \Rightarrow$
 - $z = 0$ para todos os clientes
 - Sigmoides correspondente:
 - $p = 1 / (1 + e^0) = 1/(1+1) = 0,5$



Regressão Logística

- $z = \beta_0 + \beta_1(\text{idade}) + \beta_2(\text{renda})$
- **Chute inicial (iteração zero) - cont.:**

Cliente	Classe	Probabilidade
A	0	0,5
B	0	0,5
C	1	0,5
D	1	0,5
E	1	0,5
F	1	0,5



Regressão Logística

- Calculamos agora a chamada **Log Loss** de cada cliente:

$$\text{Log Loss} = - [y * \ln(p) + (1-y) * \ln(1-p)]$$

- Para o cliente A, por exemplo:
 - Classe real $y = 0$
 - Então $\text{Log Loss} = -[0 * \ln(0,5) + (1-0) * \ln(0,5)]$
 - Como $\ln(0,5) = -0,6931$, então $\text{Log Loss} = -(-0,6931) = 0,6931$
- Para o cliente C, por exemplo:
 - Classe real $y = 1$
 - Então $\text{Log Loss} = -[1 * \ln(0,5) + (1-1) * \ln(0,5)]$
 - Como $\ln(0,5) = -0,6931$, então $\text{Log Loss} = -(-0,6931) = 0,6931$



Regressão Logística

- Como todos os clientes receberam probabilidade 0,5, todos terão a mesma perda (0,6931).
- Log Loss média = $(6 * 0,6931) / 6 = 0,6931$



Regressão Logística

- Agora, o objetivo é realizar **iterações** de forma a **diminuir** cada vez mais a Log Loss média.
- Em cada nova iteração, devemos considerar novos valores para β_0 , β_1 e β_2 , apurar seus sigmoides e checar se a Log Loss média cai.
- Para o cálculo dos novos valores de β_0 , β_1 e β_2 , teremos de apurar os **gradientes** de cada um:

$$\text{Gradiente } \beta_j = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_{ij}$$



Regressão Logística

Cliente	p	y	$p - y$
A	0,5	0	0,5
B	0,5	0	0,5
C	0,5	1	-0,5
D	0,5	1	-0,5
E	0,5	1	-0,5
F	0,5	1	-0,5



Regressão Logística

▪ Iteração 1

- Apuração de novo valor para β_0 , começando pelo seu gradiente:
 - Na equação de z , β_0 é a variável constante ou viés (não multiplica nenhuma variável independente x_i).
 - β_0 recebe o nome de **intercepto**.
 - O intercepto é tal como se fosse multiplicado por uma variável fictícia $x_0 = 1$.
 - Primeiramente, calculamos o **gradiente de β_0** :

$$\frac{1}{n} \sum (p - y)$$

- **Gradiente de $\beta_0 = 0,5 + 0,5 - 0,5 - 0,5 - 0,5 - 0,5)/6 = -0,1667$**



Regressão Logística

- Agora, vamos realizar o mesmo para os demais β_N :
- Para β_1 (renda):

Cliente	$p-y$	Renda	$(p-y)x_1$
A	0,5	3	1,5
B	0,5	4	2
C	-0,5	6	-3
D	-0,5	7	-3,5
E	-0,5	10	-5
F	-0,5	8	-4

- **Gradiente $\beta_1 = (1,5 + 2 - 3 - 3,5 - 5 - 4)/6 = -2$**



Regressão Logística

- Para β_2 (imóvel):

Cliente	$p-y$	Imóvel	$(p-y)x_2$
A	0,5	0	0
B	0,5	0	0
C	-0,5	1	-0,5
D	-0,5	1	-0,5
E	-0,5	1	-0,5
F	-0,5	1	-0,5

- **Gradiente $\beta_2 = (0 + 0 - 0,5 - 0,5 - 0,5 - 0,5)/6 = -0,3333$**



Regressão Logística

- Obtivemos então os gradientes de β_N da Iteração 1:
 - Gradiente de $\beta_0 = -0,1667$
 - Gradiente de $\beta_1 = -2$
 - Gradiente de $\beta_2 = -0,3333$



Regressão Logística

- **Iteração 1 – cont.**
- Calculados os gradientes, vamos agora obter os valores de β_N .
- A regra é: $\beta_{\text{novo}} = \beta_{\text{anterior}} - \eta * \text{gradiente}$
- $\eta = \text{taxa de aprendizado}$ – é um valor tentativo (normalmente, 0,1, 0,001 ou 0,0001) – deve-se experimentar e verificar se o erro está diminuindo, se está havendo oscilação, se o algoritmo está convergindo.
- Algoritmos como *Adam*, *RMSProp* ou *AdaGrad* fazem os ajustes automaticamente.
- Vamos adotar no nosso exemplo $\eta = 0,01$.



Regressão Logística

- Novos valores de β_N .
- $\beta_{\text{novo}} = \beta_{\text{anterior}} - \eta * \text{gradiente}$

β_0

$$0 - 0,01(-0,1667) = 0,001667$$

β_1

$$0 - 0,01(-2) = 0,0200$$

β_2

$$0 - 0,01(-0,3333) = 0,003333$$

- Com isto, $z = -0,001667 + 0,0200x_1 + 0,003333x_2$



Regressão Logística

- $z = -0,001667 + 0,0200x_1 + 0,003333x_2$
- Então, para cada cliente dos dados de treinamento, temos:

Cliente A

$$\begin{aligned} z &= 0,001667 + 0,02(3) + 0,003333(0) \\ &= 0,061667 \end{aligned}$$

Cliente B

$$0,081667$$

Cliente C

$$0,125000$$



Regressão Logística

- $z = -0,001667 + 0,0200x_1 + 0,003333x_2$
- Continuando...

Cliente D

0,145000

Cliente E

0,205000

Cliente F

0,165000



Regressão Logística

- $z = -0,001667 + 0,0200x_1 + 0,003333x_2$
- Em resumo, os novos z após a Iteração 1 são:

Cliente	z
A	0,0617
B	0,0817
C	0,1250
D	0,1450
E	0,2050
F	0,1650



Regressão Logística

- Aplicando-se agora a nova sigmoide para cada cliente, temos:

$$p = \frac{1}{1 + e^{-z}}$$

Cliente	Classe	Probabilidade
A	0	0,5154
B	0	0,5204
C	1	0,5312
D	1	0,5362
E	1	0,5511
F	1	0,5412



Regressão Logística

- Calculando agora o novo Log Loss para cada cliente:

$$\text{Log Loss} = - [y \cdot \ln(p) + (1-y) \cdot \ln(1-p)]$$

- Cliente A (classe 0), por exemplo:

$$-\ln(1 - 0,5154) = -\ln(0,4846) = 0,724$$

- Cliente C (classe 1), por exemplo:

$$-\ln(0,5312) = 0,633$$



Regressão Logística

- A apuração de todos os Log Loss conduz ao seguinte Log Loss médio:
 - $\text{Log Loss médio} = (0,724+0,735+0,633+0,623+0,596+0,614)/6 = 0,654$
- Antes da primeira iteração do treinamento, tínhamos:
 - $\text{Log Loss médio} = 0,693$
- Portanto, o Log Loss médio diminuiu, ou seja, os coeficientes β_N caminharam na direção certa.



Regressão Logística

- A iteração 2 repetirá exatamente o mesmo ciclo:
 - Partir dos coeficientes β_0 , β_1 e β_2 da iteração 1.
 - Apurar seus gradientes, com as probabilidades oriundas da iteração 1 e com isto chegar a novos valores de β_0 , β_1 e β_2 .
 - Com isto, obter a nova equação z .
 - Recalcular o valor de z para cada cliente.
 - Aplicar a sigmoide para obter as novas probabilidades.
 - Calcular uma nova Log Loss média (que tende a ser menor que 0,654).
- E assim por diante, até os novos coeficientes β_N não melhorarem mais o modelo (praticamente não causarem mais reduções do Log Loss médio).



Regressão Logística

- **A importância da normalização dos dados de treinamento na execução do algoritmo de regressão logística**
 - Para ilustrar este ponto, vejamos um exemplo onde um dos atributos tem escala diferente e não foi normalizado.
 - Faremos isto para o atributo “idade”.



Regressão Logística

Cliente	Idade x_1	Renda x_2	Imóvel x_3	Classe y
A	25	3	0	0
B	28	4	0	0
C	35	6	1	1
D	40	7	1	1
E	45	10	1	1
F	38	8	1	1



Regressão Logística

- Iteração zero

Cliente	Classe y	Probabilidade p
A	0	0,5
B	0	0,5
C	1	0,5
D	1	0,5
E	1	0,5
F	1	0,5



Regressão Logística

- Como todos os clientes receberam probabilidade 0,5, todos terão a mesma perda (0,6931).
- Log Loss média = $(6 * 0,6931) / 6 = 0,6931$

Cliente	Loss
A	0,6931
B	0,6931
C	0,6931
D	0,6931
E	0,6931
F	0,6931



Regressão Logística

- Obtivemos então os quatro gradientes da Iteração 1:
 - Gradiente de $\beta_0 = -0,1667$
 - Gradiente de $\beta_1 = -8,75$
 - Gradiente de $\beta_2 = -2$
 - Gradiente de $\beta_3 = -0,3333$
- Então:

Coeficiente	Novo valor
β_0	0,001667
β_1	0,0875
β_2	0,0200
β_3	0,003333



Regressão Logística

- Novos z:

Cliente	z
A	2,249
B	2,532
C	3,188
D	3,665
E	4,139
F	3,487



Regressão Logística

- Novas probabilidades (aplicação da sigmoide):

Cliente	z	Probabilidade
A	2,249	0,905
B	2,532	0,926
C	3,188	0,960
D	3,665	0,975
E	4,139	0,984
F	3,487	0,970



Regressão Logística

- Note que o modelo passou a prever probabilidade muito alta para todos, inclusive para A e B, cuja classe correta é 0.
- O problema aqui é a escala das variáveis.
- A idade varia entre 25 e 45. Quando multiplicamos $0,0875 \times 45$ já obtemos quase 4. Isso faz z ficar muito grande.
 - Isto vai gerar uma piora no Log Loss médio.
- Para resolver este fenômeno, devemos normalizar o atributo “idade” antes de começarmos o treinamento.



Regressão Logística

- Por exemplo:

Idade original	Idade normalizada
25	-1,3
28	-0,9
35	0
40	0,8
45	1,5

$$x' = \frac{x - \mu}{\sigma}$$

x' = valor normalizado

X = valor original da variável

μ = média dos valores da amostra

σ = desvio padrão

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n}}$$

- Agora os valores ficam próximos de zero. Os coeficientes deixam de explodir e o treinamento fica muito mais estável.

Regressão Logística

- Sem a normalização, a quantidade de iterações do algoritmo para conseguir convergência seria muito maior (converge, mas demora).
- Os algoritmos modernos de otimização (como *Adam* e *RMSProp*) conseguem lidar melhor com diferenças de escala do que o gradiente descendente "puro".
- Mesmo assim, padronizar as variáveis continua sendo uma boa prática, porque geralmente melhora a velocidade de convergência, reduz problemas numéricos e torna o treinamento mais previsível.
- Portanto, a normalização não muda o objetivo do algoritmo (encontrar os melhores coeficientes), mas costuma tornar o caminho até essa solução muito mais eficiente.



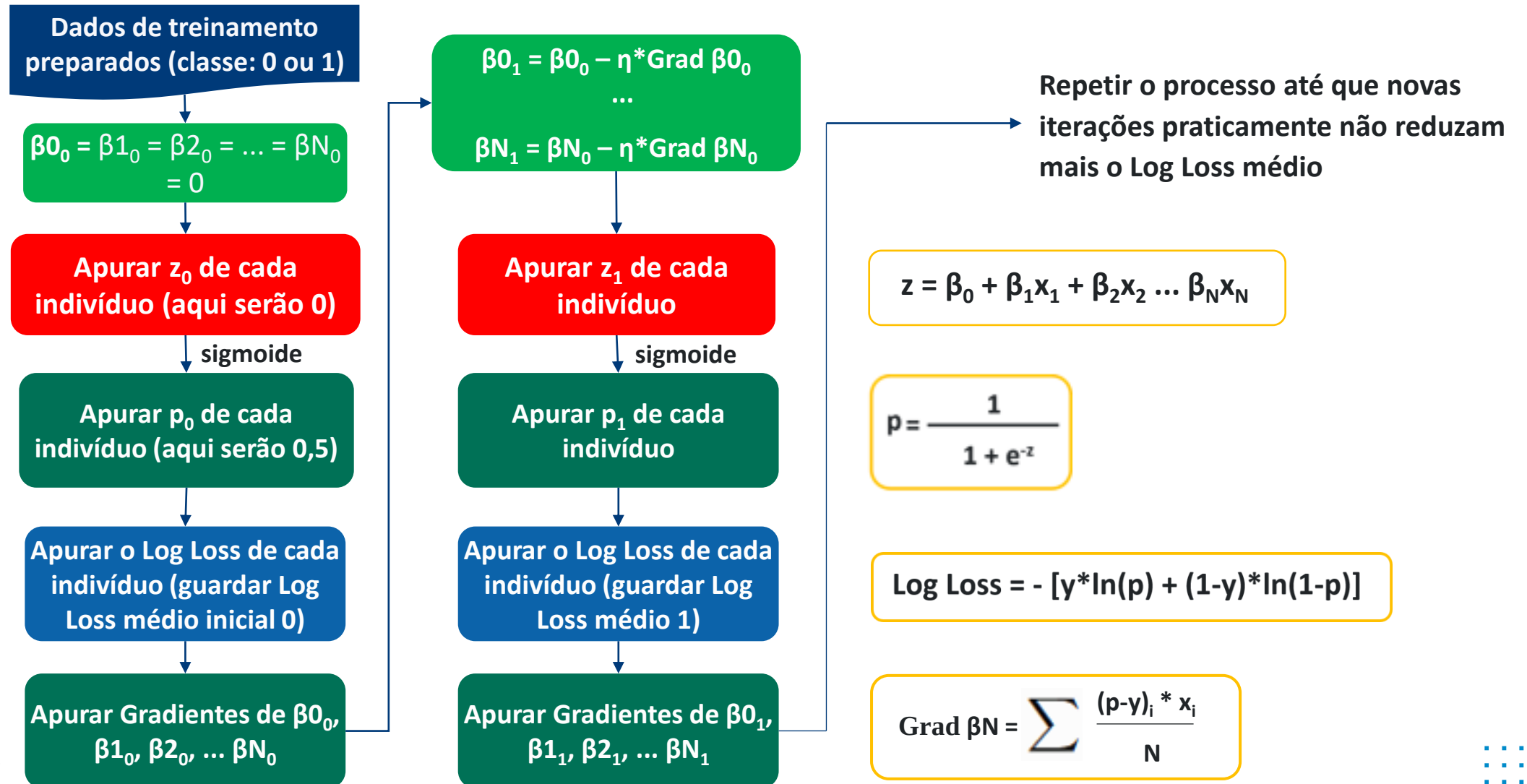
Regressão Logística

- O polinômio z pode ter grau diferente de 1?
 - Sim, podemos treinar modelos com polinômios de grau 2 ou 3 e comparar o desempenho, lembrando que os graus mais altos aumentam o risco de sobreajuste.
 - **Princípio da Parcimônia** ou **Navalha de Occam**: entre dois modelos com desempenho semelhante, prefere-se o mais simples. Modelos mais simples tendem a generalizar melhor.



Regressão Logística

- Regressão Logística– resumo esquemático do algoritmo:



Regressão Logística

- Quando é melhor usar árvore de decisão ou regressão logística?
 - Use Árvore de Decisão quando:
 - Existem regras de decisão do tipo "se... então..."
 - Há relações não lineares ou interações complexas entre os atributos
 - Deseja um modelo facilmente explicável para usuários de negócio
 - Não quer se preocupar com normalização das variáveis
 - Use Regressão Logística quando:
 - Deseja estimar probabilidades
 - A relação entre as variáveis e a classe é aproximadamente linear
 - A interpretabilidade dos coeficientes é importante
 - O risco de sobreajuste (*overfitting*) precisa ser baixo



Modelagens preditivas - recapitulação

Máquina Vetorial de Suporte



Máquina Vetorial de Suporte - conceito

- A **Máquina Vetorial de Suporte** (*Support Vector Machine – SVM*) é um algoritmo supervisionado que tem como objetivo encontrar a fronteira de decisão (faixa) que melhor separa classes, maximizando a distância entre essa fronteira e os exemplos mais próximos.

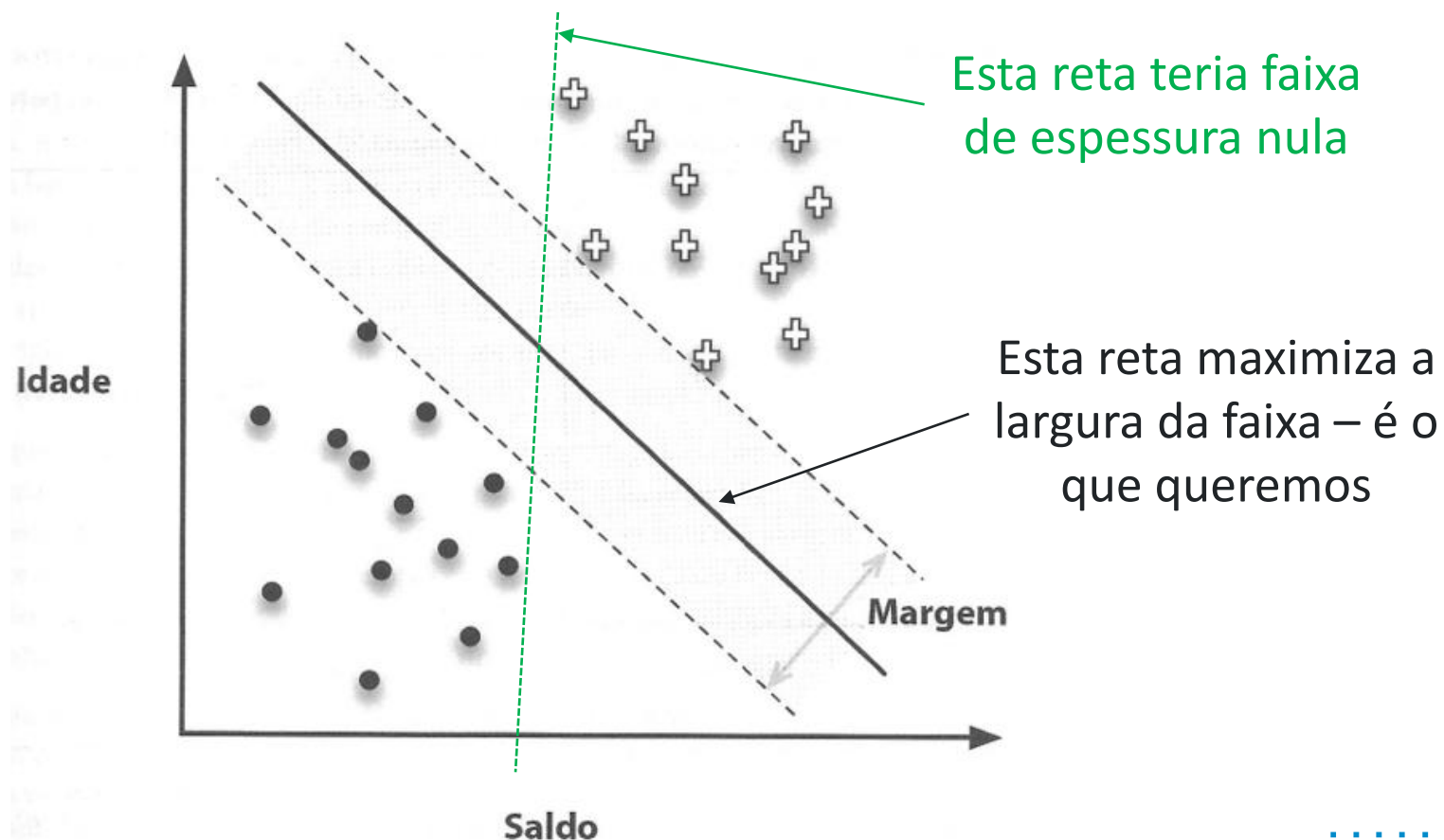


Máquina Vetorial de Suporte

- Um discriminante linear é a linha central da faixa e deveremos maximizar a margem (para dar “segurança” ao separador).

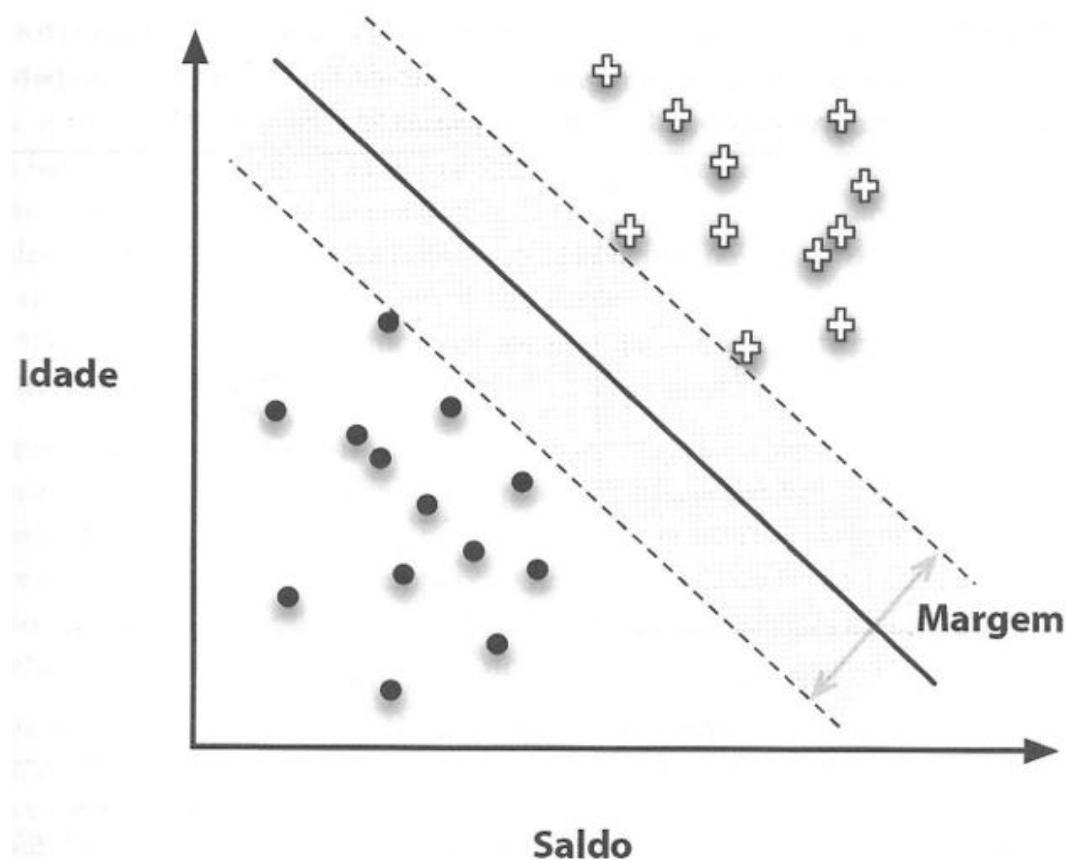
Exemplo em plano
bidimensional

Note que há uma inclinação da reta central que maximiza a largura da faixa onde não há pontos dentro



Máquina Vetorial de Suporte

- Quem define a faixa não são todos os pontos dos dados de treinamento, mas sim os pontos mais próximos da fronteira.



**Módulo 3 – Engenharia de
Variáveis (*Feature
Engineering*)**

Feature Engineering

- A **qualidade das variáveis (*features*)** do conjunto de treinamento normalmente influencia mais o desempenho do modelo do que a escolha do algoritmo (árvore, regressão logística etc.).
- *“Garbage in, garbage out”*.
- **Engenharia de Variáveis (*Feature Engineering*)** é o processo de preparar e transformar as variáveis que serão utilizadas por um algoritmo de Machine Learning.



Feature Engineering

- *A Feature Engineering* envolve:
 - Selecionar variáveis relevantes
 - Criar novas variáveis
 - Transformar variáveis
 - Combinar variáveis
 - Remover variáveis inúteis
 - Reduzir dimensionalidade (do espaço amostral) quando necessário



Feature Engineering

- O fluxo de modelagem fica então:
 - Obtenção de dados brutos
 - Limpeza dos dados
 - Aplicação de técnicas de *Feature Engineering*
 - Preparação e treinamento do modelo
 - Avaliação e refinamento do modelo



Feature Engineering

- **Avaliação das variáveis candidatas**
 - Nem toda variável disponível deve entrar no modelo
 - É preciso julgar quais **possuem relação com o problema de negócio**
 - Exemplo – para prever acidentes automobilísticos:
 - Podem ser variáveis úteis:
 - Idade do condutor
 - Tempo de habilitação
 - Histórico de acidentes
 - Quilometragem do veículo
 - Não são úteis:
 - RG do condutor
 - Placa do veículo



Feature Engineering

- **Avaliação das variáveis candidatas – *cont.***
 - **Devemos também remover:**
 - **Variáveis redundantes**
 - **Exemplos:**
 - Data de nascimento
 - Idade
 - **Variáveis com muitos valores ausentes**
 - **Exemplos:**
 - Estado civil com 80% dos campos vazios
 - Renda que não se deseja informar



Feature Engineering

- Devemos também remover – *cont.*:
 - Variáveis com baixa variabilidade
 - Exemplos:
 - Sexo com 98,5% sendo masculino
 - Estado de saúde “bom” para todos os indivíduos do grupo
 - Variáveis altamente correlacionadas
 - Exemplos (melhor manter uma só):
 - Peso em Kg
 - Peso em libras



Feature Engineering

- Avaliação das variáveis candidatas – *cont.*
 - Como preencher valores ausentes corretamente:
 - Para variáveis com poucos valores ausentes, podemos adotar **mediana** (somente dos dados de treinamento) para preenchê-los.
 - Apuramos então a mediana dos dados da partição de treinamento (usando o Excel, consegue-se isto com =MED(intervalo de valores)).
 - Para os dados de teste (retenção), **não** calculamos sua mediana, mas sim preenchemos seus vazios com o mesmo valor de mediana usada para os dados de treinamento, para evitar o fenômeno do **vazamento** (*data leakage*).



Feature Engineering

- Avaliação das variáveis candidatas – *cont.*
 - Como preencher valores ausentes corretamente – *cont.*:
 - O que é o vazamento?
 - Vazamento é permitir que informações dos dados de teste influenciem o treinamento ou o pré-processamento do modelo. Isso não queremos, pois os dados de teste devem representar dados que o modelo encontrará no futuro e que não participaram da definição das regras utilizadas pelo modelo.



Feature Engineering

- Avaliação das variáveis candidatas – *cont.*
 - Como preencher valores ausentes corretamente – *cont.*:
 - Mas preencher os vazios dos dados de teste com a mediana dos dados de treinamento não é permitir que os dados de teste “espionem” a modelagem?
 - Não, pois a mediana utilizada no teste foi calculada exclusivamente a partir dos dados de treinamento. Ela é um parâmetro aprendido durante o treinamento e já está definido antes de os dados de teste serem utilizados. O teste apenas recebe e aplica essa regra, sem fornecer qualquer informação para sua definição.



Feature Engineering

- Avaliação das variáveis candidatas – *cont.*
 - Como preencher valores ausentes corretamente – *cont.*:
 - Se preenchermos os vazios dos dados de teste com a mediana calculada a partir dos próprios dados de teste, **estaremos usando informações do teste para definir como ele será pré-processado**. Isso é uma forma de **vazamento de pré-processamento**. Não podemos usar o teste para descobrir as regras que serão usadas para processá-lo.
 - *Pense na situação de chegar UM indivíduo novo com o valor vazio: não haveria uma "mediana dos dados de teste" para preenchê-lo. Usamos a mediana que foi calculada no treinamento e já está disponível, exatamente como faríamos com qualquer novo indivíduo que chegasse no futuro.*



Feature Engineering

- **Seleção de variáveis entre as candidatas**
 - 1 - Seleção baseada em conhecimento
 - Especialistas opinam
 - Por exemplo, um médico intui que idade, pressão arterial e nível de colesterol são importantes para um contexto
 - Aqui entra um pouco o caráter “de arte” da Ciência de Dados



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística
 - Exemplos:
 - Correlação
 - Qui-quadrado (Chi-square)
 - ANOVA (análise de variância)
 - *Mutual Information*
 - *Information Gain*



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - **Correlação** – mede o quanto duas features variam juntas.

Altura	Peso
1,60	60
1,70	70
1,80	80

Forte correlação

Número da casa	Salário
12	5000
87	3200
15	9000

Sem correlação

Se queremos prever um valor numérico (regressão), procuramos variáveis com maior correlação com a variável resposta.

Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - **Correlação** – como medir
 - **Coefficiente de correlação de Pearson** (para duas *features* x e y):

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

- Uma IA generativa ou o Excel pode calcular para você.
- Supondo **Valor_Compras** na coluna D e **Número_Compras** na coluna E, a correlação no Excel é apurada com **=CORREL(D2:D16;E2:E16)**
 - ✓ $r = +1 \rightarrow$ correlação linear positiva perfeita
 - ✓ $r = 0 \rightarrow$ nenhuma correlação linear
 - ✓ $r = -1 \rightarrow$ correlação linear negativa perfeita



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - **Qui-quadrado** – avalia a associação entre uma variável categorica e a classe.

Sexo	Inadimplente
Masc.	Sim
Masc.	Não
Fem.	Sim
Fem.	Não

Associação fraca

Estado Civil	% inadimplentes
Solteiro	35%
Casado	8%
Divorciado	30%

Associação forte



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - Qui-quadrado – como apurar
 - Uma IA generativa pode realizar o teste para você.
 - Prompt sugerido: “Para a variável categórica da coluna X, apure sua força de associação com a classe que se encontra na coluna Y, trazendo o valor de V de Cramér”.
 - A classe **não** precisa estar representada com 0 / 1.
 - V de Cramér 0 – nenhuma associação
 - V de Cramér 1 – associação perfeita



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - **ANOVA** – avalia o poder discriminatório de uma *feature* numérica.

Aprovados	Reprovados
82	45
79	50
88	39
85	42

Notas de matemática X classe de aprovação =>

Possui forte caráter discriminatório, pois a diferença de médias é grande em cada classe.

Ou seja, esta feature numérica apresenta comportamento significativamente diferente entre as classes

Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - ANOVA – como apurar
 - Uma IA generativa pode realizar o teste para você.
 - Prompt sugerido: “Para a variável numérica da coluna X, avalie se existe diferença estatisticamente significativa entre suas médias nas diferentes classes da coluna Y, aplicando o teste ANOVA de uma via. Apresente o p-valor e o eta quadrado (η^2), informado se existe associação estatisticamente significativa entre a variável e a classe, considerando nível de significância de 5%”.



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - ANOVA – como apurar – *cont.*
 - O teste trará valores de p e η^2 , suficientes para nossa interpretação
 - Se $p > 0,05 \rightarrow$ a feature não apresenta associação estatisticamente significativa com a classe $\rightarrow$ marque como "Não significativa".
 - Se $p \leq 0,05 \rightarrow$ a feature é estatisticamente significativa $\rightarrow$ prossiga.
 - Para as features significativas, use η^2 (que varia de 0 a 1) para fazer o ranking:
 - η^2 maior $\rightarrow$ maior poder discriminatório
 - η^2 menor $\rightarrow$ menor poder discriminatório



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - ANOVA – como apurar – cont.
 - Exemplo:

Variável	p	η^2	Resultado
Valor_Compras	<0,001	0,24	 Mais discriminatória
Renda	<0,001	0,11	
Idade	0,024	0,04	
Nº_Filhos	0,37	0,002	 Não significativa



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - *Mutual information* – avalia quanto conhecer sobre uma variável reduz a incerteza sobre a classe.
 - Obs.: a Correlação procura relações lineares, enquanto que a Informação Mútua mede qualquer tipo de dependência, inclusive relações complexas e não lineares.
 - Ex.:
 - Idade ajuda muito a prever resistência física.
 - Número do apartamento não ajuda a prever resistência física.



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.*
 - *Information Gain* – avalia se a divisão de um conjunto com base em uma variável produz subconjuntos com menor entropia (mais homogêneos).
 - Trata-se do GI (Ganho de Informação) da segmentação supervisionada por árvores de decisão.



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 2 - Seleção estatística – *cont.* (resumo)

Método	Tipo de variável	O que mede	Quando usar
Correlação	Numéricas	Relação linear entre variáveis	Regressão e detecção de redundância
Qui-quadrado	Categóricas	Associação entre feature e classe	Classificação com atributos categóricos
ANOVA	Feature numérica + classe categórica	Diferença entre médias dos grupos	Classificação
Mutual Information	Numéricas e/ou categóricas	Qualquer tipo de dependência entre feature e classe	Seleção geral de features
Information Gain	Principalmente em Árvore de Decisão	Redução da entropia após uma divisão	Construção de árvores (ID3, C4.5, C5.0)



Feature Engineering

- Seleção de variáveis entre as candidatas – *cont.*
 - 3 - Seleção automática pelo algoritmo
 - Árvores de decisão / Random forests(*) naturalmente checam a importância das variáveis no caminho da predição da classe.
- (*) Veremos mais adiante a técnica.



Feature Engineering

- **Definição das variáveis**
 - Definir uma feature significa decidir:
 - Tipo da variável
 - Escala
 - Representação
 - Exemplo: variável “idade”
 - Pode ser:
 - 37 (não categórica)
 - Em faixas: 18-25, 26-40, 40-60
 - Outro exemplo: transformar data de nascimento em idade pode ser muito mais útil.



Feature Engineering

- Definição das variáveis – cont.
 - **Atenção!** Em problemas de classificação, é conveniente transformar variáveis numéricas em **faixas** (categorias), especialmente quando os valores possuem grande variedade e quando se deseja uma árvore mais simples e interpretável.
 - Variáveis numéricas contínuas (não categóricas) podem gerar muitos pontos de corte possíveis, aumentando demais a complexidade da árvore de classificação e, se não houver controle de profundidade ou poda, isto pode favorecer fortemente o sobreajuste.



Feature Engineering

- **Recombinação de variáveis**
 - Consiste em trocar um conjunto de variáveis por uma mais representativa e útil para o problema de negócio.
 - Exemplos:
 - A partir de “peso” e “altura”, criar “índice de massa corporal (IMC)”, onde $IMC = peso / altura^2$
 - A partir de “data da compra”, criar “dias desde a última compra”
 - A partir de “renda” e “quantidade de moradores”, criar “renda per capita”
 - A partir de “valor de compra” e “número de compras”, criar “ticket médio”



Feature Engineering

- **Transformação de variáveis**

- Em alguns casos, é importante mudar a representação de uma variável para que os modelos que a usam funcione melhor.

- Técnicas:

- Normalização (*)

$$x' = \frac{x - \mu}{\sigma}$$

- Criação de faixas (para variáveis não categóricas)

- *One-hot encoding* (*)

- Binarização (ex.: “maior de idade” SIM ou NÃO)

- Colocação em uma mesma escala / transformação logarítmica (para situações de muita assimetria de valores)

$$\text{Log Var} = \ln (1 + \text{Var})$$

(*) Vimos em Regressão Logística



Feature Engineering

- **Descarte de variáveis**
 - Consiste em desconsiderar variáveis que não têm ou que perderam o sentido.
 - Exemplos:
 - Descartar peso e altura, quando da criação do IMC.
 - Descartar data de nascimento e adotar idade.
 - Descartar valor total de vendas e número de vendas e adotar ticket médio.
 - Remover dos dados de treinamento variáveis que não influenciam em nada no problema de negócio.



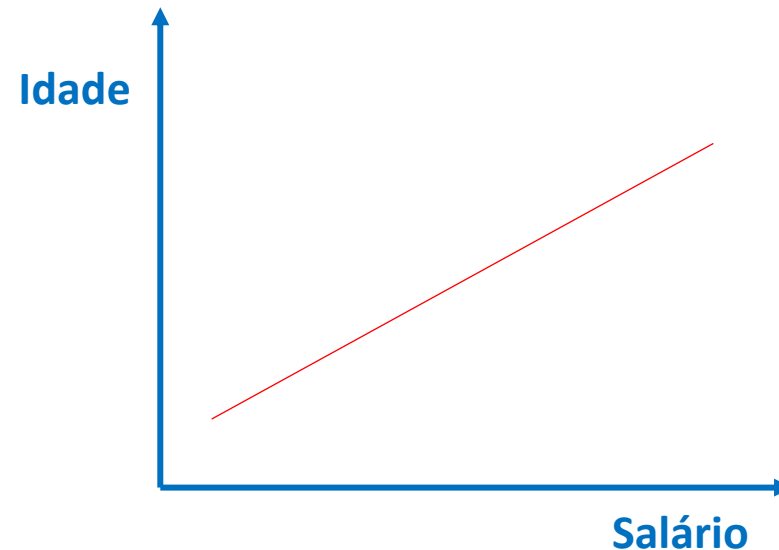
Feature Engineering

- **Partição da base de dados**
 - Consiste em dividir os dados em:
 - Treinamento – serão usados na modelagem.
 - Teste (retenção) – não participam da modelagem e serão usados para checar quanto a predição gerada pela modelagem se comporta bem com dados que até então não foram “vistos”.
 - Ou seja, é checado se o modelo generaliza bem para o mundo real (se ele está com pouco sobreajuste).



Feature Engineering

- **PCA (*Principal Component Analysis*)**
 - Trata-se de uma técnica de redução de dimensionalidade.
 - Em Machine Learning, cada variável é uma dimensão.
 - Duas features (por exemplo “idade” e “salário”) podem ser plotadas em um espaço então de duas dimensões.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont.
 - O problema com muitas dimensões: quanto maior o número de variáveis:
 - Maior o tempo de treinamento
 - Maior o consumo de memória
 - Maior o risco de sobreajuste
 - Maior a quantidade de dados necessária para treinar bem o modelo
 - Maior a chance de existirem variáveis redundantes ou irrelevantes
 - Esse fenômeno é conhecido **como maldição da dimensionalidade** (*Curse of Dimensionality*).



Feature Engineering

- PCA (*Principal Component Analysis*) – cont.
 - Para reduzir as dimensões, lançamos mão de:
 - Seleção de variáveis (escolhemos as que podem ser mais relevantes para o problema de negócio, **eliminando as inúteis**).
 - Extração de variáveis – aqui entra o PCA.
 - No PCA, reduzimos as dimensões mantendo grande parte da informação, pois as variáveis reduzidas (trocadas por uma que as agrupa) possuem correlação alta.
 - As variáveis suprimidas, no PCA, **implicam na criação de uma nova dimensão que passa a representá-las**. É diferente então da seleção de variáveis.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 1 de PCA – alunos e desempenho escolar
 - Consideremos o quadro de notas de alunos em disciplinas:

Aluno	Mat.	Física	Química	Estat.
A	90	88	92	85
B	80	82	78	84
C	50	55	48	52
D	40	42	45	38

- Perceba que as quatro notas tendem a subir e descer juntas.
- Existe uma dimensão subjacente que poderíamos chamar de “desempenho geral em Exatas”.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 2 de PCA – características físicas
 - Podemos agrupar as dimensões:
 - Peso
 - Altura
 - IMC
 - Circunferência abdominal
 - Percentual de gordura
 - Massa muscular

em “composição corporal”, calculada de alguma forma.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 3 de PCA – clientes de um banco
 - Há várias dimensões em jogo, que podem estar correlacionadas:
 - Renda
 - Patrimônio
 - Limite do cartão
 - Valor dos investimentos
 - Gastos mensais
 - Número de cartões
 - O PCA pode descobrir que grande parte da variação dos clientes está relacionada a uma dimensão que poderíamos considerar como “poder financeiro”.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 4 de PCA – imagens
 - Imagine uma imagem em preto e branco de 28×28 pixels.
 - Cada pixel é uma variável. Portanto, temos $28 \times 28 = 784$ variáveis.
 - Uma imagem de um número "7", por exemplo, seria representada por 784 valores. Há então 784 dimensões.
 - O PCA pode transformar essas 784 variáveis em, por exemplo, 50 que consigam representar grande parte das características necessárias para distinguir as imagens de um número.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 5 de PCA – sensores industriais
 - Considere o conjunto de grandezas medidas:
 - Temperatura
 - Pressão
 - Ruído
 - Corrente elétrica
 - Tensão
 - Se, por exemplo, temperatura, pressão e ruído estiverem muito correlacionados, poderão motivar uma redução para uma dimensão “energia do fluido circulante”.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - Exemplo 6 de PCA – mercado financeiro
 - Imagine que temos cotações dos valores das ações de:
 - Petrobras
 - Vale
 - Itaú
 - Bradesco
 - Banco do Brasil
 - Weg
 - Ambev
 - Etc.
 - Podemos agrupar valores por setores, reduzindo assim as dimensões.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - PCA – como avaliar uma redução de dimensões
 - Para reduzir um conjunto de N variáveis para X dimensões ($X < N$), apuramos os chamados PCAs (PCA-1, PCA-2, PCA-3, PCA-4 etc.).
 - Cada PCA trará uma variância. Apurando a variância acumulada, quando esta for acima de 80% teremos a indicação do número de dimensões reduzidas X adequada (a variância acumulada de 80% ou mais foi conseguida em PCA- X).



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - PCA – como avaliar uma redução de dimensões
 - Exemplo: para um conjunto de N variáveis onde se deseja redução

Componente	Variância	Acumulada
PCA-1	50%	50%
PCA-2	25%	75%
PCA-3	12%	87%
PCA-4	6%	93%
PCA-5	3%	96%

- Nesse caso, PCA-3 seria uma escolha bastante razoável (redução para três dimensões através de novas variáveis obtidas a partir da combinação matemática das N originais), pois preserva 87% da variância.



Feature Engineering

- PCA (*Principal Component Analysis*) – cont. (exemplos)
 - PCA – como avaliar uma redução de dimensões – *cont.*
 - Uma IA Generativa poderá nos ajudar a apurar os PCAs
 - Sugestão de prompt; “Para o conjunto de variáveis < citar o conjunto >, apure os PCAs (de 1 a 10), mostrando para cada um a variância e a variância acumulada”.
 - Com base no resultado acima, encontramos a redução de dimensionalidade razoável, conforme vimos.



Exercício: preparação de dados com *Feature Engineering*



**Módulo 4 – Construção e
validação de modelagens**

Construção dos modelos

- Utilizando as técnicas de modelagem preditiva, pode ser interessante:
 - realizarmos diferentes modelagens para um mesmo problema (por exemplo: árvore de decisão e regressão logística) e
 - em cada modelagem, gerarmos diferentes versões (por exemplo, com variáveis diferentes ligeiramente diferentes em cada treinamento, diversas profundidades de árvore, diversos graus de polinômio de regressão etc.).
- Cada modelagem deve ser avaliada na sua robustez contra o sobreajuste e no seu desempenho em relação aos devidos indicadores de eficácia.



Sobreajuste

- O **sobreajuste** envolve o fato de um modelo apresentar alguns padrões interessantes de dados, mas que não se generalizam.
- Generalizações inadvertidas são armadilhas traiçoeiras e comuns.
- Muitos padrões interessantes apontados por um modelo são, na verdade, situações aleatórias ou muito particulares, que não deveriam induzir regras para grupos maiores.



Sobreajuste

- **Generalização** = aplicabilidade de um modelo a dados (exemplos) que não foram utilizados para construí-lo.
- Um modelo pode funcionar perfeitamente para o conjunto de dados de treinamento e não ser bom para dados novos – diz-se que tal modelo é sobreajustado (não generaliza bem).
- Não se trata aqui de os dados de treinamento não constituírem uma amostra representativa – o problema está na mineração de dados usada para construir o modelo.



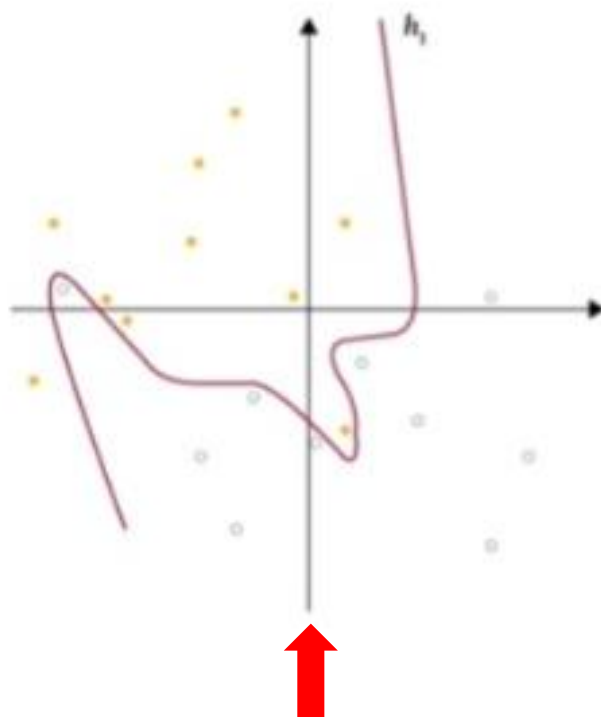
Sobreajuste

- No sobreajuste, a mineração de dados avança demais na adaptação aos dados de treinamento, em detrimento da aderência a uma generalização.
- O sobreajuste sempre estará presente de alguma forma, e o que podemos fazer é trabalhar para identificá-lo e minimizá-lo.

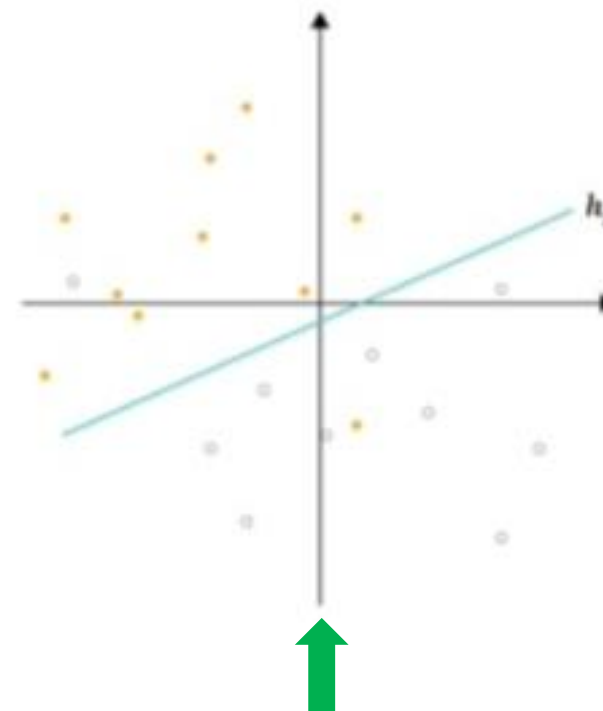


Sobreajuste

- Sobreajuste em separadores



Modelo com sinal de sobreajuste



Modelo mais generalizável

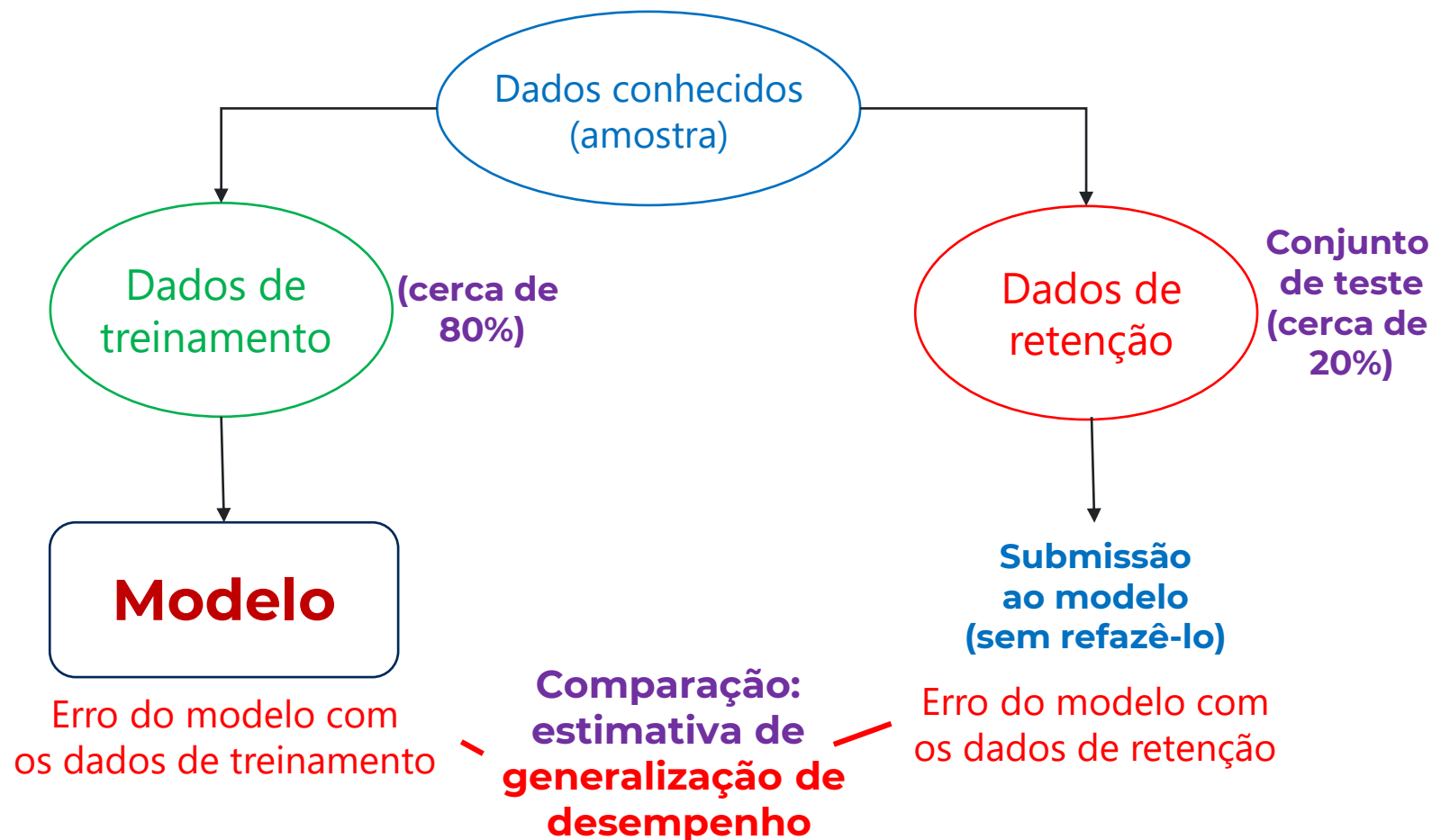
Sobreajuste

- **Dados de retenção (*holdout data*)**
 - São dados com valores da variável alvo conhecidos, mas que não vamos usar para construir o modelo.
 - Estes dados ficarão “separados” para serem usados em uma espécie de “teste de laboratório” para o modelo.
 - Os dados de retenção ficam ocultos ao modelo e serão usados depois do mesmo estar pronto para ver quanto o modelo funciona com eles.
 - Os dados de retenção funcionam como uma massa de teste à qual o modelo não se ajustou (o ajuste ocorreu com os dados de treinamento).



Sobreajuste

- Uso dos dados de retenção



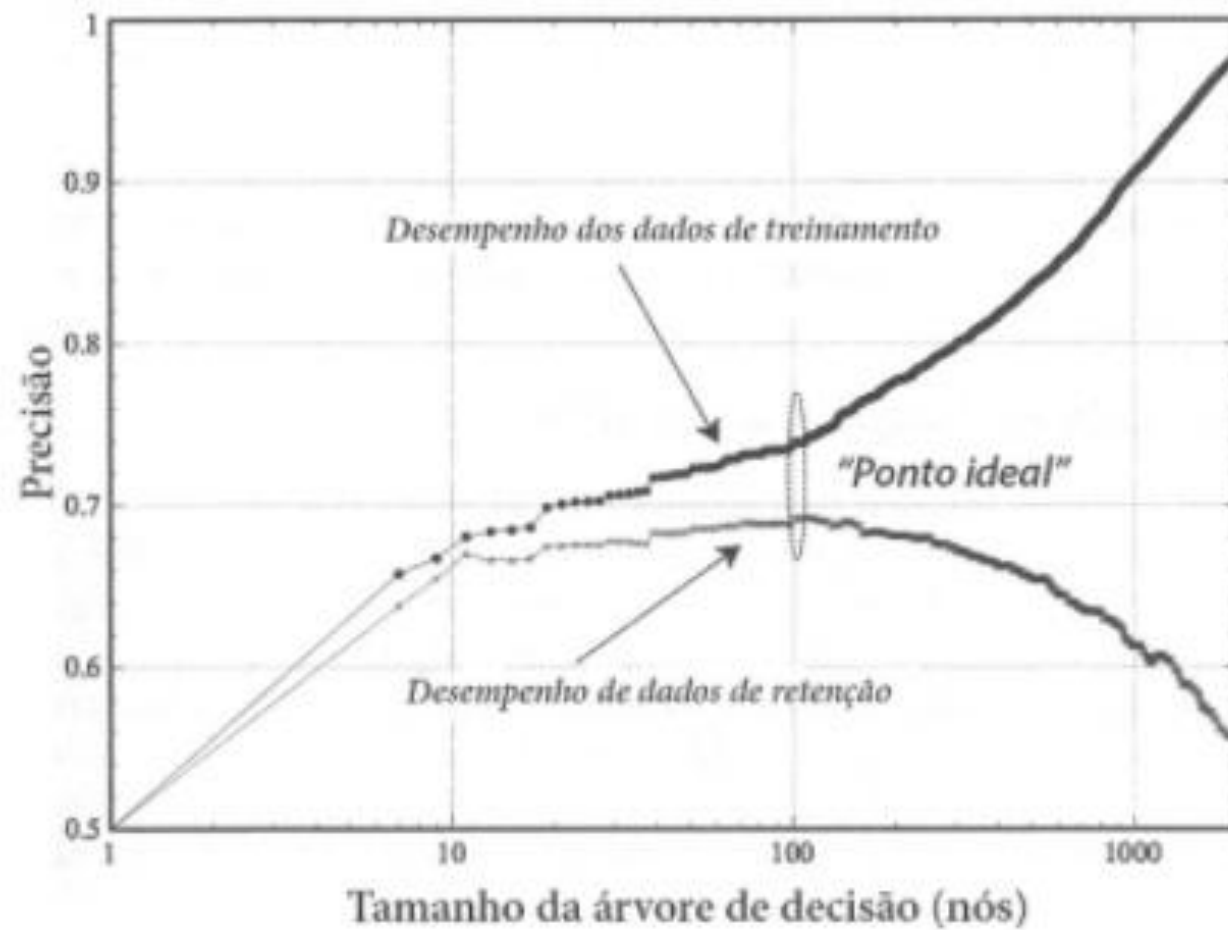
Sobreajuste

- **Comparação do modelo com os dados de treinamento e com os dados de retenção em árvores de classificação**
 - **Criamos a árvore com os dados de treinamento (fazendo as subdivisões com base nestes dados).**
 - **Apuramos o erro deste modelo comparando sua previsão com os valores reais dos dados de treinamento.**
 - **Para os dados de retenção, não refazemos a árvore com base neles (isto geraria uma nova árvore que se adaptaria só aos dados de retenção). O que devemos fazer é submeter os dados de retenção à árvore feita.**



Sobreajuste

- Ponto de melhor ajuste – árvores de classificação



Sobreajuste

- **Taxa base**

- É proporção de uma classe específica em um conjunto de dados.
- Exemplo:
 - Se um conjunto de dados tem 100 amostras e 80 delas pertencem à Classe A e 20 à Classe B, a taxa base para a Classe A é 80% e para a Classe B é 20%.



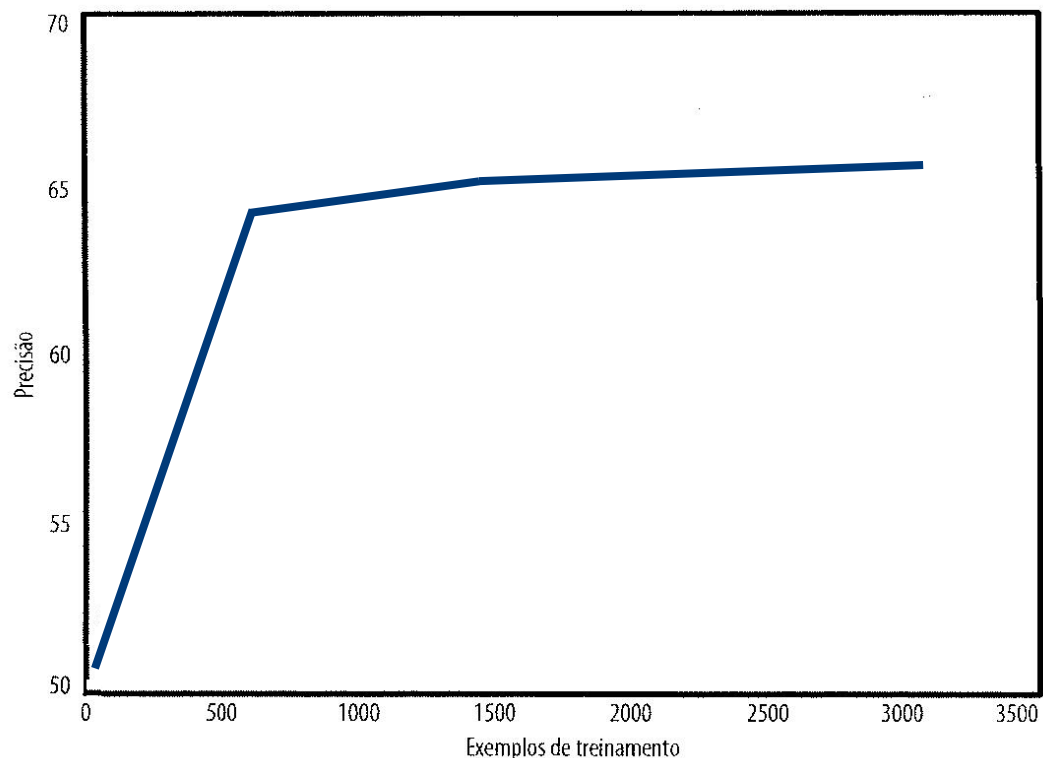
Sobreajuste

- Taxa base – *cont.*
 - Ao avaliar um modelo de classificação, é importante saber a taxa base.
 - Se o modelo simplesmente prevê a classe mais frequente (no exemplo dado, a Classe A), sua acurácia seria de 80%.
 - Assim, um modelo que obtém uma acurácia inferior a essa taxa base pode não estar aprendendo nada útil e pode ser considerado ineficaz.



Sobreajuste

- Com volume maior de dados de treinamento, a precisão do modelo aumenta, mas isto tende a se estabilizar (observamos isto nas chamadas curvas de aprendizado).



Sobreajuste

- **Sobreajuste em indução com árvores de decisão**
 - O desafio destas árvores, como vimos, é que elas tendem a crescer até chegar a nós puros, ajustando sobremaneira o modelo aos dados de treinamento.
 - Estratégias de combate ao sobreajuste:
 - Interromper o crescimento da árvore antes que se torne muito complexa.
 - Deixar a árvore crescer ao máximo e então “podá-la”, reduzindo seu tamanho e complexidade.



Sobreajuste

- **Controlando o crescimento da árvore**
 - Podemos estipular um número mínimo de exemplos que uma folha pode conter, evitando mais segmentações no ramo.
 - Outras ideias:
 - Construir árvores com diferentes complexidades (níveis de segmentação) e, neste conjunto, apurar qual a árvore com maior desempenho em generalização.
 - Pegar um conjunto de teste e dividi-lo novamente em um conjunto de treinamento (conjunto de subtreinamento) e outro de teste (conjunto de validação) – trata-se da retenção aninhada.



Sobreajuste

- Outras ideias – *cont.*
 - Descobrir o tamanho do modelo com melhor desempenho (com separação de conjunto de retenção) e depois refazer a árvore com todos os dados (treinamento + retenção) até atingir o tamanho descoberto.
 - Jogar com diferentes conjuntos de atributos informativos no conjunto de treinamento de partida e ir descobrindo (e deixando de lado) os que contribuem menos.



Sobreajuste

- **Sobreajuste em funções matemáticas**
 - No caso de modelos paramétricos, a complexidade cresce com a introdução de mais variáveis e/ou com introdução de aspectos não lineares complicando a fórmula.
 - Também aqui, uma curva pode estar ajustada demais aos dados de treinamento.
 - Regularização: busca de uma combinação (otimização) entre ajuste aos dados e simplicidade do modelo.



Combate ao sobreajuste

- O sobreajuste pode ser combatido com:
 - Poda das árvores de decisão (gerar versões que não chegam até o nível mais baixo possível).
 - Ajuste no grau do polinômio de regressão.
 - Uso de dados de retenção / validação cruzada.
 - Aumento da quantidade e a da qualidade dos dados de treinamento.
 - Utilização de métodos de conjunto (*ensemble*), como a Floresta Aleatória (*Random Forest*), que reduz a variância ao combinar diversas árvores construídas sobre amostras e subconjuntos aleatórios de atributos. Veremos no Módulo 5.



Avaliação de modelos

- Avaliação de classificadores
 - Primeira ideia: aplicar em dados de retenção.
 - Acurácia:

$$\text{Acurácia} = \frac{\text{Número de decisões corretas tomadas}}{\text{Número total de decisões tomadas}}$$

$$= 1 - \text{taxa de erro}$$

- A acurácia por si só pode ser simplista na avaliação de modelos, como veremos mais adiante.



Avaliação de modelos

- Confusão de classe / **matriz de confusão**
 - Uma matriz de confusão mostra as quantidades de positivos/negativos verdadeiros/falsos nas decisões tomadas pelo classificador.
 - Positivo = valor da classe merecedor de atenção.
 - Negativo = valor da classe não merecedor de atenção.
 - Exemplo: exame médico positivo / negativo para uma infecção (classe binária: reagente = positivo / não reagente = negativo).



Avaliação de modelos

- Matriz de confusão (exemplo com duas classes)

		Classes de fato do exemplo	
		Positivo	Negativo
Classes previstas pelo modelo	Sim, é positivo	Verdadeiros positivo	Falsos positivo
	Não, é negativo	Falsos negativo	Verdadeiros negativo

Diagonal principal = decisões corretas do modelo



Avaliação de modelos

- **Classe desequilibrada**

- É uma classe incomum ou rara, o que torna a distribuição dos exemplos entre as classes desequilibrada ou distorcida.
- É uma situação comum em detecção de fraudes, por exemplo (operação fraudulenta “sim” ou “não”), onde a maioria das operações não são fraudes.
- A taxa base para a classe majoritária é alta.



Avaliação de modelos

- Classe desequilibrada – *cont.*
 - Nestes casos, a acurácia do modelo pode ser enganosa (é a coisa errada a ser medida).
 - Exemplo:
 - Em uma população, sabemos que a taxa base para “sim” é de 97% e nos interessa capturar os 3% que são da classe “não”.
 - Neste caso, se classificarmos todos como “sim”, a precisão será de 97% mas, apesar deste número ser alto, não nos ajuda em nada em nosso interesse (que é de detectar os “não”).



Avaliação de modelos

- Custos e benefícios desiguais
 - Outro problema a enfrentar na avaliação de modelos é que a importância dos falsos positivos e dos falsos negativos pode ser diferente.
 - Exemplo:
 - Diagnóstico de câncer falso positivo (paciente informado com a doença quando na verdade ele não a tem) – resultado estressante, inconveniente, que exigiria ações, mas não fatal.
 - Diagnóstico de câncer falso negativo (paciente informado sem a doença quando na verdade ele a tem) – resultado que afastaria o paciente de uma detecção precoce, o que seria uma situação muito pior.



Avaliação de modelos

- **Valor esperado**
 - É uma média ponderada dos valores de diferentes resultados possíveis, onde a ponderação é dada pela probabilidade de ocorrência.

$$VE = p(o_1)*v(o_1) + p(o_2)*v(o_2) + ... + p(o_n)*v(o_n)$$



Avaliação de modelos

- Valor esperado para usar um classificador
 - Consideremos um modelo preditivo que dá uma estimativa de probabilidade de um exemplo com vetor de características x pertencer a uma classe “dos que respondem a uma oferta”.
 - A probabilidade de x responder é $p_R(x)$, o valor obtido de uma resposta é v_R e o valor obtido por nenhuma resposta é v_{NR} .
 - Devemos decidir se direcionamos a oferta para um consumidor com vetor de características x .

Benefício esperado do direcionamento =

$$p_R(x) * v_R + \underbrace{[1 - p_R(x)]}_{p_{NR}(x)} * v_{NR}$$



Avaliação de modelos

- Valor esperado para usar um classificador

$$\text{Benefício esperado do direcionamento} = p_R(x) * v_R + [1 - p_R(x)] * v_{NR}$$

Digamos que se um cliente comprar o produto lucraremos 100,00 e que o envio da oferta custa 1,00 (custo de envio de um material de marketing).

Valerá a pena enviar a oferta se o benefício esperado do direcionamento > 0 .

Ou seja:

$$p_R(x) * \underbrace{(100,00 - 1,00)}_{\text{Resultado de uma resposta}} + [1 - p_R(x)] * \underbrace{(0 - 1,00)}_{\text{Resultado de uma não resposta}} > 0$$



Avaliação de modelos

- Valor esperado para usar um classificador

$$p_R(x) \cdot (100,00 - 1,00) + [1 - p_R(x)] \cdot (0 - 1,00) > 0$$

$$p_R(x) \cdot 99 + [1 - p_R(x)] \cdot (-1) > 0$$

$$p_R(x) \cdot 99 - 1 + p_R(x) > 0$$

$$p_R(x) \cdot 100 - 1 > 0$$

$$p_R(x) \cdot 100 > 1$$

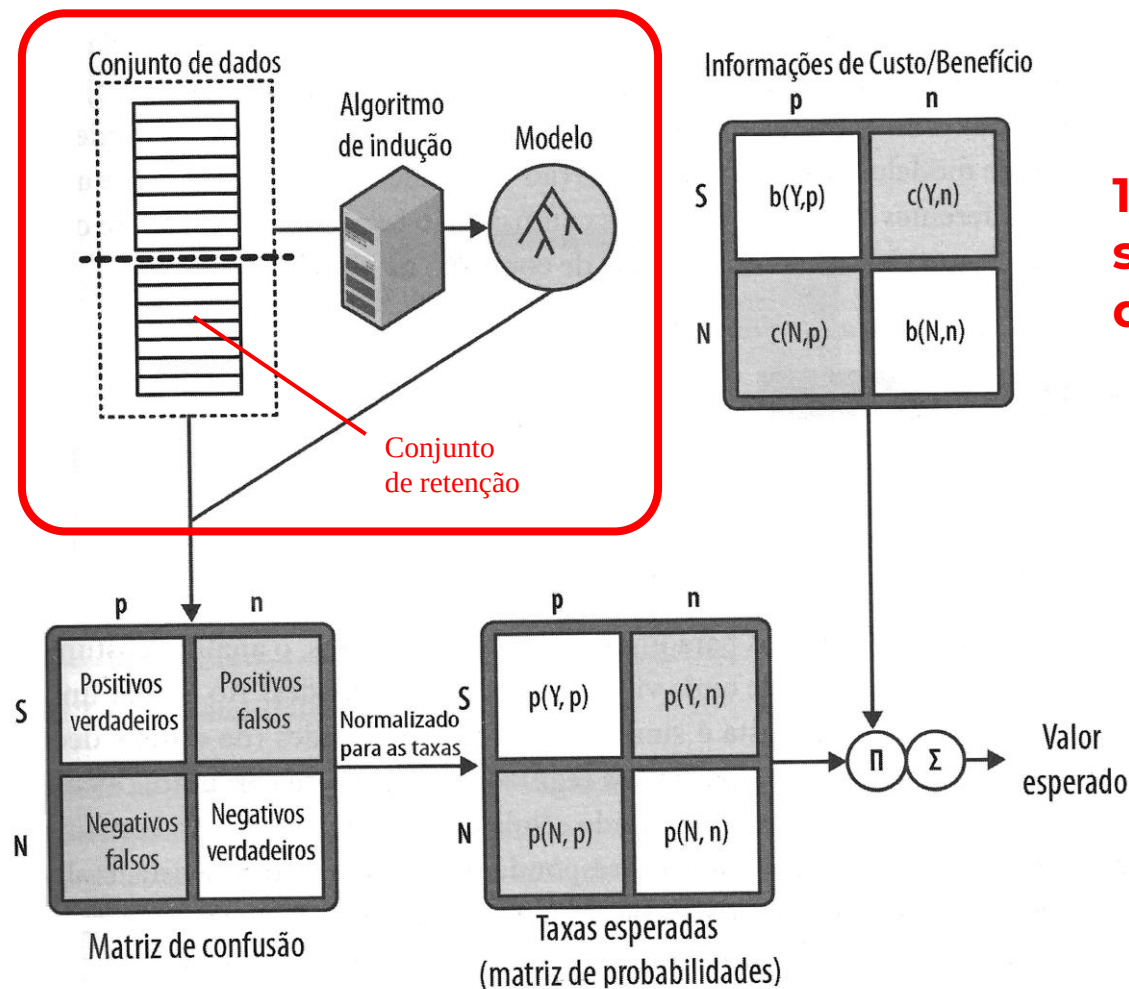
$$p_R(x) > 1/100$$

Portanto, devemos direcionar a oferta ao consumidor x se a probabilidade de ele responder for maior que 1%



Avaliação de modelos

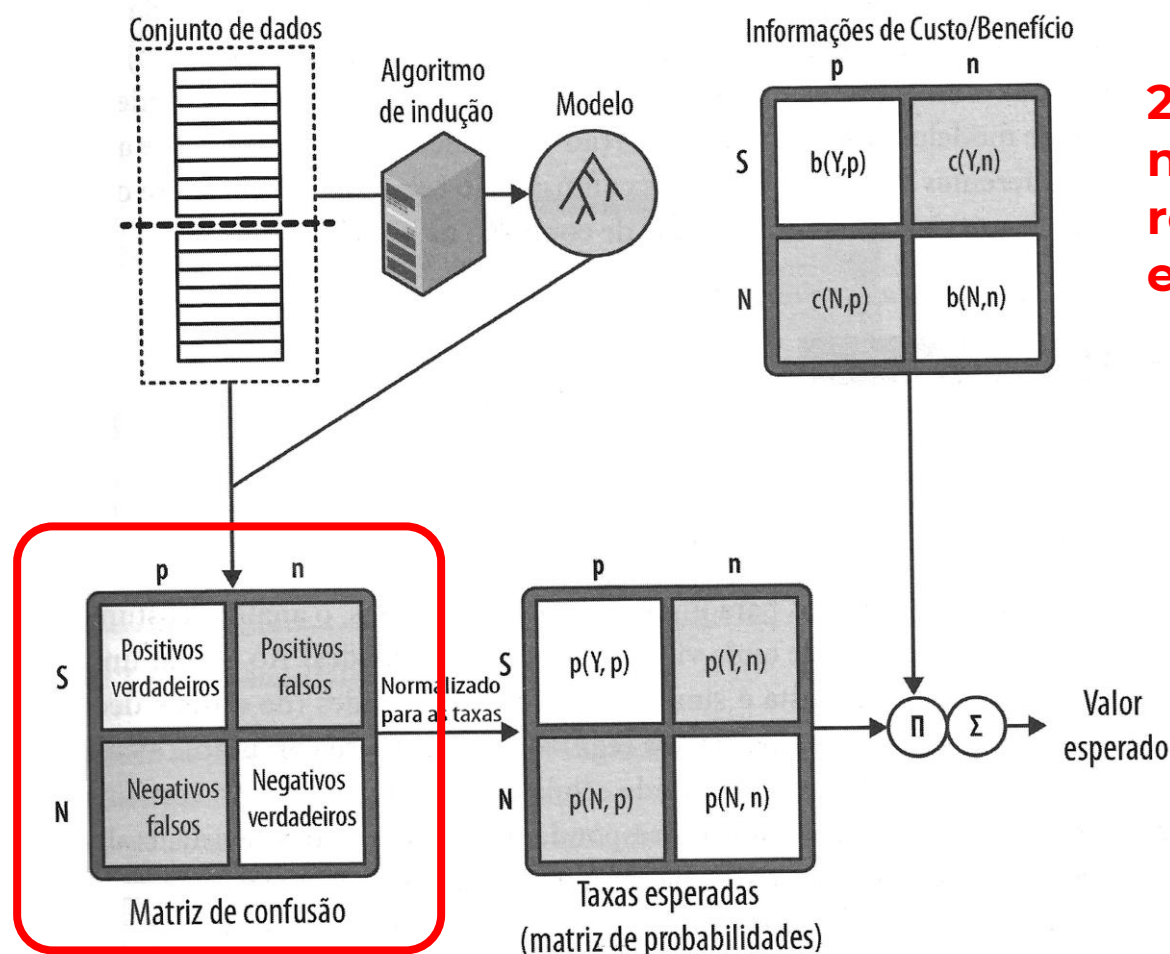
- Valor esperado para usar um classificador



1 - Geramos um modelo, separando um conjunto de dados de retenção.

Avaliação de modelos

- Valor esperado para usar um classificador



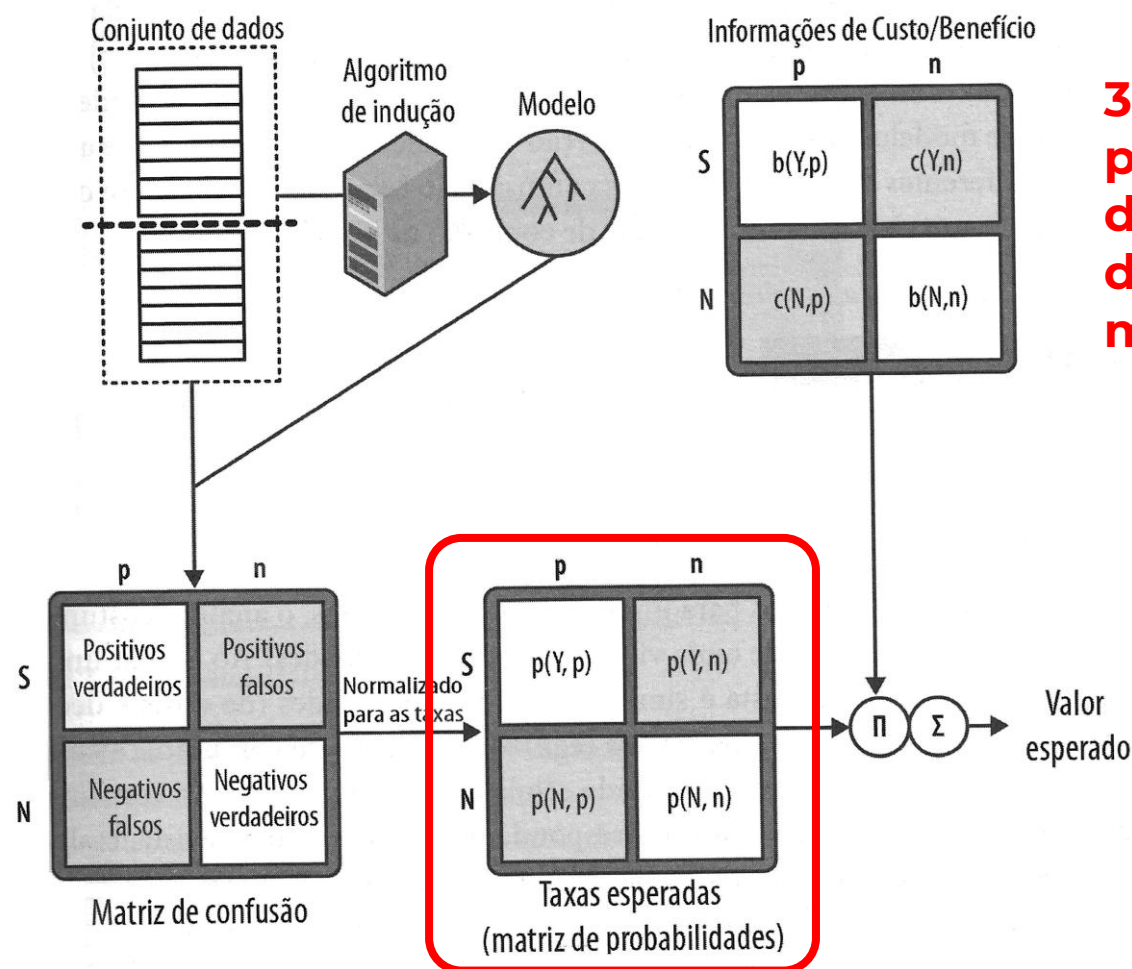
2 – Aplicamos o modelo nos dados de retenção, registrando o resultado em uma matriz de confusão.

Suponhamos que o conjunto de retenção tenha 110 exemplos e que a matriz de confusão resultante tenha ficado assim:

	p	n
S	56	7
N	5	42

Avaliação de modelos

- Valor esperado para usar um classificador



3 - Agora, apuramos as probabilidades estimadas de cada valor da matriz de confusão, chegando na matriz de probabilidades.

No nosso exemplo:

$$56 / 110 = 0,51$$

$$7 / 110 = 0,06$$

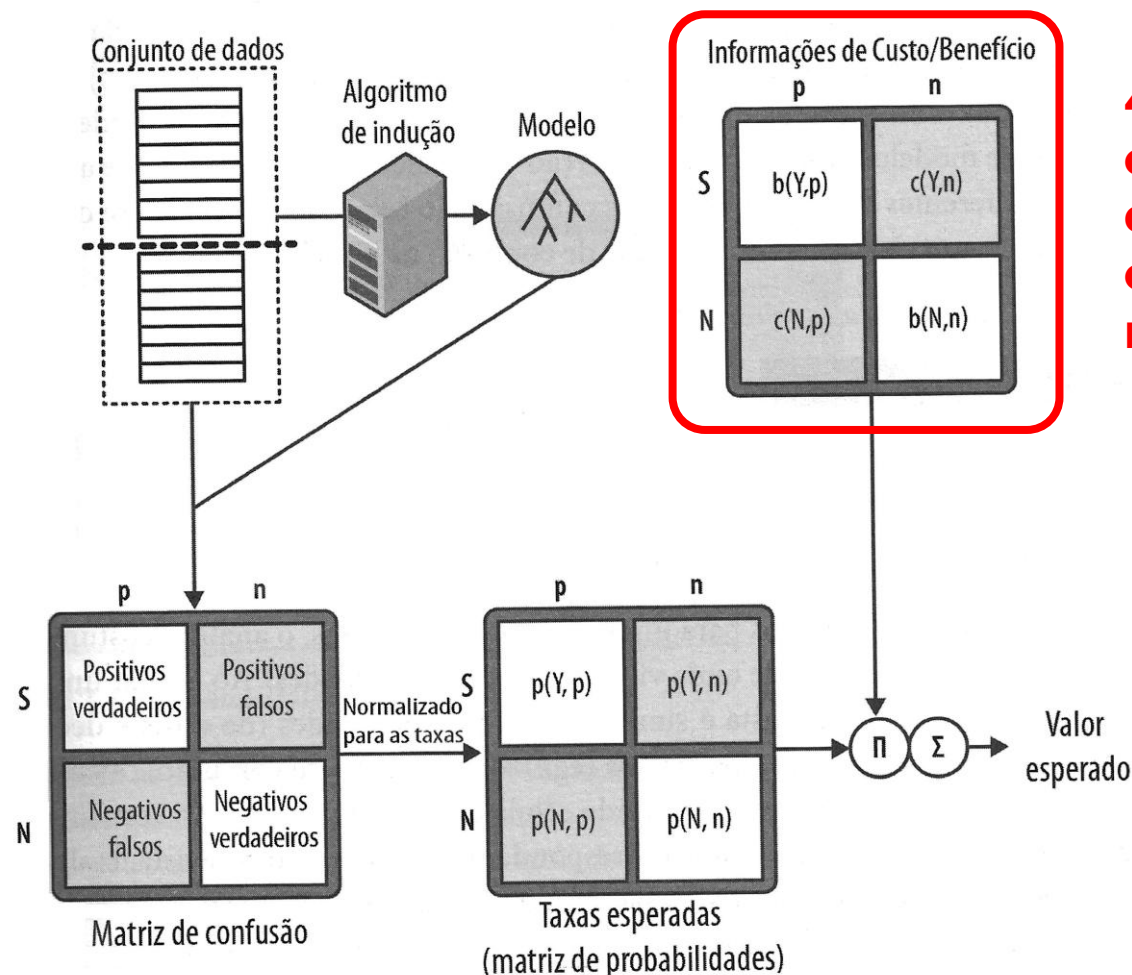
$$5 / 110 = 0,05$$

$$42 / 110 = 0,38$$

	p	n
S	0,51	0,06
N	0,05	0,38

Avaliação de modelos

- Valor esperado para usar um classificador



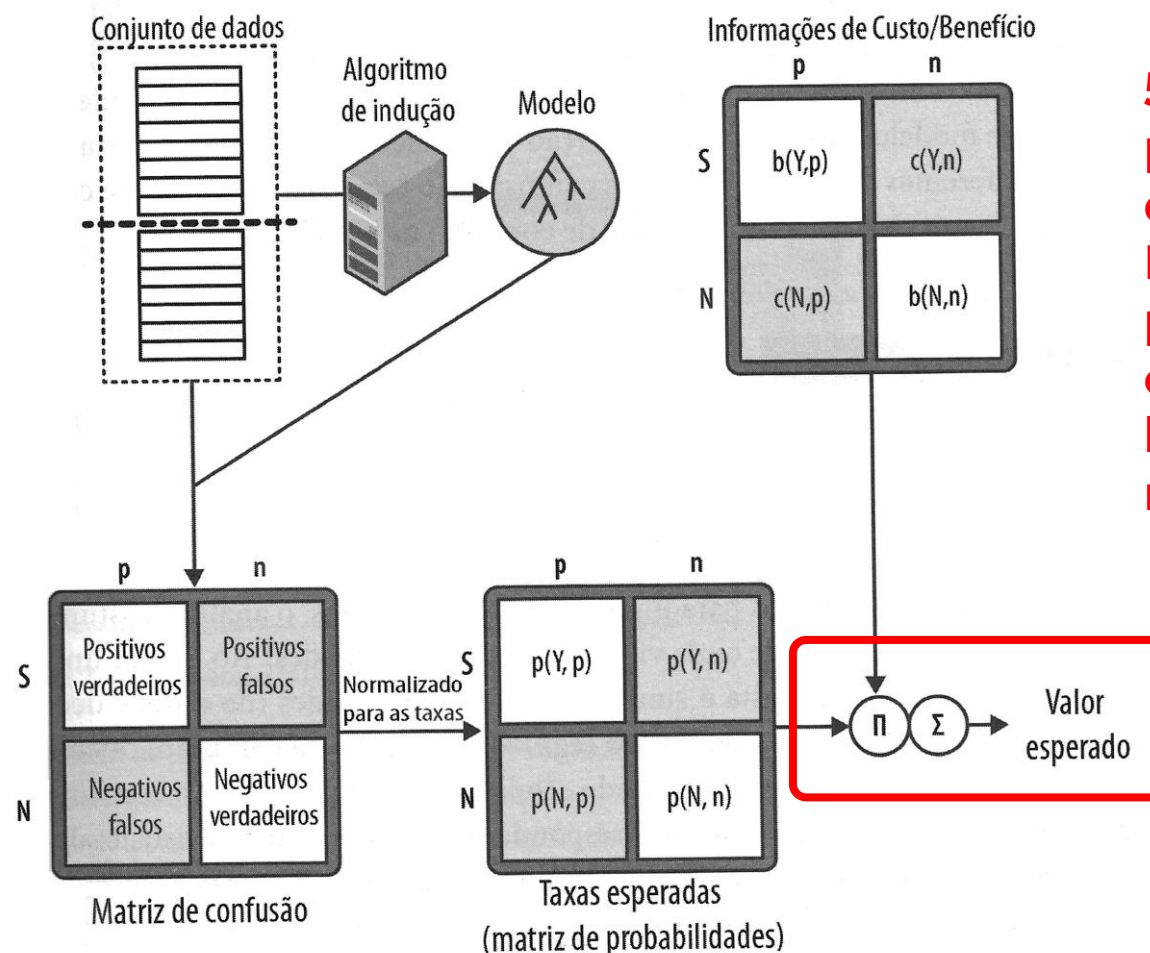
4 – Os custos / benefícios de cada situação da matriz de confusão são definidos, com base em aspectos de negócio.

Custos = valores negativos.

Benefícios = valores positivos.

Avaliação de modelos

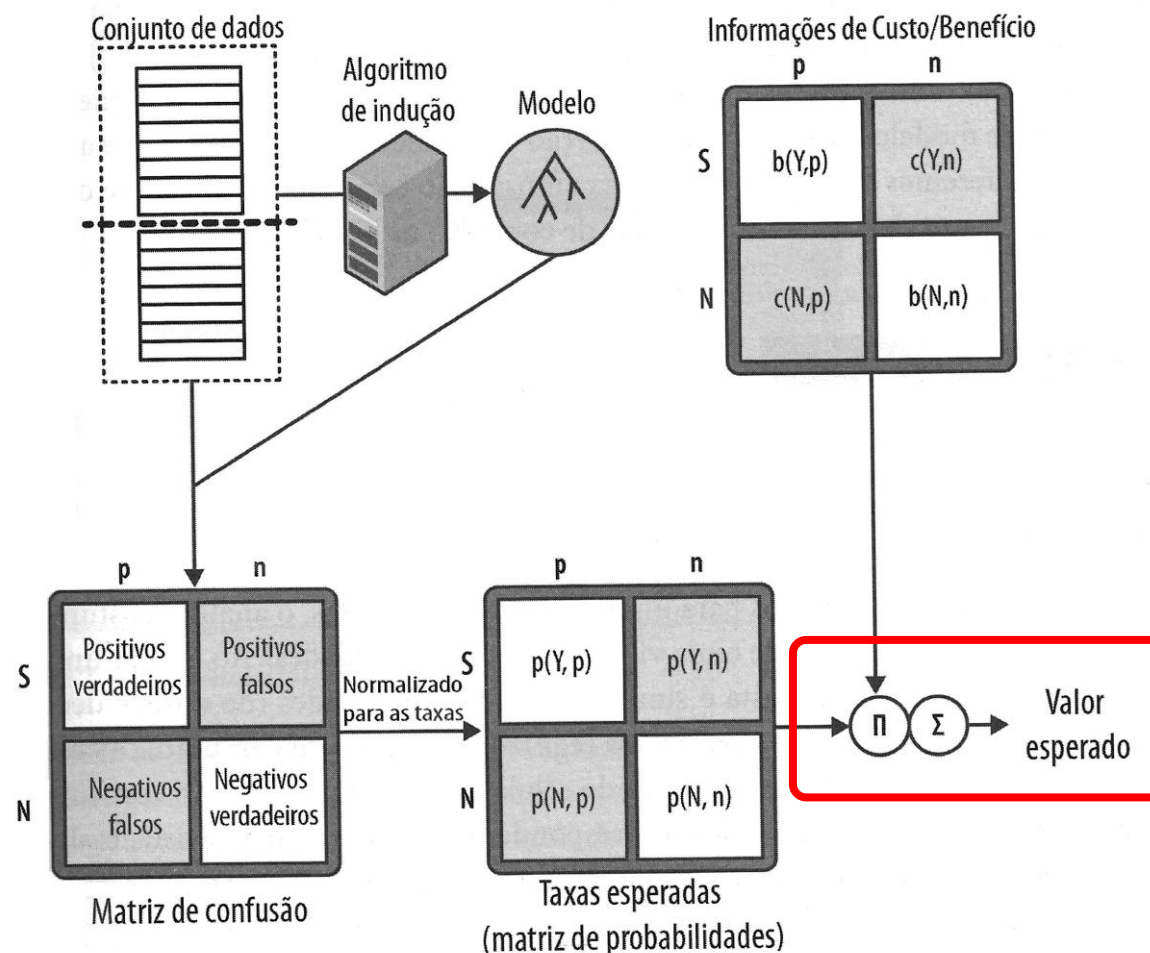
- Valor esperado para usar um classificador



5 – Finalmente então podemos aplicar a fórmula do valor esperado. Multiplicaremos cada probabilidade por seu correspondente custo / benefício, totalizando o resultado.

Avaliação de modelos

- Valor esperado para usar um classificador



Apuramos o valor esperado para várias versões de modelos, comparando-os e chegando então no classificador mais vantajoso.

Avaliação de modelos

- Outras métricas fundamentais de avaliação de modelos

	p	n
s	VP	FP
N	FN	VN

Taxa de verdadeiros positivos = $VP / (VP + FN)$
(conhecida também por **Sensibilidade** ou **Recall**)

Recall: situação de classes altamente desbalanceadas

Taxa de falsos negativos = $FN / (VP + FN)$

Taxa de verdadeiros negativos = $VN / (FP + VN)$
(conhecida também por **Especificidade**)

Taxa de falsos positivos = $FP / (FP + VN)$

Avaliação de modelos

- Outras métricas fundamentais de avaliação de modelos

	p	n
s	VP	FP
n	FN	VN

$$\text{Precisão} = \text{VP} / (\text{VP} + \text{FP})$$

Precisão: situação de classes altamente desbalanceadas

$$\text{Acurácia} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{VN} + \text{FP} + \text{FN}}$$

Acurácia: situação de classes balanceadas

$$\text{F1-score} = 2 * \frac{\text{Precisão} * \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}}$$

F1-score: situação de classes moderadamente desbalanceadas

Avaliação de modelos

- Uso das métricas de avaliação de modelos

Cenário	Métrica Prioritária	Justificativa
Diagnóstico de doença grave	Alta Sensibilidade (Recall)	Prioriza não deixar passar casos positivos reais, mesmo que aumentem falsos positivos.
Filtro de spam	Alta Precisão (Precision)	Importa não classificar e-mails legítimos como spam, mesmo que alguns spams passem.
Deteção de fraude	F1-score	Busca equilíbrio entre detectar fraudes (recall) e evitar falsos alarmes (precisão).
Classificação de imagens balanceadas (ex: tipos de frutas, animais, etc.)	Acurácia	Útil quando as classes estão equilibradas e erros têm o mesmo peso — mede a proporção total de acertos.

Atenção: **Recall** e **Precisão** aplicam-se a classes altamente desbalanceadas, mas em cenários diferentes!



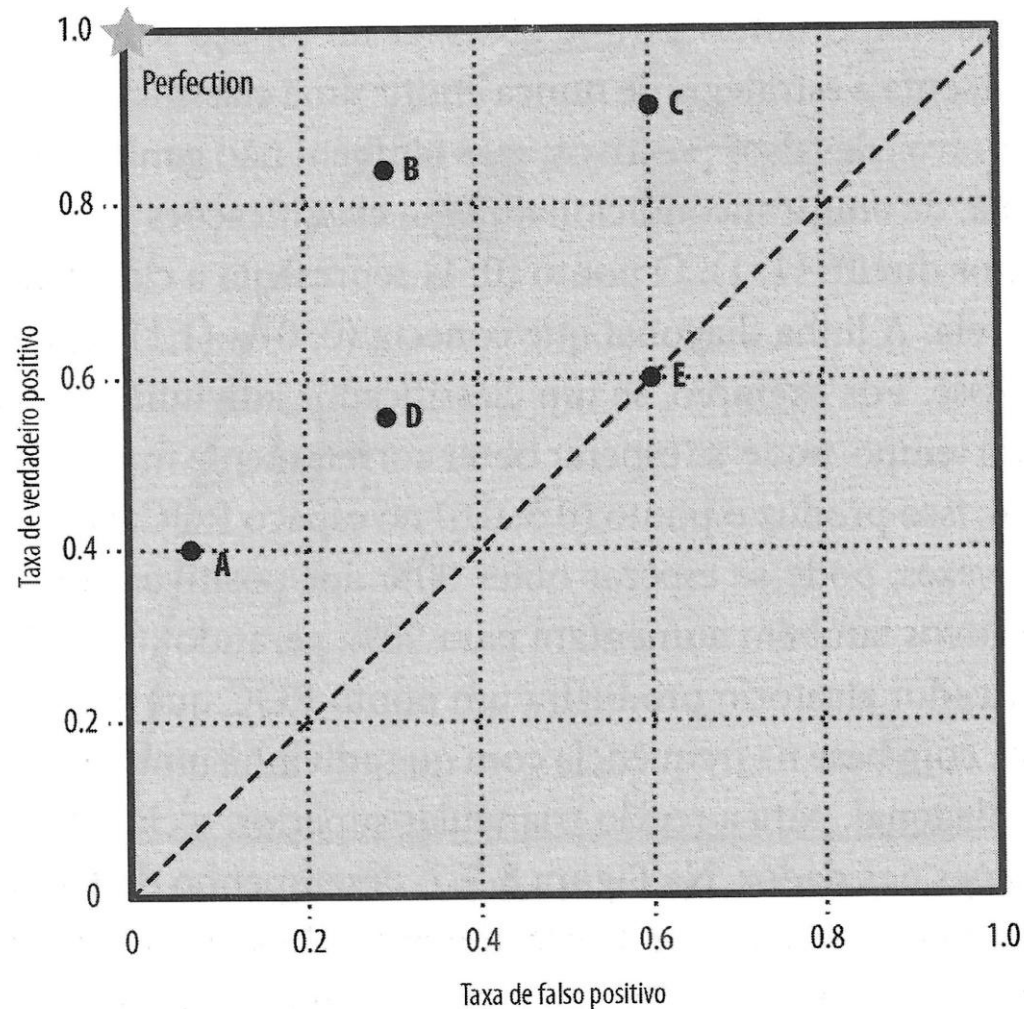
Avaliação de modelos

- Base de comparação para modelos
 - Vimos como comparar modelos entre si, mas é importante termos uma base razoável para comparar os modelos com ela.
 - Uma possibilidade é simular um modelo completamente aleatório e apurar seu desempenho, embora ser superior a um modelo assim possa não ser interessante nem informativo.



Avaliação de modelos

Gráfico ROC (Receiver Operating Characteristic)



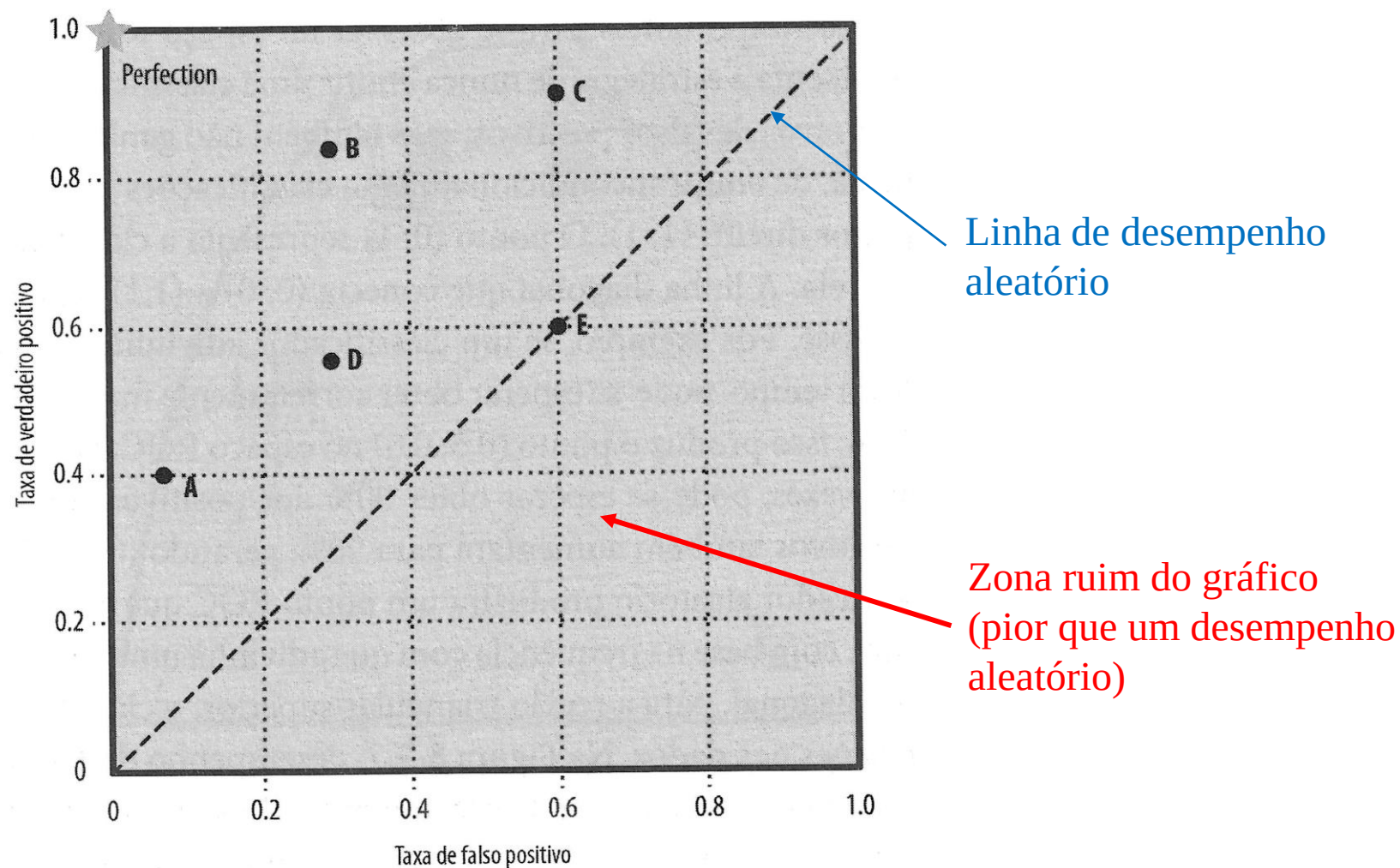
Os pontos de A a E são os classificadores em avaliação, cada um com sua matriz de confusão.

	p	n
s	VP	FP
N	FN	VN

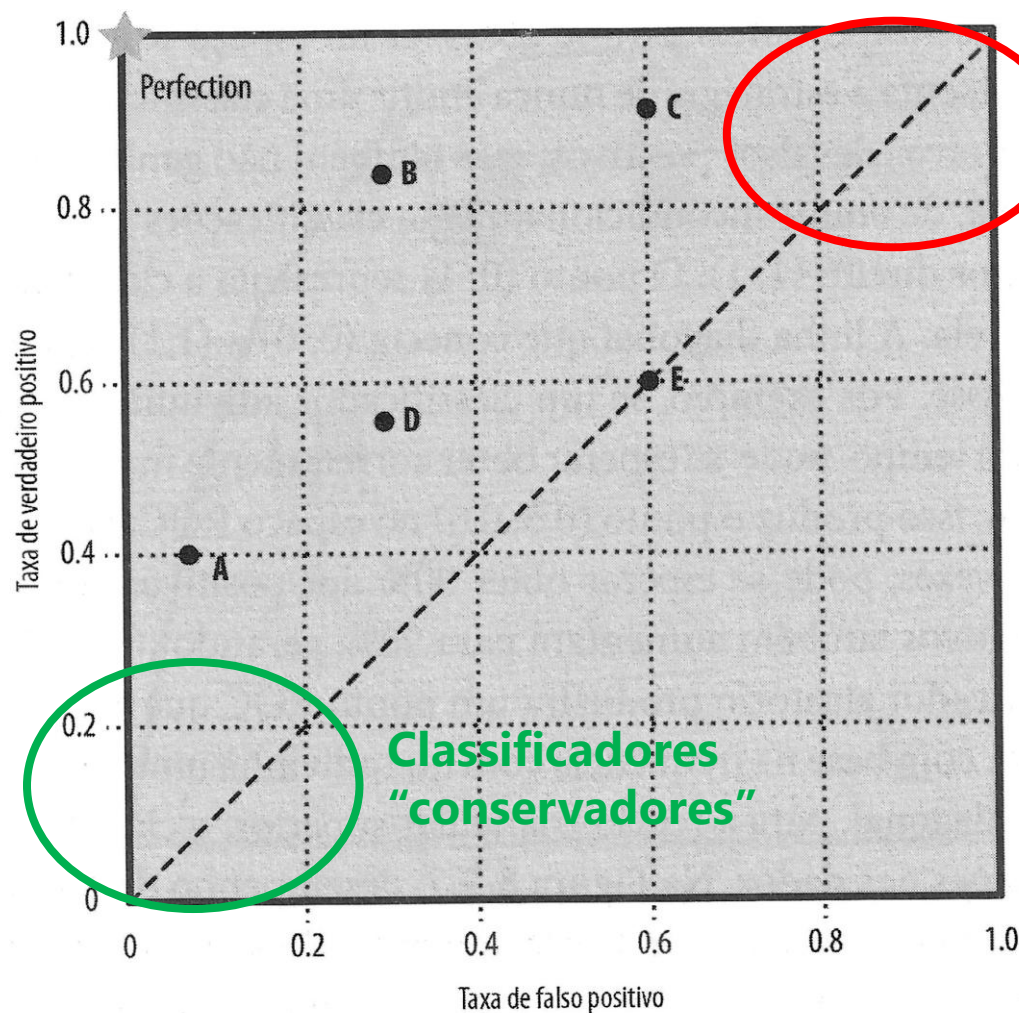
Taxa de verdadeiros positivos
 $= VP / (VP + FN)$

Taxa de falsos positivos
 $= FP / (FP + VN)$

Avaliação de modelos



Avaliação de modelos



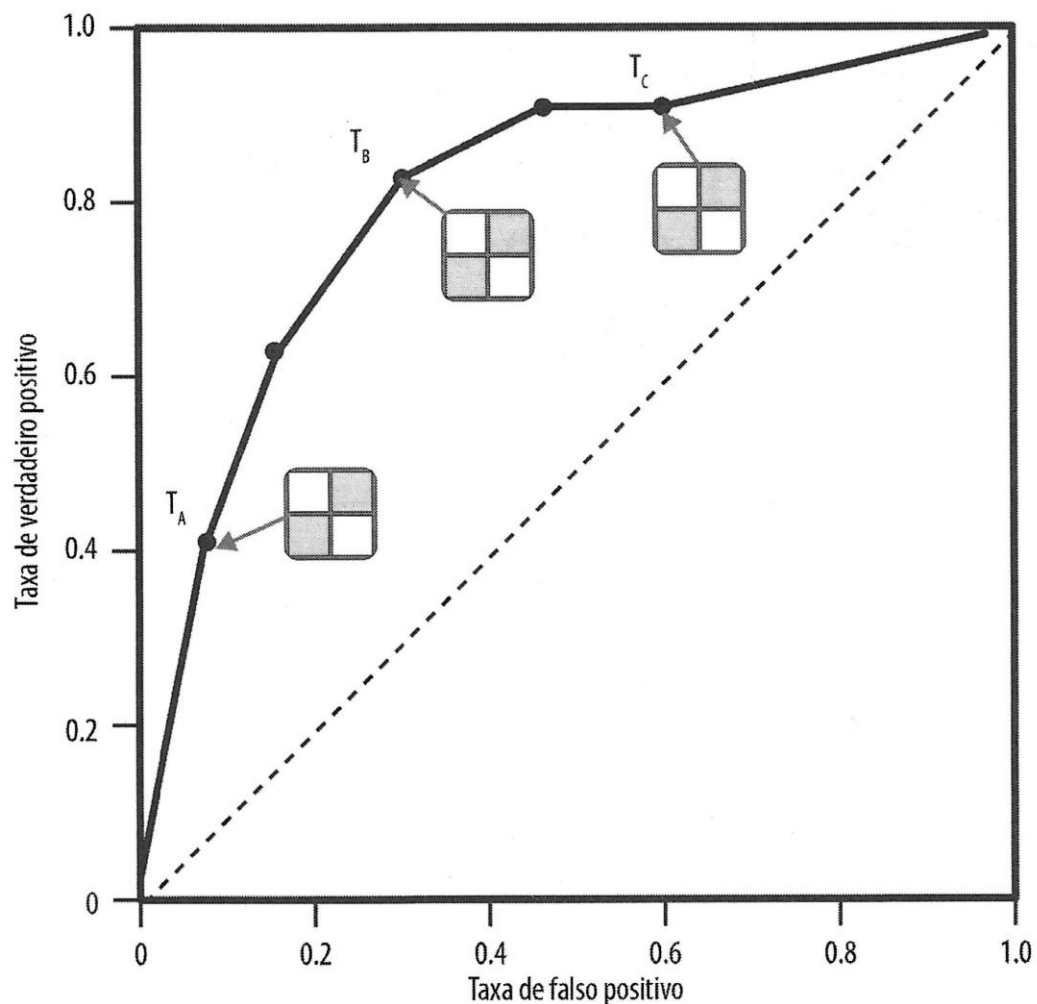
**Classificadores
"permissivos"**

No gráfico ROC, um modelo é superior a outro se estiver a noroeste do primeiro.

Classificadores "conservadores": classificam positivamente somente com fortes evidências. Geram então poucos VPs e poucos FPs.

Classificadores "permissivos": classificam positivamente com fracas evidências. Classificam corretamente então quase todos os VPs, mas com altas taxas de FPs.

Avaliação de modelos



Para cada limiar de um conjunto de dados de treinamento de um classificador, obtemos uma matriz de confusão específica (cada uma com seus VPs e FNs).

Cada resultado ocupa então um ponto diferente do espaço ROC.

A área sob a Curva ROC (denominada **AUC**) é um indicador geral de desempenho do modelo. A AUC varia de 0 a 1.

Avaliação de modelos

- Avaliação de modelos com dados de retenção
 - Situação ideal: treino e retenção com indicadores de desempenho bons e parecidos:

Métrica	Treino	Retenção
AUC	0,89	0,86
Recall	82%	79%

- Sintoma de sobreajuste:

Métrica	Treino	Retenção
AUC	0,95	0,68



Avaliação de modelos

- Avaliação de modelos com dados de retenção – *cont.*
 - Sintoma de modelagem que não capturou um padrão (indicadores ruins em ambos os conjuntos):

Métrica	Treino	Retenção
AUC	0,62	0,60



Avaliação de modelos

- **Avaliação da eficácia de um modelo de regressão logística**
 - Tal como em uma árvore de decisão ou qualquer método supervisionado, na regressão logística:
 - Fazemos uso de dados de retenção
 - Construimos matriz de confusão
 - Apuramos indicadores e curva ROC
 - Todavia, temos que a regressão logística já fornece a **probabilidade de classe** diretamente, e não a **predição de classe** antes, como no caso de uma árvore de decisão.
 - Então, devemos usar **limiares** para arbitrar a predição gerada na regressão logística.



Avaliação de modelos

- Uso dos limiares para determinar a classe predita na regressão logística
 - O mais comum é adotar um limiar de 0,5, ou seja:
 - Se $p \geq 0,5 \rightarrow$ prevê classe 1.
 - Se $p < 0,5 \rightarrow$ prevê classe 0.



Avaliação de modelos

- Por exemplo:

Cliente	Classe real	Probabilidade	Classe prevista
1	1	0,92	1
2	1	0,81	1
3	1	0,74	1
4	1	0,55	1
5	1	0,43	0
6	0	0,62	1
7	0	0,38	0
8	0	0,25	0
9	0	0,18	0
10	0	0,05	0



Avaliação de modelos

- Com isto, podemos criar a matriz de confusão e as métricas

Verdadeiros Positivos (VP): clientes 1, 2, 3 e 4 → 4

Falsos Negativos (FN): cliente 5 → 1

Falsos Positivos (FP): cliente 6 → 1

Verdadeiros Negativos (VN): clientes 7, 8, 9 e 10 → 4

	Classe Prevista = 0	Classe Prevista = 1
Classe Real = 0	4 (VN)	1 (FP)
Classe Real = 1	1 (FN)	4 (VP)



Avaliação de modelos

- Com isto, podemos criar a matriz de confusão e as métricas – *cont.*

Acurácia

$$\frac{VP + VN}{Total} = \frac{4 + 4}{10} = 0,80$$

Recall

$$\frac{VP}{VP + FN} = \frac{4}{4 + 1} = 0,80$$

Precisão

$$\frac{VP}{VP + FP} = \frac{4}{4 + 1} = 0,80$$

F1-score

Como precisão e recall são iguais:

$$F1 = 0,80$$



Avaliação de modelos

- Se utilizarmos limiar = 0,7:

Cliente	Probabilidade	Nova classe
1	0,92	1
2	0,81	1
3	0,74	1
4	0,55	0
5	0,43	0
6	0,62	0
7	0,38	0
8	0,25	0
9	0,18	0
10	0,05	0



Avaliação de modelos

- Neste caso, temos:

VP = 3

VN = 5

FP = 0

FN = 2

	Prevê 0	Prevê 1
Real = 0	5	0
Real = 1	2	3

- Escolhemos o limiar que produz o melhor equilíbrio entre falsos positivos (FP) e falsos negativos (FN).



Avaliação de modelos

- Testando vários limiares:

Limiar	Acurácia	Precisão	Recall	F1
0,30	82%	74%	97%	84%
0,40	87%	81%	93%	87%
0,50	91%	89%	90%	89%
0,60	92%	94%	83%	88%
0,70	90%	98%	70%	82%



Avaliação de modelos

- A escolha do limiar para predizer a classe na regressão logística também **depende do cenário de negócio**:
 - 0,5: ponto de partida e padrão quando os custos dos erros são semelhantes.
 - Limiar menor (0,3–0,4): aumenta o Recall (encontra mais casos positivos), mas também aumenta os falsos positivos. Melhor, por exemplo, no caso de detecção de doenças graves (classe positiva: com a doença).
 - Limiar maior (0,6–0,9): aumenta a Precisão (menos falsos positivos), mas deixa escapar mais casos positivos. Melhor, por exemplo, na concessão de empréstimos de valor elevado (classe positiva = bom pagador).



**Módulo 5 – Consolidação
de modelagens
(*Ensemble*)**

Consolidação de modelagens

- Depois de treinar um modelo normalmente perguntamos: "Existe alguma forma de deixá-lo ainda melhor?"
- Caminhos:
 - Melhorar os dados (preparar / ampliar)
 - Ajustar os parâmetros
 - **Combinar vários modelos**
- Essa terceira estratégia é chamada de **Ensemble Learning**.
- O termo “ensemble” significa “conjunto”, “grupo” ou “reunião de elementos”.



Ensemble Learning

- O **Ensemble Learning** é um conjunto de técnicas que **combinam múltiplos modelos** para produzir um modelo mais robusto.
 - Imaginemos três médicos avaliando um paciente. Um deles pode errar, mas a chance de os três errarem exatamente da mesma forma é muito menor. Assim, tomar uma decisão baseada na opinião dos três costuma produzir melhores resultados.
 - O mesmo ocorre em Machine Learning. Em vez de usar apenas um modelo, utilizamos vários. Cada modelo produz uma previsão. Depois essas previsões são combinadas. O resultado normalmente é melhor.



Ensemble Learning

- Vantagens do Ensemble Learning
 - Maior precisão
 - Menor variância
 - Menor risco de sobreajuste
 - Maior robustez
- Grandes famílias de Ensemble Learning
 - Bagging
 - Boosting



Bagging (*Bootstrap Aggregating*)

- Ideia principal do **Bagging** é:
 - Treinar vários modelos de forma independente.
 - Cada modelo recebe uma amostra diferente do conjunto de treinamento.
 - Depois todos “votam”, ou seja, é verificado qual o resultado que aparece na maioria dos modelos para um indivíduo novo submetido.
 - Algoritmo mais famoso: **Floresta Aleatória (*Random Forest*)**.



Modelagem de Floresta Aleatória

- A **Floresta Aleatória** (*Random Forest*) é uma técnica de aprendizado supervisionado utilizada tanto para classificação quanto para regressão.
- Ela é considerada uma evolução das árvores de decisão, pois reduz um dos principais problemas delas: o sobreajuste.
- A ideia básica é: em vez de confiar em uma única árvore de decisão, constrói-se uma "floresta" composta por dezenas ou centenas de árvores, e todas “votam” na resposta final.



Modelagem de Floresta Aleatória

- Etapas do algoritmo de Random Forest
 - 1 – Criar diversos grupos de dados (amostras) para treinamento
 - Por exemplo, em um conjunto com 10 clientes, criar:
 - 1,2,3,4,5,6,7,8,9,10
 - 1,2,2,4,5,6,7,8,9,10
 - 2,3,4,4,5,6,7,8,9,10
 - Observar que indivíduos podem aparecer repetidos em algumas amostras e em outras nem aparecer.
 - Este processo de sorteios com reposição chama-se *bootstrap sampling*.
 - Para cada conjunto, será construída sua árvore.



Modelagem de Floresta Aleatória

- Etapas do algoritmo de Random Forest – *cont.*
 - **2 – Escolher variáveis aleatórias**
 - Em cada nó de uma árvore, são sorteadas apenas algumas variáveis (atributos) para verificação de qual contribui mais para a próxima divisão.
 - Isto aumenta a diversidade das árvores.
 - Em uma divisão tradicional, todas as variáveis disponíveis no nível são testadas. Aqui isto não ocorre.



Modelagem de Floresta Aleatória

- Etapas do algoritmo de Random Forest – *cont.*
 - 3 – Construir as árvores
 - Cada árvore é desenvolvida conforme seu algoritmo (entropia, Gini).
 - Na Random Forest clássica, normalmente deixamos todas as árvores crescerem até quase o máximo possível, sem realizar poda (*pruning*), Definir uma profundidade máxima é aceitável.
 - O controle do sobreajuste vem principalmente da agregação de muitas árvores diferentes, e não da limitação da profundidade de cada uma.
 - As árvores finais da “floresta” poderão ter profundidades diferentes, pois cada uma teve um *bootstrap* diferente.



Modelagem de Floresta Aleatória

- Etapas do algoritmo de Random Forest – *cont.*
 - 4 – Submeter um indivíduo novo a todas as árvores e realizar “votação”
 - Cada árvore gerada pode predizer uma classe (por exemplo: “bom pagador” ou “mal pagador”).
 - Então por exemplo, se para 500 árvores tivermos:
 - 412 predições como “bom pagador”
 - 88 predições como “mal pagador”
- o resultado final é “bom pagador” (*votação majoritária*).



Modelagem de Floresta Aleatória

- Parâmetros da modelagem de Random Forest (biblioteca *scikit-learn*)

Parâmetro	Função
<code>n_estimators</code>	Número de árvores da floresta. Mais árvores costumam melhorar a estabilidade até certo ponto.
<code>max_depth</code>	Profundidade máxima de cada árvore. Limita a complexidade.
<code>max_features</code>	Número de variáveis candidatas avaliadas em cada divisão. É o principal diferencial da Random Forest.
<code>min_samples_split</code>	Número mínimo de amostras para dividir um nó. Antes da divisão, checa se pode fazê-la.
<code>min_samples_leaf</code>	Número mínimo de amostras em uma folha. Depois da divisão, checa se ela é aceitável (senão, o algoritmo busca outra).
<code>bootstrap</code>	Define se será utilizada amostragem Bootstrap (normalmente <code>True</code>).
<code>random_state</code>	Controla a aleatoriedade para tornar os resultados reproduzíveis.



Modelagem de Floresta Aleatória

- Vantagens da Random Forest
 - Alta precisão em muitos problemas.
 - Reduz bastante o sobreajuste em relação a uma árvore isolada.
 - Funciona bem com muitas variáveis.
 - Captura relações não lineares.
 - É robusta a ruídos e valores extremos.
 - Estima automaticamente a importância das variáveis (*feature importance*).
 - Exige pouca preparação dos dados (não requer normalização).



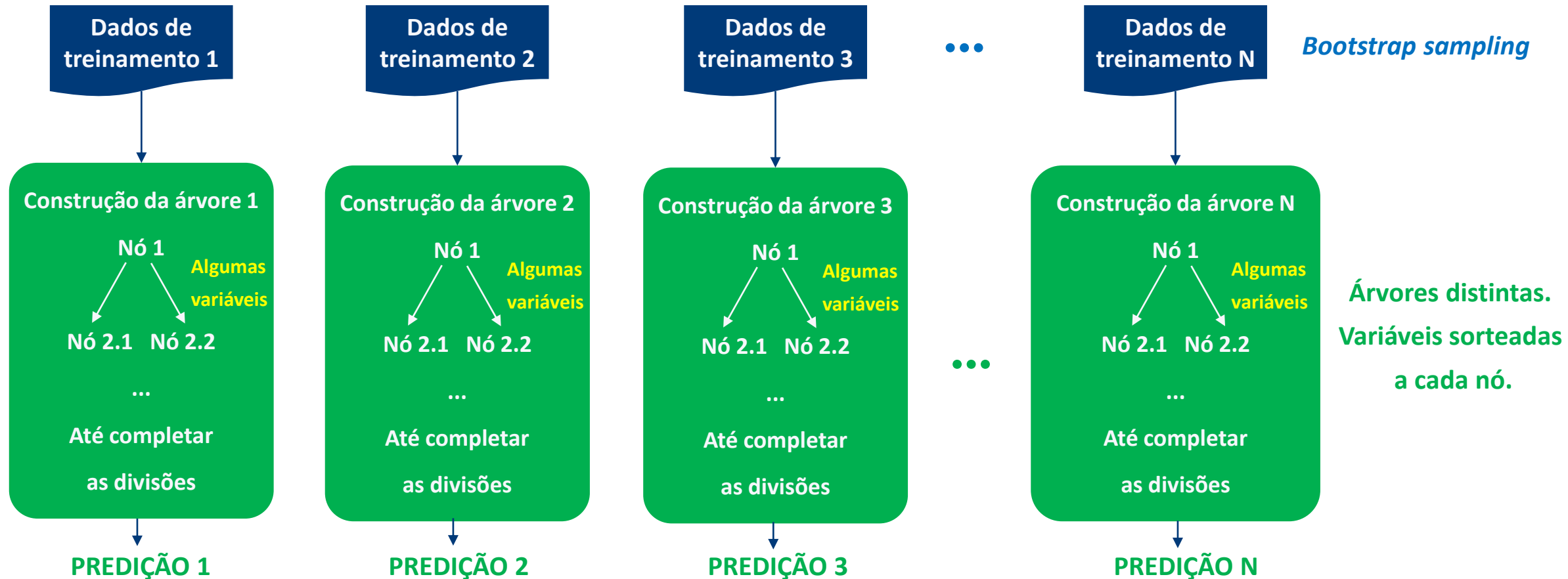
Modelagem de Floresta Aleatória

- **Desvantagens da Random Forest**
 - É menos interpretável do que uma árvore de decisão única.
 - Pode consumir bastante memória e processamento quando há muitas árvores.
 - O tempo de treinamento aumenta com o número de árvores.
 - O modelo final pode ocupar bastante espaço em disco.



Modelagem de Floresta Aleatória

▪ Random Forest – *resumo esquemático do algoritmo*



Indivíduo novo: submeter a todas as previsões e o resultado majoritário será a previsão final para ele

Boosting

- A ideia principal do **Boosting** é:
 - Enquanto Bagging trabalha em paralelo, o **Boosting** trabalha em sequência.
 - No Boosting, cada modelo “aprende” com os erros do anterior e tenta melhorar.
 - “Modelo anterior” aqui não se trata, por exemplo, de uma mesma árvore com profundidade diferente – cada uma poderá ter uma estrutura completamente distinta.
 - Então, a poda da árvore não é um mecanismo de Boosting.



Boosting

- Na enorme maioria dos algoritmos de Boosting, cada modelo é uma árvore de decisão pequena, chamada de árvore fraca (*weak learner*).
- As árvores normalmente possuem profundidade 1 (*decision stump*) ou profundidades 2 e 3. Raramente são árvores grandes.
- A filosofia do Boosting é: "Em vez de uma árvore enorme tentando resolver tudo, vamos usar centenas de árvores pequenas, cada uma corrigindo um pequeno pedaço do problema."



Boosting

- No Boosting, as árvores participantes **NÃO** são versões de uma mesma árvore com profundidades distintas.
- Cada árvore deve se construída do zero e de forma que sejam diferentes.
- Para o “aprendizado com os erros”, exemplos de algoritmos são:
 - **AdaBoost**
 - **Gradient Boosting**



Boosting

- **AdaBoost**

- Aqui, construímos a primeira árvore e identificamos para quais elementos dos dados de treinamento a predição falha. Por exemplo, falhou nos elementos A, C e D.
- Estes elementos A, C e D então são marcados como “de maior peso”.
- Na construção da próxima árvore (distinta da anterior), usam-se os mesmos dados e atributos. A cada nível, são checadas quais das possíveis divisões geram mais acertos para os elementos que receberam “maior peso” na árvore anterior.



Boosting

- AdaBoost – *cont.*
 - Então, na divisão do AdaBoost, não se considera o atributo de melhor ganho de informação ou Gini, mas, em vez disto, o que maximiza o acerto dos elementos de maior peso da árvore anterior?
 - Não. No AdaBoost, não se abandona o critério de qualidade tradicional da divisão (como ganho de informação ou Gini).
 - O que muda é que essa qualidade passa a ser avaliada levando em conta **também** os pesos das observações (ponderação). Veremos como isto ocorre.
 - Portanto, uma divisão com menor ganho “tradicional” pode ser escolhida se, quando os pesos são considerados, ela for melhor para os exemplos de maior peso.



Boosting

- AdaBoost – *cont.*
 - Exemplo:
 - **Primeira árvore**: para o conjunto de clientes A, B, C, D, E, F, G, H, I, e J, os clientes H, I e J falham na predição da árvore.
 - H, I e J são então considerados os “casos difíceis”, recebendo peso 4, por exemplo (os demais seguem com peso 1).
 - Vamos agora para a construção da **segunda árvore**:
 - Ela usa os mesmos clientes, de A a J.
 - Ela usa o mesmo conjunto de atributos.
 - Na sua **primeira divisão**, são apurados os ganhos de informação tradicionais (GI da entropia, por exemplo) de todos os atributos, com ponderação.



Boosting

- AdaBoost – *cont.*
 - Exemplo – *cont.*:
 - Suponhamos que ocorra o seguinte:
 - Atributo **Renda** produziria GI “tradicional” = 0,40 mas acerta apenas 1 dos 3 “casos difíceis” da árvore 1.
 - Atributo **Histórico de crédito** produziria GI “tradicional” = 0,25 mas acerta todos os 3 “casos difíceis” da árvore 1.
 - Se olharmos apenas para o GI “tradicional”, **Renda** venceria para a divisão, pois $0,40 > 0,25$.
 - Contudo, é preciso considerar a questão dos “casos difíceis” (de maior peso) na decisão da divisão também, e neste caso poderá ocorrer que o algoritmo escolha dividir por **Histórico de crédito**.



Boosting

- AdaBoost – *cont.*
 - Exemplo – *cont.*:
 - Na decisão de divisão, utiliza-se então uma ponderação entre GI (ou Gini) “tradicional” e pesos dos elementos “difíceis”.
 - O AdaBoost não faz uma conta do tipo: "50% GI (ou Gini) + 50% acerto dos casos de maior peso". O mecanismo é mais rigoroso.

Vejamos um exemplo numérico de como funciona a ponderação entre um critério tradicional de divisão como GI e os pesos dos “casos difíceis”



Boosting

- AdaBoost – *cont.*
 - Consideremos a seguinte situação de “casos difíceis” vinda da primeira árvore:

Caso	Classe	Peso
A	0	1
B	0	1
C	1	4
D	1	1
E	0	1
F	1	1

Boosting

- AdaBoost – *cont.*
 - O peso total é $1+1+4+1+1+1=9$.
 - Proporções ponderadas no nó:

Classe 0

A + B + E:

$$1 + 1 + 1 = 3$$

Classe 1

C + D + F:

$$4 + 1 + 1 = 6$$

Logo, as proporções ponderadas são:

$$P(0) = \frac{3}{9} = 0,333$$

$$P(1) = \frac{6}{9} = 0,667$$



Boosting

- AdaBoost – *cont.*
 - Entropia ponderada do nó:

$$H = -P(0) \log_2 P(0) - P(1) \log_2 P(1)$$

Portanto:

$$H = -0,333 \log_2(0,333) - 0,667 \log_2(0,667)$$

$\approx 0,918$

- Os pesos modificaram os valores de proporções de classes utilizadas na fórmula – desta maneira ocorre a ponderação.
- Calculamos as entropias dos filhos de cada divisão da mesma forma e apuramos os GIs, que serão então ponderados pelos pesos.



Boosting

- AdaBoost – *cont.*
 - A sequência então é:
 - Pesos ->
 - Distribuição ponderada ->
 - Entropia / Gini ponderada ->
 - Qualidade da divisão a ser adotada para a decisão de divisão

Vamos retomar agora a segunda árvore



Boosting

- AdaBoost – *cont.*
 - Exemplo – *cont.*:
 - Na **segunda divisão** da nossa **segunda árvore**, o processo continua usando os mesmos “casos difíceis” e seus pesos (definidos na primeira árvore) e a escolha da divisão é feita dentro de cada um dos subgrupos criados pela primeira divisão.
 - Para todas as demais divisões da segunda árvore, sempre consideramos como os “casos difíceis” na ponderação os definidos na primeira árvore, com seus pesos oriundos de lá.
 - Ou seja, os pesos não são atualizados entre os níveis de uma mesma árvore.



Boosting

- AdaBoost – *cont.*

- Exemplo – *cont.*:

- Concluída a segunda árvore, o conjunto de “casos difíceis” sofre uma atualização, ou seja, seus pesos numéricos são revistos.

- Exemplo simplificado de atualização de pesos:

Caso	Árvore 1	Peso
A	acertou	1
B	acertou	1
C	errou	4
D	acertou	1
E	errou	4

“Casos difíceis” iniciais



Boosting

- AdaBoost – *cont.*

- Exemplo – *cont.*:

- Exemplo simplificado de atualização de pesos – *cont.*:

- A árvore 2 foi construída considerando-se estes elementos “difíceis” C e E, da primeira árvore.
 - Suponhamos que o resultado da árvore 2, depois pronta, seja:

Caso		Árvore 2
A		acertou
B	Novo “caso difícil”	errou
C	Era um “caso difícil” e agora houve acerto	acertou
D		acertou
E	Erro de predição persistiu	errou

Boosting

- AdaBoost – *cont.*

- Exemplo – *cont.*:

- Exemplo simplificado de atualização de pesos – *cont.*:

- Consequências (atualização):

Caso	Árvore 2
A	acertou
B	errou
C	acertou
D	acertou
E	errou

Novo “caso difícil”

=> “B” passa de peso 1 para um peso mais alto

Era um “caso difícil” e agora houve acerto

=> “C” tem seu peso diminuído de 4 para um valor menor

Erro de predição persistiu

=> “E” tem seu peso aumentado de 4 para um valor maior

Boosting

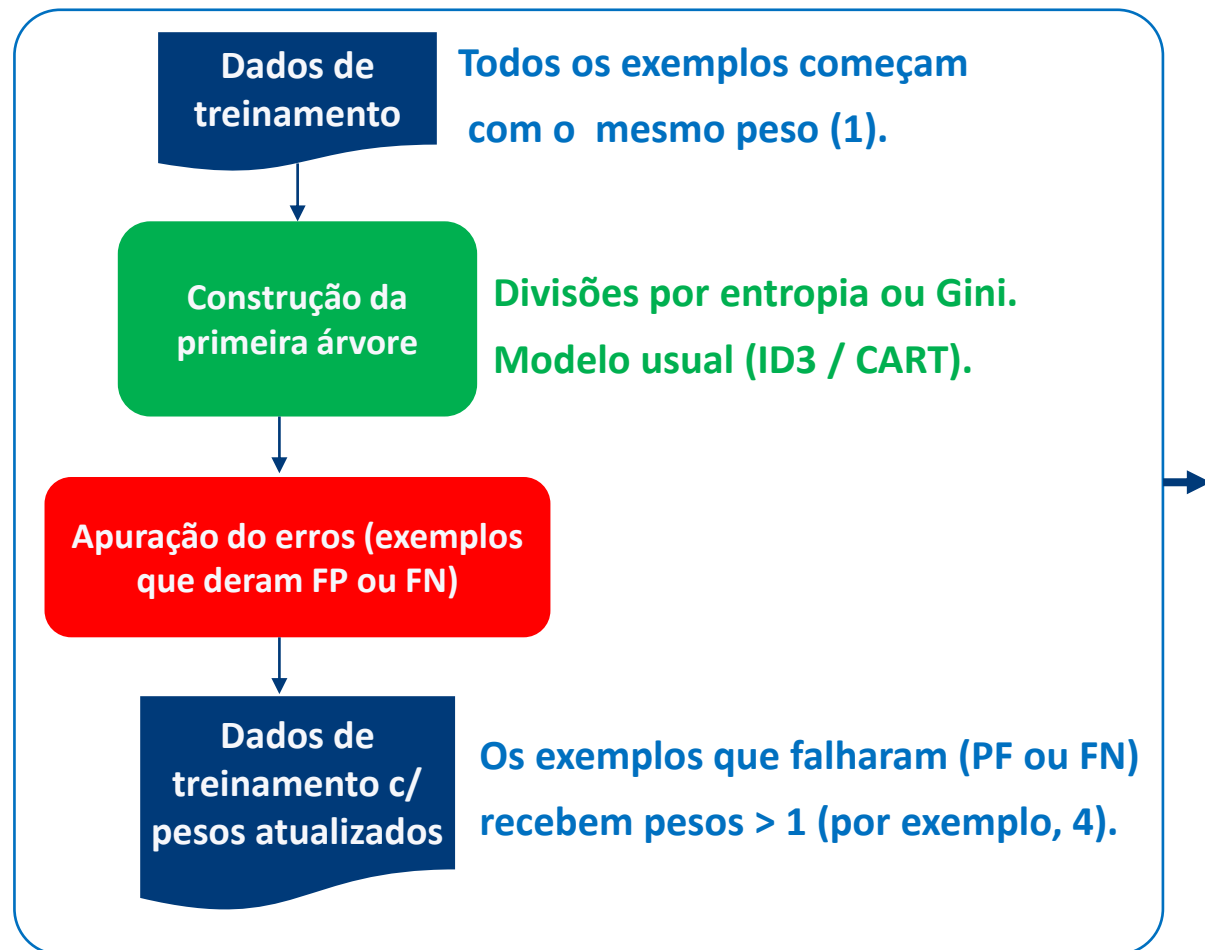
- AdaBoost – *cont.*
 - Exemplo – *cont.*:
 - Vamos agora para a construção da **terceira árvore**:
 - Ela continua usando os mesmos clientes, de A a J.
 - Ela continua usando o mesmo conjunto de atributos.
 - As divisões seguem o mecanismo adotado para a árvore anterior, mas considerando-se agora nas ponderações a configuração de pesos atualizada após a construção **da árvore 2**.
 - E assim por diante...
 - O AdaBoost constrói então árvores sequencialmente até atingir o número máximo definido ou até que não haja mais ganho útil (*early stopping*).



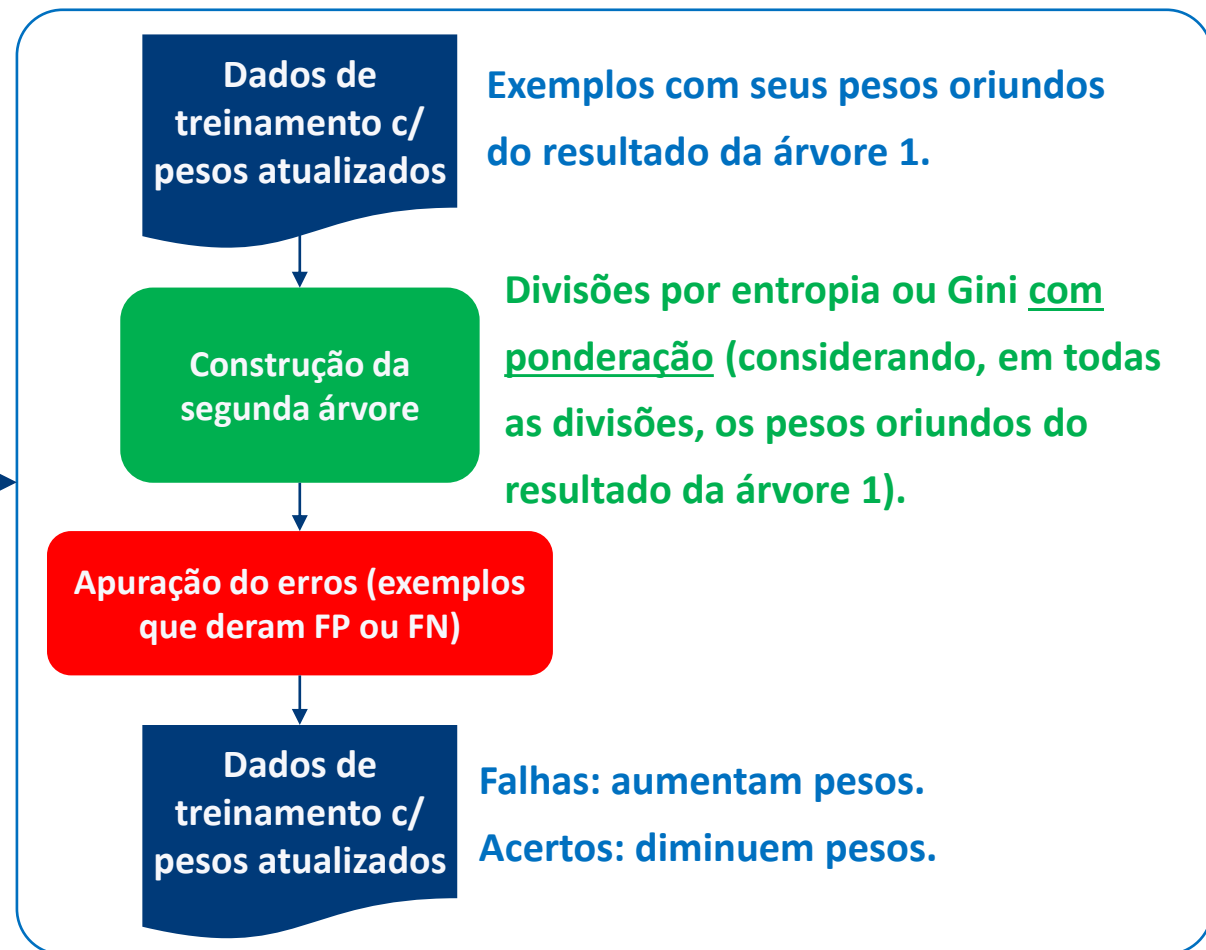
Boosting

▪ AdaBoost – *resumo esquemático do algoritmo:*

ÁRVORE 1



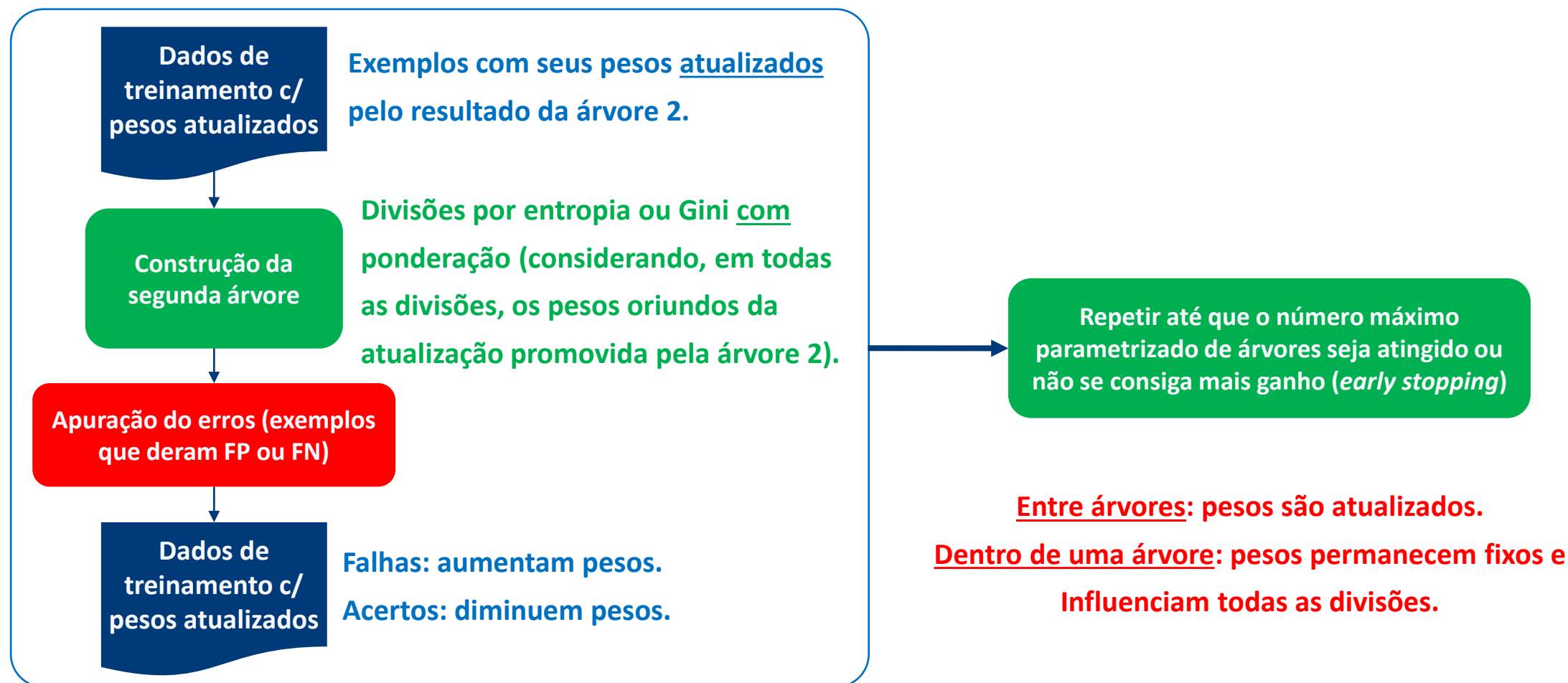
ÁRVORE 2



Boosting

▪ AdaBoost – *resumo esquemático do algoritmo – cont.:*

ÁRVORE 3



Boosting

- **Gradient Boosting**

- Trata-se de um método de ensemble que combina várias árvores de decisão pequenas para produzir um modelo muito mais poderoso.
- Cada nova árvore tenta corrigir aquilo que o conjunto das árvores anteriores ainda está fazendo errado.
- No **AdaBoost**, trabalhamos aumentando o **peso** das observações que foram classificadas incorretamente.
- No **Gradient Boosting**, a nova árvore é treinada para prever o **erro/resíduo** que o modelo atual ainda apresenta.
- O Gradient Boosting pode ser usado para regressão e classificação.



Boosting

- Gradient Boosting – *cont.*
 - Vejamos um exemplo de **regressão** (previsão de valor de uma casa)
 - Dados de treinamento:

Casa	Área	Preço real
A	50 m ²	R\$ 200 mil
B	60 m ²	R\$ 230 mil
C	80 m ²	R\$ 300 mil
D	100 m ²	R\$ 380 mil



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - O Gradient Boosting começa com um modelo inicial bastante simples.
 - No caso de regressão, podemos começar simplesmente apurando a média dos preços.
 - Suponhamos média = 277,5 mil (vamos chamar este valor de F_1).
 - O ponto de partida (Modelo 1) é então:

Casa A → 277,5

Casa B → 277,5

Casa C → 277,5

Casa D → 277,5



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Agora, apuramos os erros:

Casa	Real	Modelo 1	Erro
A	200	277,5	-77,5
B	230	277,5	-47,5
C	300	277,5	+22,5
D	380	277,5	+102,5



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Temos que:
 - Casa A → modelo está superestimando
 - Casa B → modelo está superestimando
 - Casa C → modelo está subestimando
 - Casa D → modelo está subestimando
 - A aprendizado aqui seria algo como:
 - Área < 70 m²: erro de valor negativo
 - Área ≥ 70 m²: erro de valor positivo



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - O Modelo 2 subsequente buscará então uma correção para o Modelo 1 da seguinte forma:

$$F_2(x) = F_1(x) + \eta h_1(x)$$

- Onde:
 - $F_2(x)$ = modelo novo
 - $F_1(x)$ = modelo anterior
 - η = **learning rate** (taxa de aprendizado)
 - $h_1(x)$ = nova árvore



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - O *learning rate* serve para atenuar a correção trazida pela nova árvore, evitando que ela fique “rápida demais”.
 - Adotamos, por exemplo, $\eta = 0,1$
 - Então: Modelo antigo + 10% de correção = Modelo novo
 - Isto torna o aprendizado mais gradual.
 - Depois de muitas árvores h_N , temos o seguinte ensemble:
 - $F(x) = F_1(x) + \eta h_1(x) + \eta h_2(x) + \eta h_3(x) + \dots$



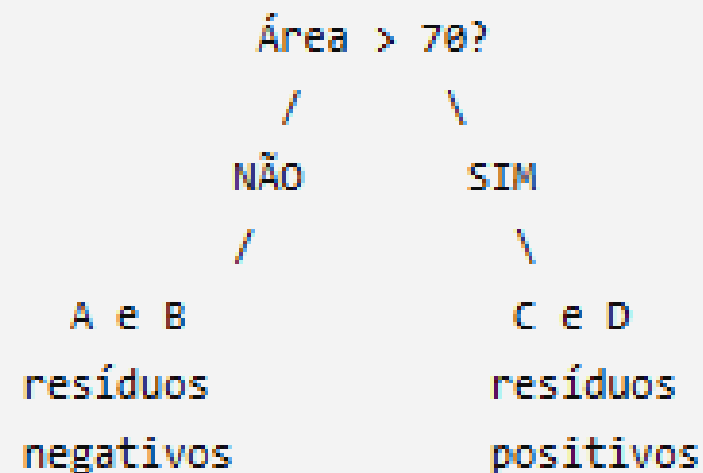
Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Voltando ao nosso caso, vamos avançar para os Modelos 1 e 2:
 - No Modelo 1 (chute inicial com base na média), temos:

Erros apurados (resíduo):

Área	Resíduo
50 m ²	-77,5
60 m ²	-47,5
80 m ²	+22,5
100 m ²	+102,5

Árvore simples decorrente:



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Vamos prosseguir na ideia de trabalhar com médias.
 - Então:

Grupo A + B

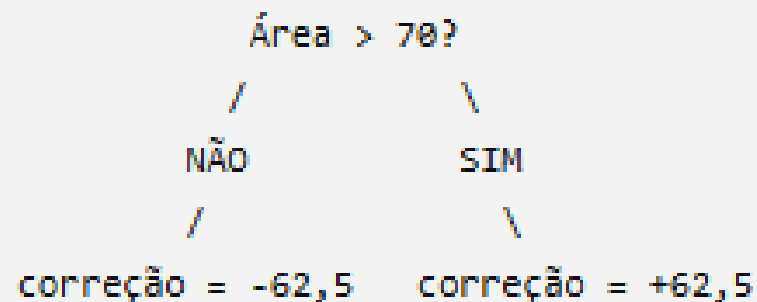
Média dos resíduos:

$$\frac{-77,5 - 47,5}{2} = -62,5$$

Grupo C + D

$$\frac{22,5 + 102,5}{2} = 62,5$$

Árvore h_1 decorrente:



- Para o gradiente, vamos adotar $\eta = 0,1$.

Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - A árvore h_1 encontrou:
 - $A/B \rightarrow -62,5$
 - $C/D \rightarrow +62,5$
 - Mas não vamos aplicar toda esta correção de uma vez, mas sim atenuar com $\eta = 0,1$, de forma que:
 - $A/B \rightarrow -6,25$
 - $C/D \rightarrow +6,25$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - O Modelo 2 é então:
 - $M_2 = M_1 + 0,1 * h_1$

Casa	M1	Correção Árvore	M2
A	277,5	-6,25	271,25
B	277,5	-6,25	271,25
C	277,5	+6,25	283,75
D	277,5	+6,25	283,75



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Ocorreu então o seguinte:

ANTES			DEPOIS	
A	277,5	Árvore h_1 atenuada por $\eta = 0,1$	A	271,25
B	277,5		B	271,25
C	277,5		C	283,75
D	277,5		D	283,75

- Vamos prosseguir.

Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Após M2, os novos erros (resíduos) são os seguintes:

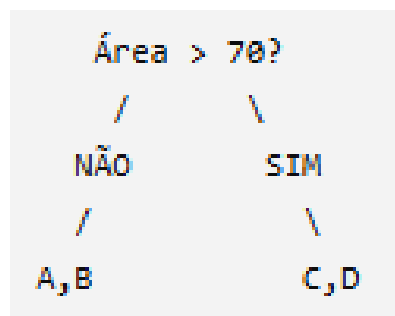
Casa	Real	M2	Novo resíduo
A	200	271,25	-71,25
B	230	271,25	-41,25
C	300	283,75	+16,25
D	380	283,75	+96,25

- Vamos então seguir para o Modelo 3.



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Novamente vamos separar por área e apurar as médias dos resíduos:



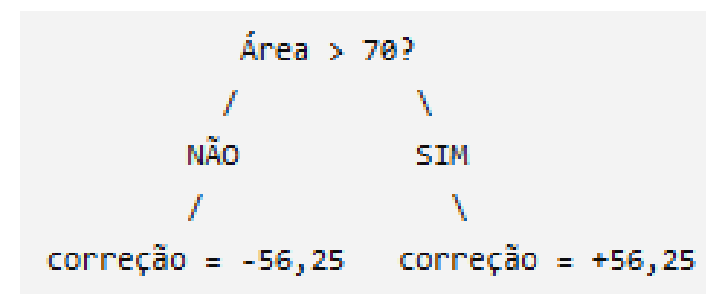
A + B

C + D

$$\frac{-71,25 - 41,25}{2} = -56,25$$

$$\frac{16,25 + 96,25}{2} = 56,25$$

Árvore h_2 :



- Aplicando novamente o *learning rate* $\eta = 0,1$:
 - A/B $\rightarrow -5,625$
 - C/D $\rightarrow +5,625$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Portanto:
 - $M_3 = M_2 + 0,1 * h_2$

Casa	M2	Correção Árvore	M3
A	271,25	-5,625	265,625
B	271,25	-5,625	265,625
C	283,75	+5,625	289,375
D	283,75	+5,625	289,375



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Percebemos que os erros estão diminuindo, o que mostra as melhorias graduais.
 - Nosso exemplo simples foi só com a variável “área” e valor de divisão 70m². O algoritmo de Gradient Boosting na prática vai testando **diferentes variáveis** e **diferentes valores (limiares) para divisão**.
 - Ou seja, a cada nova etapa do Gradient Boosting, o algoritmo procura a árvore que melhor consegue reduzir a função de perda naquele momento.



Boosting

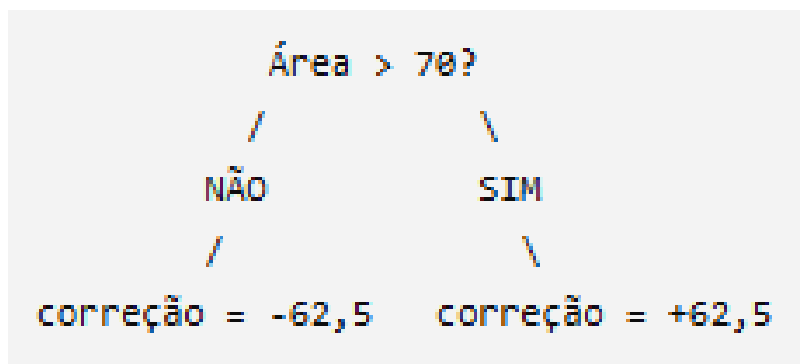
- Gradient Boosting – *cont.*
 - Frequentemente é preferível *learning rate* menor + mais árvores do que *learning rate* muito alto + poucas árvores. O caminho mais suave é mais adequado.
 - O algoritmo para quando o número parametrizado de árvores for atingido ou quando o erro começar a não melhorar, indicando sobreajuste.



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.*
 - Para uma casa de **90m²** (que não está nos dados de treinamento), qual então a predição do modelo treinado?

Árvore h_1 do modelo:



Como:

$$90 > 70$$

ela vai para o lado SIM.

A Árvore 2 produz:

$$+62,5$$

Com learning rate:

$$0,1 \times 62,5 = +6,25$$

Então, até aqui:

$$F_2(90) = 277,5 + 6,25$$

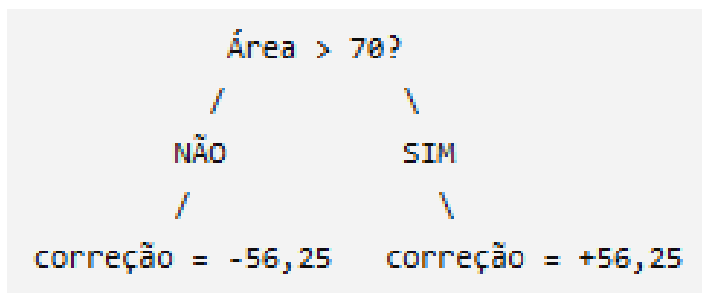
$$F_2(90) = 283,75$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.* – casa de **90m²**

Árvore h_2 do modelo:



Nossa casa de 90 m² novamente satisfaz:

$$90 > 70$$

Portanto:

$$h_3(90) = +56,25$$

Aplicando o learning rate:

$$0,1 \times 56,25 = +5,625$$

Agora:

$$F_3(90) = 283,75 + 5,625$$

$$\boxed{F_3(90) = 289,375}$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.* – casa de 90m²
 - Suponha que o treinamento tenha produzido as seguintes árvores:

Árvore	Correção para uma casa de 90 m ²	× learning rate	Correção efetiva
Árvore 1	+62,50	0,1	+6,25
Árvore 2	+56,25	0,1	+5,625
Árvore 3	+40,00	0,1	+4,00
Árvore 4	+25,00	0,1	+2,50
Árvore 5	-10,00	0,1	-1,00
...	...	...	...



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para regressão – *cont.* – casa de **90m²**
 - Imagine que depois de todas as árvores a soma das correções seja +30.
 - Então: $F(90) = 277,5 + 30 = 307,5$.
 - Assim, preço predito para a casa de 90m² = 307,5 mil.
 - Portanto, o indivíduo novo é colocado no “caminho” gerado pelo treinamento do Gradient Boosting, para a coleta das devidas correções árvore a árvore, que somadas ao chute inicial do modelo conduzem à predição desejada.
 - Fato interessante: o “caminho” construído pelo algoritmo faz as previsões para os dados de treinamento convergirem progressivamente para seus valores-alvo originais.



Boosting

- Gradient Boosting – *cont.*
 - E para classificação, como fica o uso do algoritmo? Vejamos um exemplo para os seguintes dados de treinamento (classe positiva = 1):

Cliente	Renda (R\$ mil)	Dívidas	Inadimplente
A	3	5	1
B	4	4	1
C	5	3	0
D	6	3	0
E	7	2	0
F	8	2	0
G	9	1	0
H	10	1	1



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Aqui, partimos de $p(\text{inadimplentes}) = 3/8 = 0,375$

Cliente	Real	Probabilidade inicial
A	1	37,5%
B	1	37,5%
C	0	37,5%
D	0	37,5%
E	0	37,5%
F	0	37,5%
G	0	37,5%
H	1	37,5%



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Na classificação, começamos com um score inicial para todos os clientes, usando normalmente log loss. Uma forma de visualizar essa pontuação é através do logit:

$$F_0 = \ln \left(\frac{p}{1 - p} \right)$$

- Para $p = 0,375$:

$$F_0 = \ln \left(\frac{0,375}{0,625} \right)$$

$$F_0 = \ln(0,6)$$

$$F_0 \approx -0,511$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*

Para classificação binária com log loss, podemos usar:

$$r_i = y_i - p_i$$

Como todos começam com:

$$p_i = 0,375$$

temos:

Para inadimplentes

$$1 - 0,375 = +0,625$$

Para adimplentes

$$0 - 0,375 = -0,375$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*

Cliente	Real	P inicial	Gradiente $y - p$
A	1	0,375	+0,625
B	1	0,375	+0,625
C	0	0,375	-0,375
D	0	0,375	-0,375
E	0	0,375	-0,375
F	0	0,375	-0,375
G	0	0,375	-0,375
H	1	0,375	+0,625



Boosting

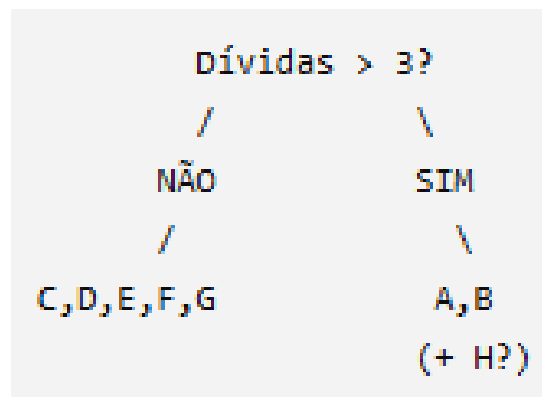
- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Agora o algoritmo testa divisões para montar a árvore 1:

Ele checa:

```
Renda > 5
Renda > 6
Renda > 7
Renda > 8
Dívidas > 1
Dívidas > 2
Dívidas > 3
Dívidas > 4
...
```

Suponhamos que ele descubra que Dívidas > 3 seja uma boa divisão.

Então:



Nem sempre a árvore é perfeita. Note que H tem apenas 1 dívida e é inadimplente.



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Dívidas > 3
 - Ramo “Sim”
 - A e B:
 - A → +0,625
 - B → +0,625
 - Ramo “Não”
 - C, D, E, F, G e H:
 - H com +0,625, demais com -0,375

Médias dos gradientes de A e B:

$$\frac{0,625 + 0,625}{2} = \boxed{+0,625}$$

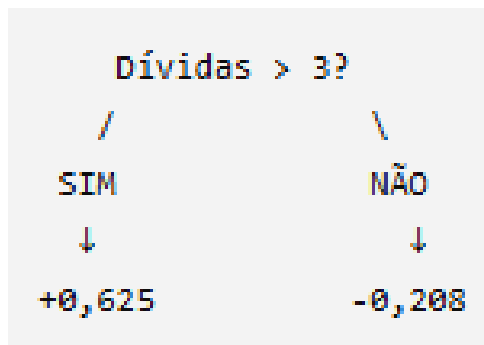
Médias dos gradientes de C, D, E, F, G e H:

$$\begin{aligned} & -0,375 \times 5 + 0,625 \\ &= -1,875 + 0,625 \\ &= -1,25 \\ & \frac{-1,25}{6} = \boxed{-0,2083} \end{aligned}$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Chegamos então ao seguinte:



- Aplicando-se *learning rate* $\eta = 0,1$, chegamos aos **ajustes**:

Dívidas > 3

$$0,1 \times 0,625 = +0,0625$$

Dívidas $\leq$ 3

$$0,1 \times (-0,2083) = -0,02083$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Temos então:

Cliente	Dívidas	Caminho na Árvore 1	Ajuste	Score anterior	Novo score F_1
A	5	SIM	+0,0625	-0,5108	-0,4483
B	4	SIM	+0,0625	-0,5108	-0,4483
C	3	NÃO	-0,02083	-0,5108	-0,5317
D	3	NÃO	-0,02083	-0,5108	-0,5317
E	2	NÃO	-0,02083	-0,5108	-0,5317
F	2	NÃO	-0,02083	-0,5108	-0,5317
G	1	NÃO	-0,02083	-0,5108	-0,5317
H	1	NÃO	-0,02083	-0,5108	-0,5317



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Transformando os Fs em probabilidades, com uso da sigmoide e apurando os **gradientes**, temos:

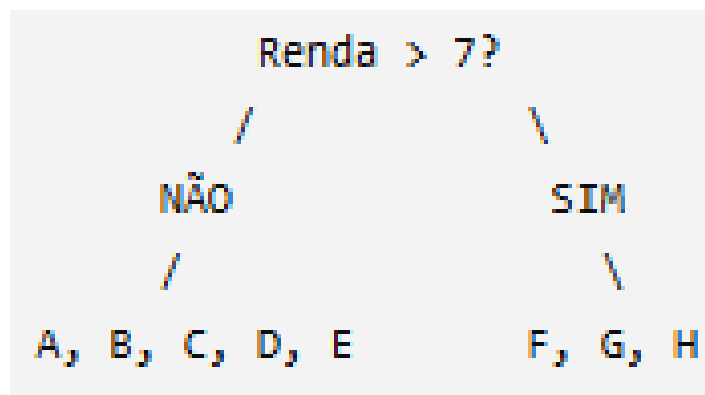
$$p = \frac{1}{1 + e^{-F}}$$

Cliente	Real	Probabilidade	Gradiente
A	1	38,99%	+0,6101
B	1	38,99%	+0,6101
C	0	37,01%	-0,3701
D	0	37,01%	-0,3701
E	0	37,01%	-0,3701
F	0	37,01%	-0,3701
G	0	37,01%	-0,3701
H	1	37,01%	+0,6299



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Vamos para a árvore 2.
 - Suponhamos que agora o algoritmo descubra que uma divisão boa seja Renda > 7.



Médias dos gradientes – ramo “NÃO”

$$\frac{0,610241 + 0,610241 - 0,370130 - 0,370130 - 0,370130}{5}$$

0,022018

Médias dos gradientes – ramo “SIM”

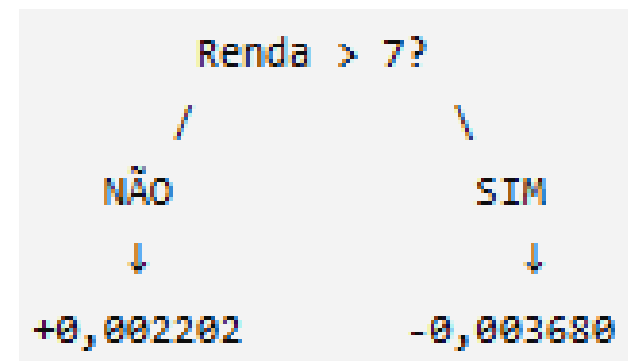
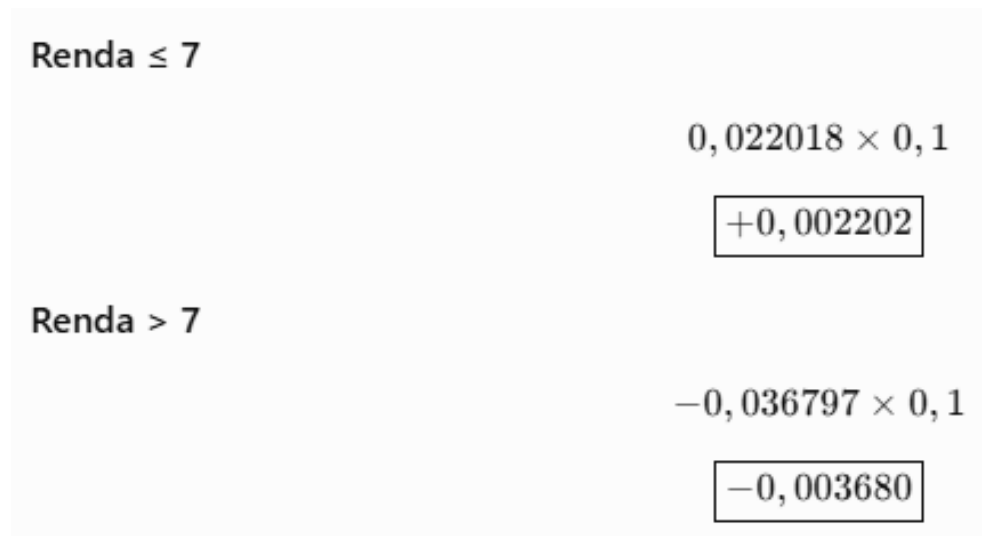
$$\frac{-0,370130 - 0,370130 + 0,629870}{3}$$

-0,036797



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Aplicando-se *learning rate* $\eta = 0,1$, chegamos aos **ajustes**:



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*

$$F_2 = F_1 + \text{ajuste da Árvore 2}$$

- Por exemplo, para A e F:

A

A tem renda 3:

$$3 > 7 \quad \text{NÃO}$$

Portanto:

$$F_2(A) = -0,448326 + 0,002202$$

$$F_2(A) = -0,446124$$

F

Renda = 8 → SIM:

$$F_2(F) = -0,531659 - 0,003680$$

$$F_2(F) = -0,535339$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Resultado após a árvore 2:

Cliente	Renda	Dívida	Real	F_0	Ajuste Árvore 1	F_1	p_1	Novo gradiente	Ajuste Árvore 2	F_2
A	3	5	1	-0,510826	+0,062500	-0,448326	38,976%	+0,610241	+0,002202	-0,446124
B	4	4	1	-0,510826	+0,062500	-0,448326	38,976%	+0,610241	+0,002202	-0,446124
C	5	3	0	-0,510826	-0,020833	-0,531659	37,013%	-0,370130	+0,002202	-0,529457
D	6	3	0	-0,510826	-0,020833	-0,531659	37,013%	-0,370130	+0,002202	-0,529457
E	7	2	0	-0,510826	-0,020833	-0,531659	37,013%	-0,370130	+0,002202	-0,529457
F	8	2	0	-0,510826	-0,020833	-0,531659	37,013%	-0,370130	-0,003680	-0,535339
G	9	1	0	-0,510826	-0,020833	-0,531659	37,013%	-0,370130	-0,003680	-0,535339
H	10	1	1	-0,510826	-0,020833	-0,531659	37,013%	+0,629870	-0,003680	-0,535339

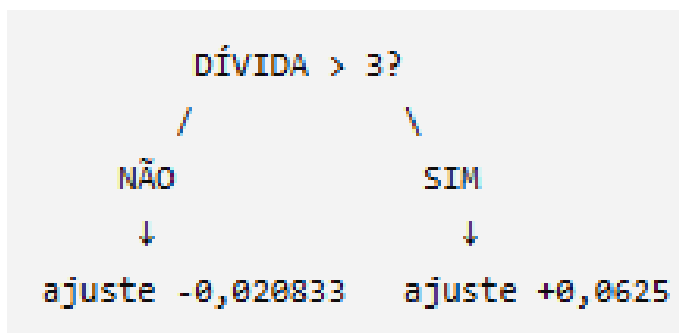
Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Cliente novo:

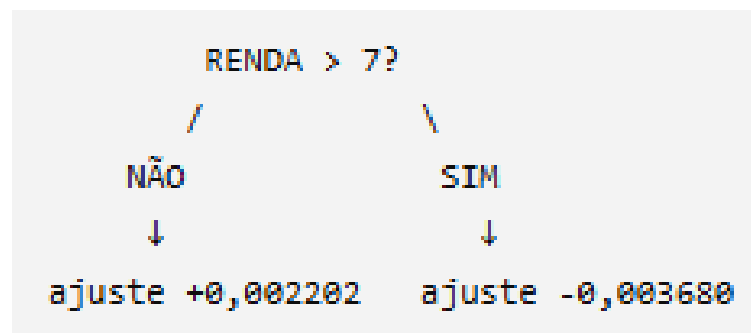
Cliente	Renda	Dívidas
X	R\$ 6 mil	4

- Para a predição do cliente novo no modelo, partimos do seguinte:

Árvore 1:



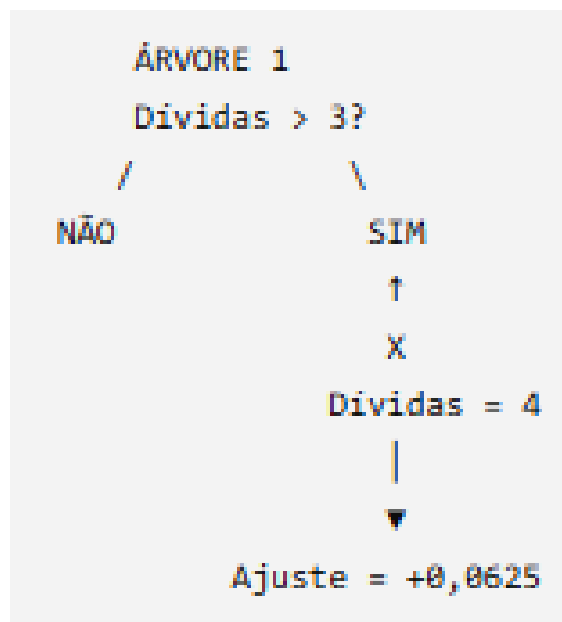
Árvore 2:



$$F_0 = -0,510826$$

Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Cliente novo X passa na **árvore 1** do modelo:
 - Dívidas X = 4 => entra no ramo “SIM”:



A primeira árvore vai acrescentar ao score +0,0625

O score era:

$$F_0 = -0,510826$$

A Árvore 1 acrescenta:

$$+0,0625$$

Logo:

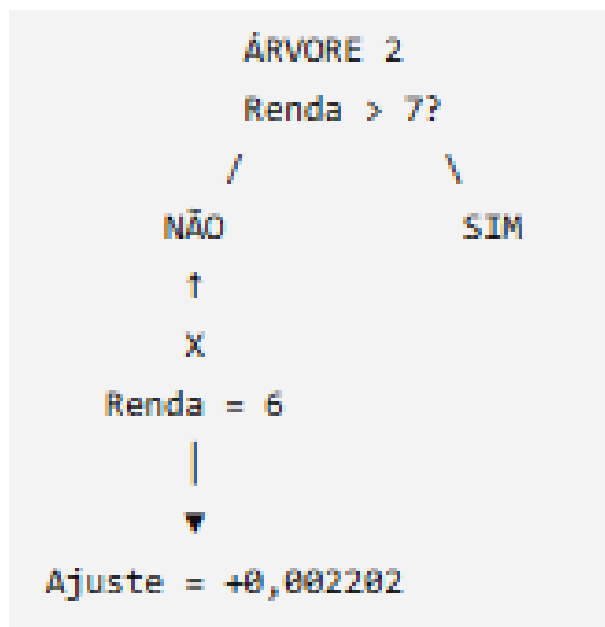
$$F_1(X) = -0,510826 + 0,0625$$

$$F_1(X) = -0,448326$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Cliente novo X passa na **árvore 2** do modelo:
 - Renda X = 6 => entra no ramo “NÃO”:



A segunda árvore vai acrescentar ao score +0,002202

Tínhamos:

$$F_1(X) = -0,448326$$

Agora:

$$F_2(X) = -0,448326 + 0,002202$$

$$F_2(X) = -0,446124$$

Portanto, depois das duas árvores:

$$F_2(X) = -0,446124$$

Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - O cliente passaria por todas as árvores do modelo, atualizando seu F, e ao final transformamos este F em probabilidade de classe positiva via sigmoide.
 - No nosso exemplo de 2 árvores, temos:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-(-0,446124)}}$$

$$P(Y = 1 | X) \approx 39,0\%$$



Boosting

- Gradient Boosting – *cont.*
 - Exemplo para classificação – *cont.*
 - Adotando um *threshold* padrão de 50%:

$$P(Y = 1) \geq 50\% \Rightarrow \text{classe 1}$$

$$P(Y = 1) < 50\% \Rightarrow \text{classe 0}$$

- Como $P = 39\%$ -> classificamos como 0 (não inadimplente).
- Seguindo com o algoritmo para mais árvores, há mais adição de correções e chegaríamos a uma probabilidade mais preciosa, com consecutiva classificação mais firme.



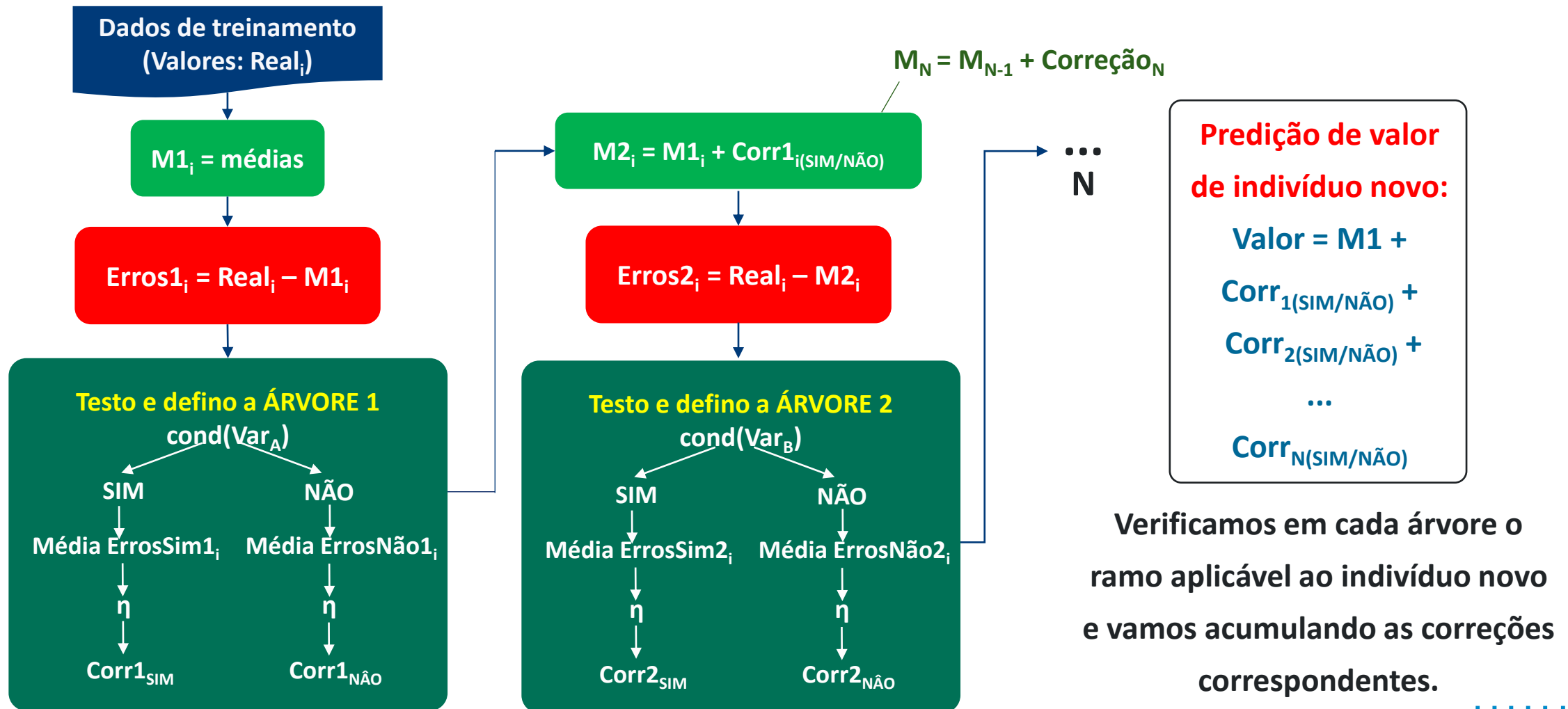
Boosting

- Gradient Boosting – *cont.*
 - No Gradient Boosting para classificação, cada árvore não tenta descobrir sozinha se um indivíduo é da classe-alvo.
 - Ela tenta fazer uma pequena correção no score que o conjunto das árvores anteriores já produziu.
 - É justamente o acúmulo dessas pequenas correções que constrói a previsão final.



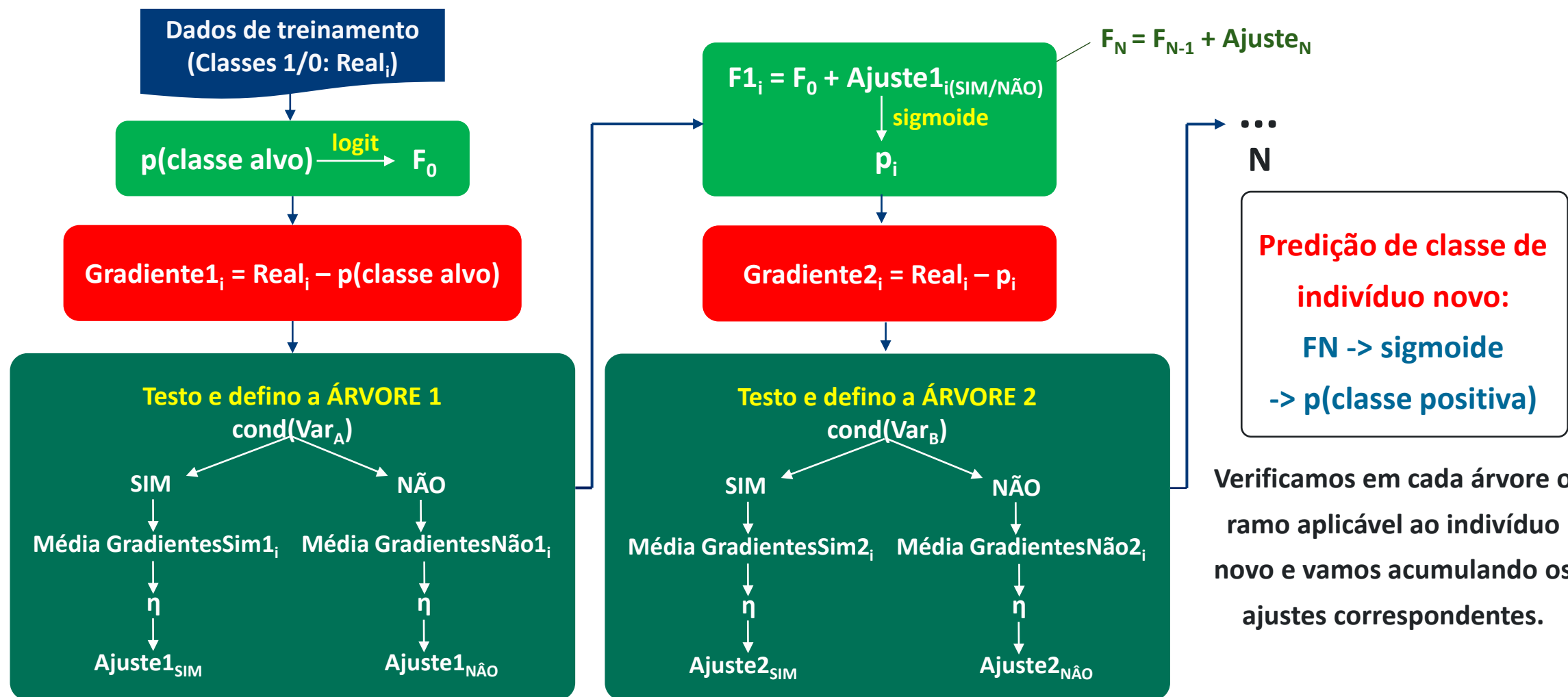
Boosting

- Gradient Boosting – *resumo esquemático do algoritmo (regressão)*:



Boosting

- Gradient Boosting – *resumo esquemático do algoritmo (classificação)*:



Boosting

- Na prática, existem implementações mais avançadas de Gradient Boosting, como XGBoost, LightGBM e CatBoost.
- Elas são muito utilizadas em problemas de dados tabulares e incorporam otimizações, regularização e melhorias de desempenho.
- Usando Random Forest e Gradient Boosting, é válido fazer um **ensemble dos dois modelos**.
- Por exemplo:
 - Predição final = 50% Random Forest + 50% Gradient Boosting e, sobre várias rodadas, pegar as com predição de percentis mais conservadores (mais pessimistas).





Todos os direitos reservados. Textos, vídeos, sons, imagens, gráficos e demais componentes deste material são protegidos por direitos autorais e outros direitos de propriedade intelectual, de forma que é proibida a reprodução no todo ou em parte, sem a devida autorização.

Como citar este material:
RIZZO, Roberto Pina. Modelagem Preditiva. Rio de Janeiro: FGV, 2026.

